

This is an IoT course



Hello!

I am Federico F. Santoro

I am here because I love things that use the Internet to share their information and functionalities.

You can find me at **@fedyfausto** or **federico.santoro@unict.it**



Instructions for use

GOOGLE IS YOUR FRIEND

Before asking for questions, it's a good habit to look for a solution with your search engine (I always do that while looking for the best solution to solve a problem).

STACKOVERFLOW IS NOT ALWAYS THE BEST SOLUTION

Please do not choose the first answer that you find, but try to understand why the solution works.

WORK TOGETHER

Cooperation is vital in this work, it could save you a lot of time sometimes.

PRACTICE, PRACTICE AND PRACTICE

It is a good practice to learn how to use the introduced technologies and to continue practicing outside the course.

DO NOT BE AFRAID TO ASK FOR HELP

WARNING: This is an experimental course, the arguments and exam methods could be change in the middle of time course.

The Course Program



- **Internet of Things Introduction**
  - What is the IoT?
  - IoT Applications
  - The IoT Ecosystem
  - IoT Challenges
- **IoT Components**
  - Sensors and Actuators
  - Microcontrollers
  - Embedded Systems
  - Frameworks
- **IoT Connectivity**
  - Wireless Technologies
  - RFID and NFC
  - LoWPAN Networks and Protocols
  - WLAN Protocols
  - WSAN Protocols
- **IoT Communication**
  - Design of Gateway Solutions
  - HTTP Protocol
  - CoAP Protocol
  - MQTT Protocol
  - WebSockets Protocol
- **IoT Integration**
  - Web Integration
  - Design of complex solution
  - LoRaWAN Introduction
  - Chirpstack Introduction
  - Docker Introduction
- **Machine Learning for IoT**
  - Keras Introduction
  - TensorFlow Lite Introduction
- **IoT Security**

The Course Exam

Written Exam and Design and Develop an iot solution to deal with a problem that will be defined by the professor.

This activity will be carried out in teams.



1

The Internet of Things

Things are connected!

“In God We Trust.
All Others Must Bring Data.”

Edwards Deming, Statistician, Professor

What is the IoT ?



- It is an ecosystem where something has the capabilities to share their information.
- It allows to collect and send information through the network.
- Finally, they can communicate through the Internet!

An IoT ecosystem consists of web-enabled smart devices that uses embedded processors, sensors and communication hardware to collect and send the collected data from their environments.

The Smart Fridge



DO NOT FORGET TO BUY MILK!

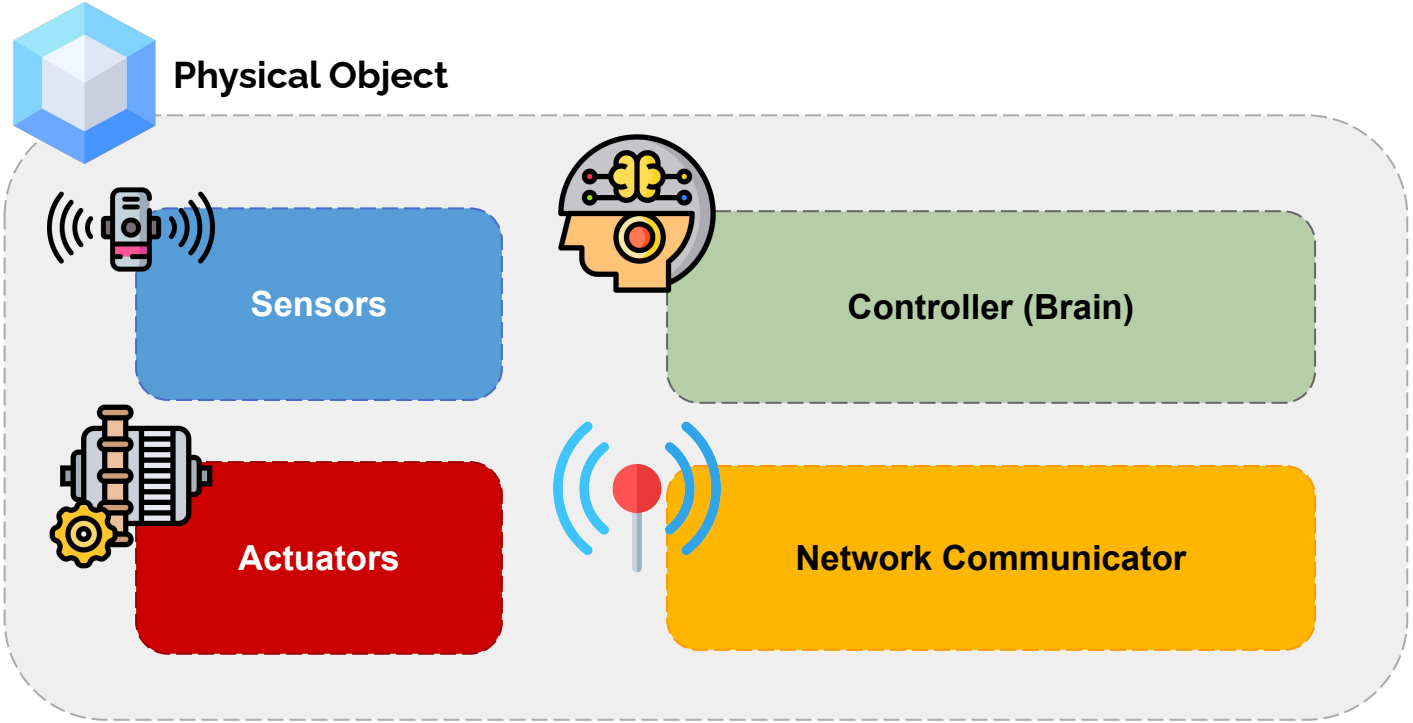
The S. Fridge | How it Works?

- **You are leaving the home (sense user)**
  - What type of sensor?
  - Identify reason for leaving home
  - Identify other context (store is open?)
- **There is no milk in fridge (sense object)**
  - What type of sensor?
  - Is milk needed?
  - No milk or not enough milk? (prediction)

The S. Fridge | How it Works?

- **Use this information to make decision (process)**
  - Where is processor?
  - What are the rules?
  - Fixed rules versus dynamic rules (learning)
- **Inform user of decision (notify)**
  - How?
  - When?
  - Privacy?
  - Information overflow?

The IoT Object



Most commonly used IoT devices (by users)



Smartband



Smartplug



Smartlight



Smartcam



Smartwatch



Domotics devices



Home assistant



Smartphones



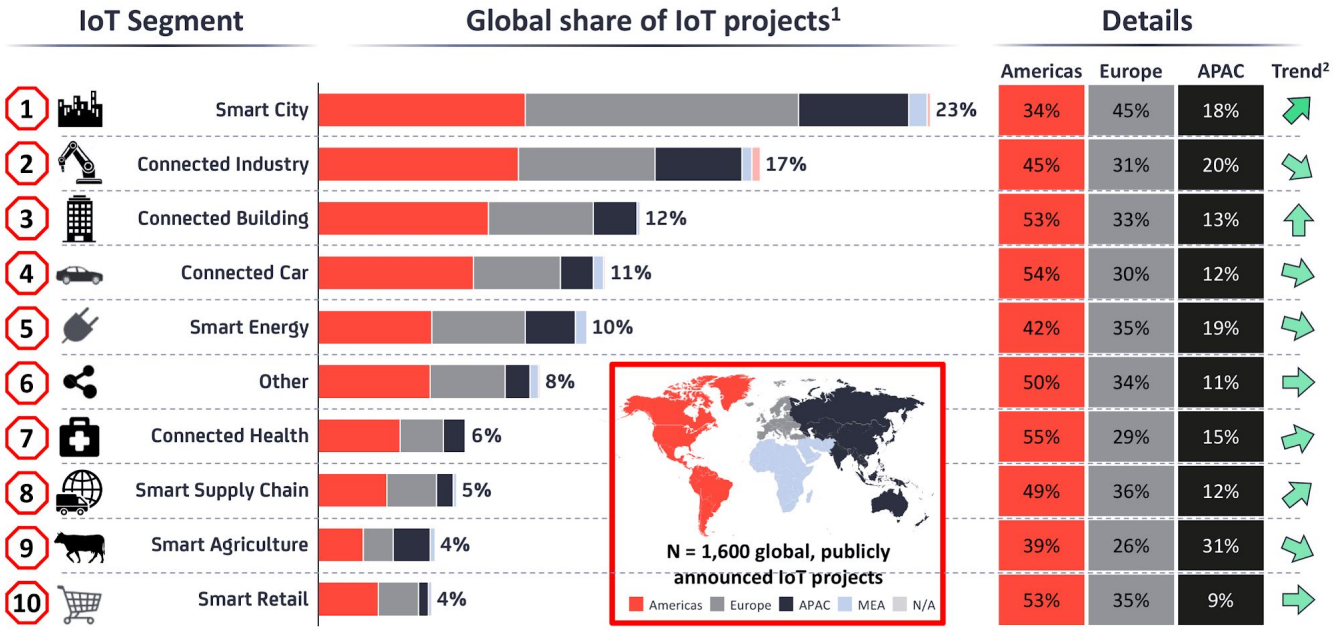
Smart Tvs

IoT application



It's that time to explore some real-world implementations for the Internet of Things!

IoT Application (2018)

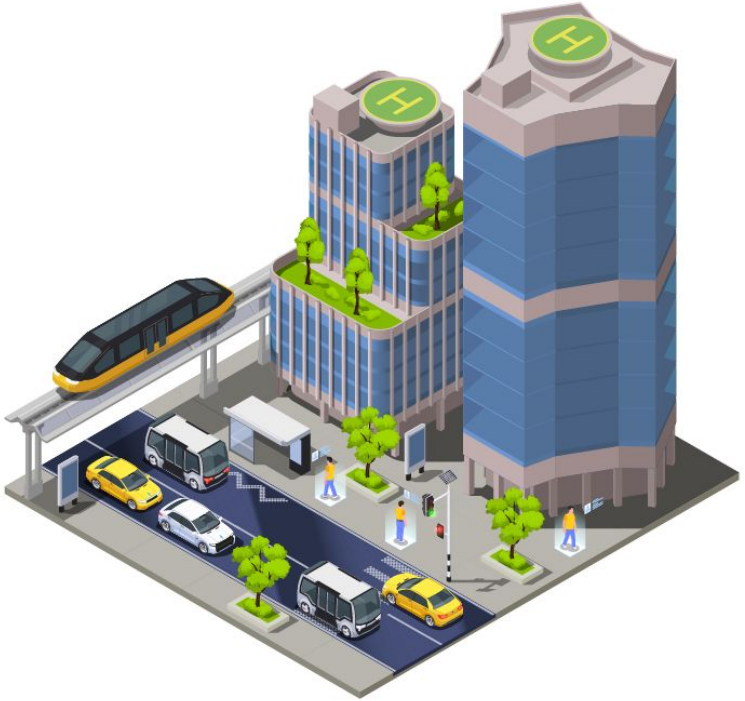


1.Based on 1,600 publicly known enterprise IoT projects (Not including consumer IoT projects e.g., Wearables, Smart Home). 2.Trend based on comparison with % of projects in the 2016 IoT Analytics Enterprise IoT Projects List. A downward arrow means the relative share of all projects has declined, not the overall number of projects 3. Not including Consumer Smart Home Solutions. **Source**: IoT Analytics 2018 Global overview of 1,600 enterprise IoT use cases (Jan 2018)

Source: IoT Analytics, Jan 2018

IoT Application (Nowadays)

- Smart Home
- Smart City
- Smart Car
- Smart Roadways
- Smart Buildings
- Smart Factory
- Smart Creatures
- Smart Grid
- .....



Example: Smart Home



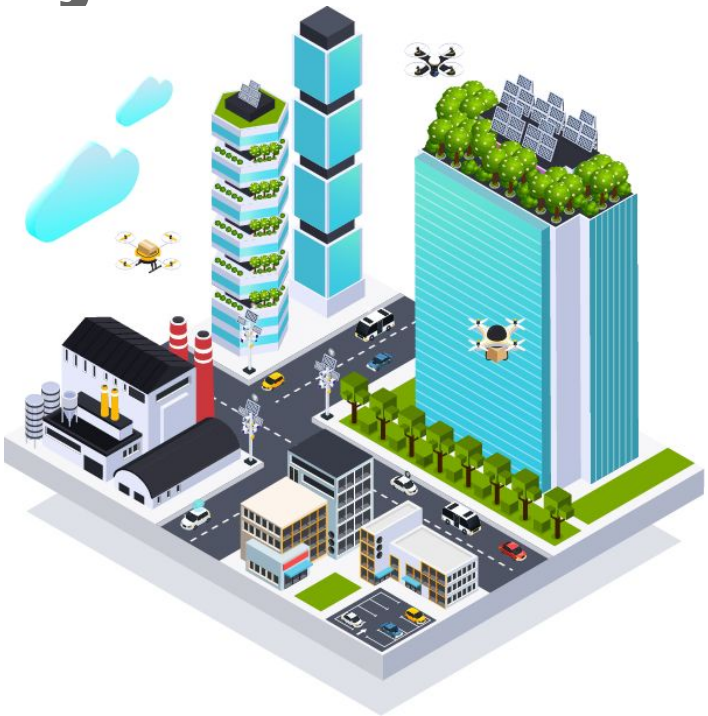
A smart home **uses sensors** to control and maintain lighting, resource management, and security systems. A smart home is a smaller, independent version of a smart city.

For instance, your electricity meter with an IoT device giving you insights into your everyday water usage, your set-top box that allows you to record shows from remote, Automatic Illumination Systems, Advanced Locking Systems, and Connected Surveillance Systems all fit into this concept of smart homes.

Example: Smart City

A smart city is an u**rban city that uses sensors and cellular or wireless technology** placed in ubiquitous places such as lamp posts and antennae.

There are multiple facets in which one can incorporate IoT into the functioning of a city: Traffic management, Pollution monitoring, Resource management, Parking solutions, Infrastructure management and Disaster management.



Example: Smart Car



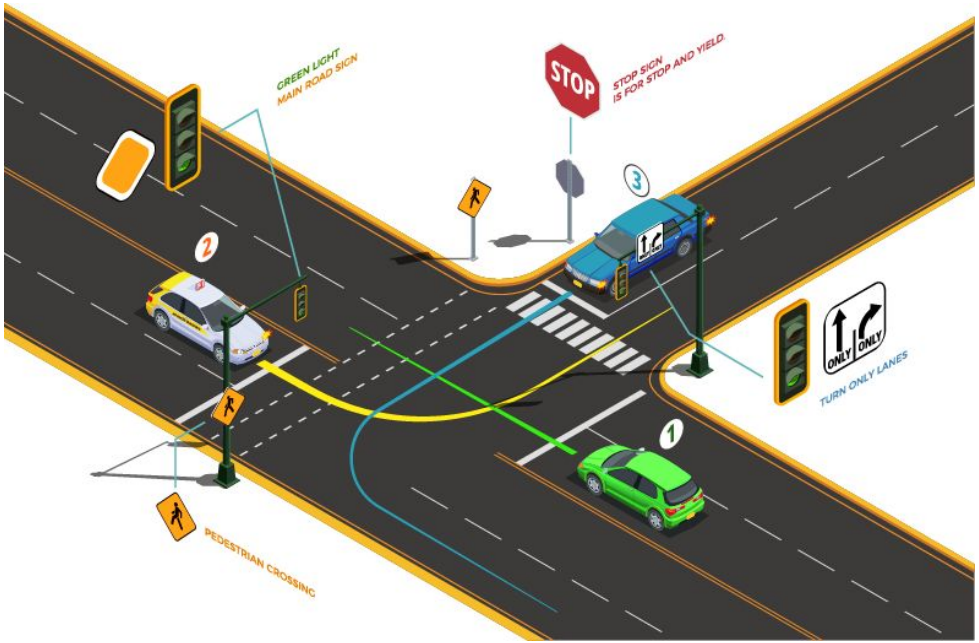
We have seen a lot about self-driven cars. Since it is human lives on the roads that we're dealing with, **we need to ensure the technology has all that it takes to ensure better safety** for the passenger and those on the roads.

The cars use several sensors and embedded systems connected to the Cloud and the internet to keep generating data and sending them to the Cloud for informed decision-making through Machine Learning.

Example: Smart Roadway

Smart roads include the **actual roads themselves, smart street lights, smart traffic signs, and smart or autonomous cars driving on these roads**. The architectures, systems and applications that make smart roads possible include communications networks, IoT sensor networks, as well as big data and artificial intelligence applications.

Smart road systems that are deployed in Smart Cities require extensive sensor networks that create huge volumes of data on traffic flow and public transit systems.



Example: Smart Healthcare



The pandemic has accelerated the use of IoT in the healthcare industry. **The Internet of Things provides a much more efficient link between patients, doctors, and pharmaceuticals**. The traditionally reactive medical cycles can now be proactive.

Attaching sensors to a patient at home allows doctors to monitor them remotely. Besides monitoring, the Internet of Things can also streamline patient medical records and access, making real-time data available across departments.

Example: Smart Agriculture

Agriculture, as an industry, could massively benefit from the Internet of Things. This, combined with climate change, has farmers marrying technology to cultivation.

Sensors are used to provide details of soil chemistry and fertilizer profiles, Smart irrigation is an IoT application to regulate and efficiently use water for farming and Predication farming is the practice of using information collected over time to hone and implement agricultural practices.



Example: Smart Factory



Industrial IoT (IIoT) is the Internet of Things at a factory level. IIoT is often referred to as the fourth wave of the industrial revolution or **Industry 4.0**.

Most factories subscribe to enterprise asset management and computerized maintenance management systems for maintenance management. Real-time monitoring of devices ensures predictive care and optimal machine health. IIoT goes beyond supply chain management and maintenance management. Aggregated data assists in the optimization of product development and quality testing.

Example: Smart Grid

Utility companies are turning to IoT to make energy provision more efficient. Appropriate sensors are installed in energy meters, transmission lines, production plants, and distribution points. This IoT system is called a **smart grid**.

They create alerts in case of failure at any point during power transmission, Sensors are used to identify abnormalities in the line and monitor energy consumption and peak usage statistics.

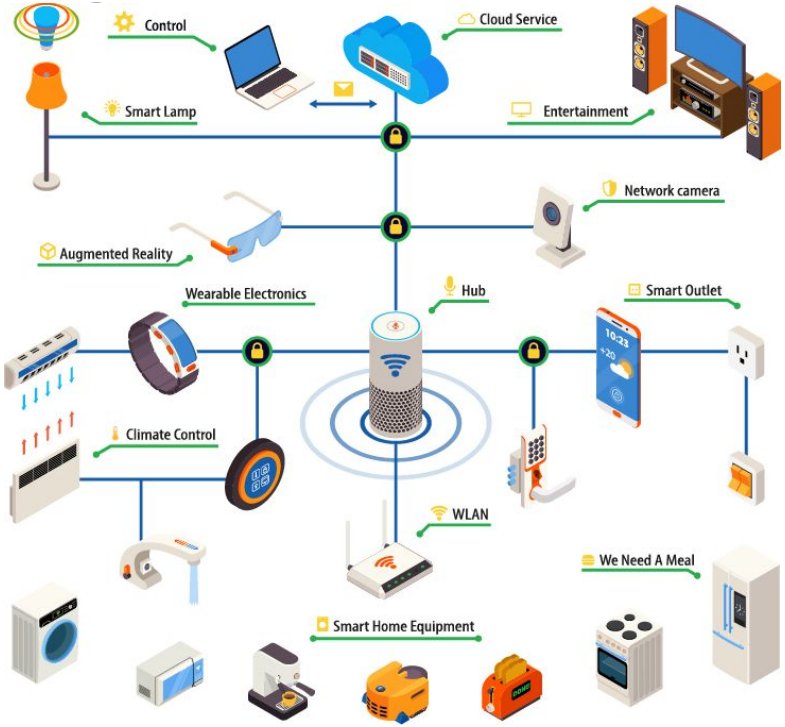


IoT Ecosystem



How things collaborate with each others and communicate with the external world!

The IoT Architecture



- **IoT End Devices**
  Typically with a LoWPAN Network
- **Gateways**
  This can be an IoT device too
- **UI Applications**
  Mobile App or Web App
- **Cloud Services**
  AWS, IoT Azure, Chirpstack
- **Back-End Systems**
  Web Servers, Databases

The IoT Architecture Examples



BLE Communication



Data Communication



Xiaomi Mi Band X
Collects data from its sensors and then sends these information to the mobile device

Smartphone
Collects data from the Mi Band and then show these information on the Xiaomi Application. Finally, it sends the information to the Xiaomi Cloud Servers. (It acts as IoT Hub)

Xiaomi Cloud Server
Collects data from the users and then uses these information to improve the quality of services, offers a backup service and for business purpose

The IoT Architecture Examples



Zigbee Communication



WiFi Communication



Xiaomi Aqua Sensor
Checks if a door/window is open or not and sends this data

Xiaomi Home Hub
Collects data from other IoT Zigbee devices and then sends these information through the Internet to the Xiaomi servers

Router Gateway
It permits Home Hub to connect to the Internet

Internet



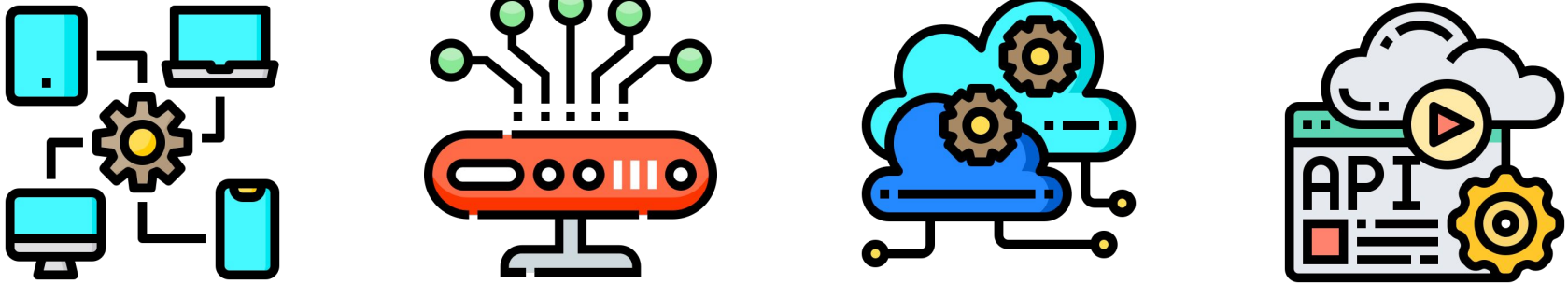
Internet



Xiaomi Cloud Server
Collects the sensors data and then sends it to the Smartphone application via Internet

Xiaomi Home Application
Receives the notification about the sensor and shows it to the user

The IoT Architecture



Devices

Gateways

Service Layer

Application Layer

LoWPAN Networks
(Zigbee, LoRa, SigFox)

IP Networks
(Ethernet, WiFi, Cellular, Fiber)

Message Protocols
(HTTP, MQTT, CoAP)

The IoT Solutions Architecture



Sensors and Actuators

Wired, Wireless



Internet Gateways

Data acquisition, Data Aggregation, Measurement, Control



Edge IT

Analytics, Pre-Processing



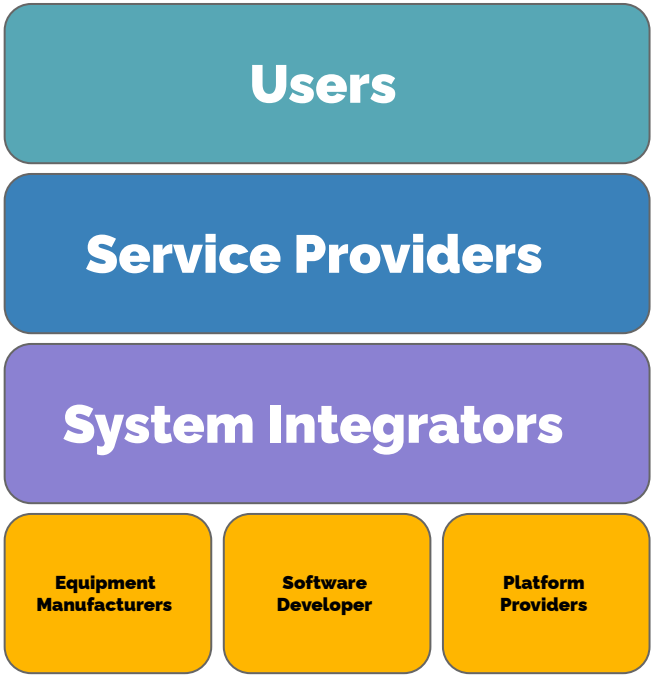
Data Center, Cloud

Analytics, Management

The Edge

The IoT Ecosystem Players

- **Equipment Manufacturers**
  Produce devices and networking/communication equipment.
- **Software Developers**
  Produce software instruments for the collection, storage and management of data.
- **Platform Providers**
  Deploy and manage data servers that collect, distribute, maintain and process data.
- **System Integrators**
  Put together devices, software and data servers.
- **Service Providers**
  Run the IoT system and manage the service it offers.
- **Users**
  Final users, companies and institutions.



The IoT Challenges

- **Network Challenges**
  - Scalability
  - Diversity
  - Open Network
  - Low Power
  - Coverage
  - Bandwidth
- **Hardware Challenges**
  - Integration
  - Compatibility
  - Testing
  - Safety
  - Cost
  - Energy and Power
- **Software Challenges**
  - Data
  - Self Configuration
  - Networking
  - Updates
  - Power and Memory
  - Low Level Programming
- **Security Challenges**
  - Unsecure Devices
  - Network Security
  - Suspicious Behavior
  - Cryptography
  - Privacy

2

IoT Components

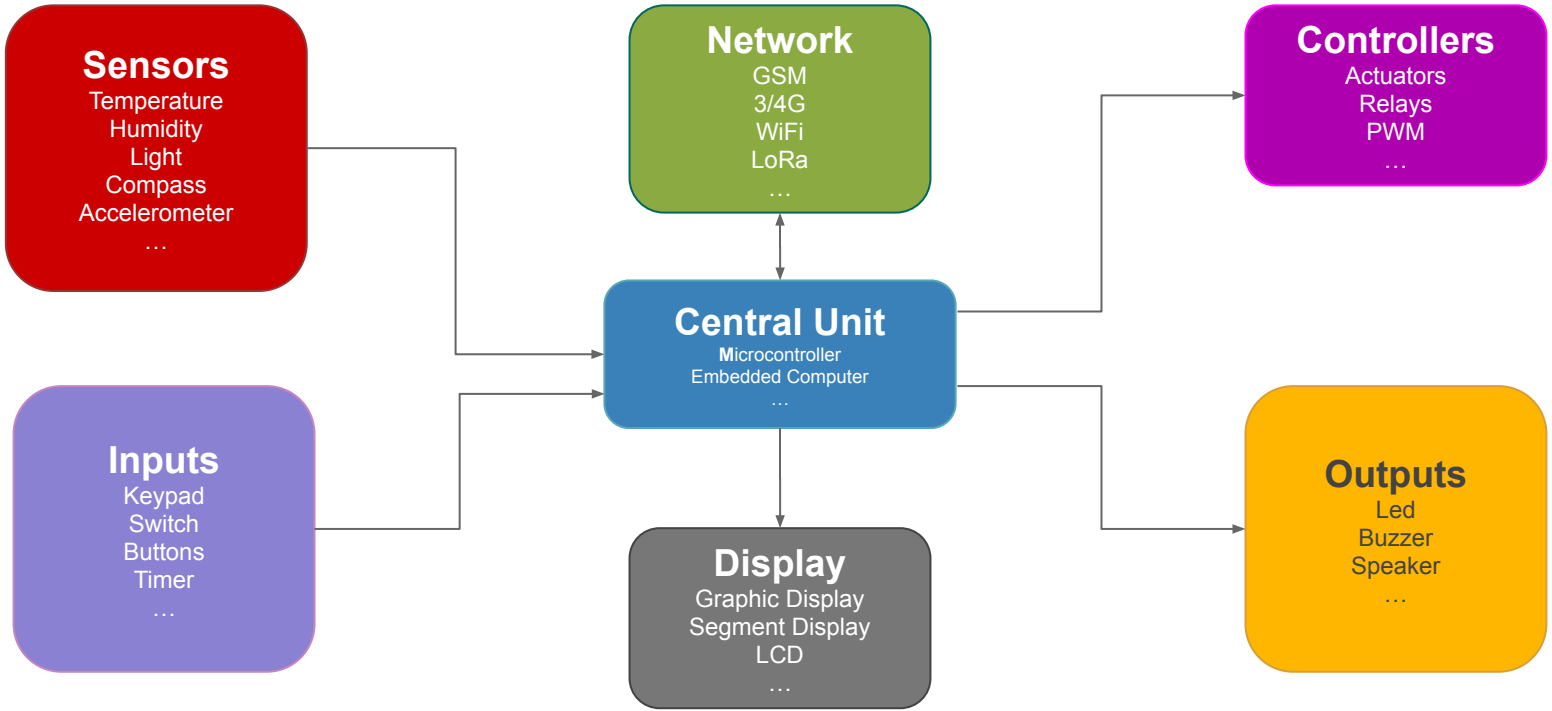
Mini computers! Yeeeeeee

There are a lot of components For the IoT!

Nowadays there are various types of sensors! IoT devices can bring a lot of information surround us and from the environment to let them to operate correctly and inform us.



The IoT Components



The IoT Sensors



A **sensor** is a device that is able to detect changes in an environment. By itself, a sensor is useless, but when we use it in an electronic system, it plays a key role.

A sensor is able to measure a physical phenomenon (like temperature, pressure, and so on) and transform it into an electric signal.

These three features should be at the base of a good sensor:

- It should **be sensitive to the phenomenon** that it measures
- It should **not be sensitive to other physical phenomena**
- It should **not modify the measured phenomenon** during the measurement process

The IoT Sensors



There is a wide range of sensors we can exploit to measure almost all the physical properties around us. A few common sensors that are widely adopted in everyday life include thermometers, pressure sensors, light sensors, accelerometers, gyroscopes, motion sensors, gas sensors and many more.

A sensor can be described using several properties, the most important being:

- **Range**: The maximum and minimum values of the phenomenon that the sensor can measure.
- **Sensitivity**: The minimum change of the measured parameter that causes a detectable change in output signal.
- **Resolution**: The minimum change in the phenomenon that the sensor can detect.

The IoT Sensors

Sensors can be grouped using several criteria:

- **Passive or Active**. Passive sensors do not require an external power source to monitor an environment, while Active sensors require such a source in order to work.
- Another classification **is based on the method used to detect and measure the property** (mechanical, chemical, etc.).
- **Analog and Digital**. Analog sensors produce an analog, or continuous, signal while digital sensors produce a discrete signal.

The IoT Sensors



The common IoT sensors that will be employed include:

- Temperature sensors
- Pressure sensors
- Motion sensors
- Level sensors
- Image sensors
- Proximity sensors
- Water quality sensors
- Chemical sensors
- Gas sensors
- Smoke sensors
- Infrared (IR) sensors
- Acceleration sensors
- Gyroscopic sensors
- Humidity sensors
- Optical sensors

To use these, we need an electronic system or a **controller**!

The IoT Microcontrollers



A **microcontroller (MCU)** is an integrated circuit (or IC) that **can be programmed** to perform a set of functions to control a collection of electronic devices.

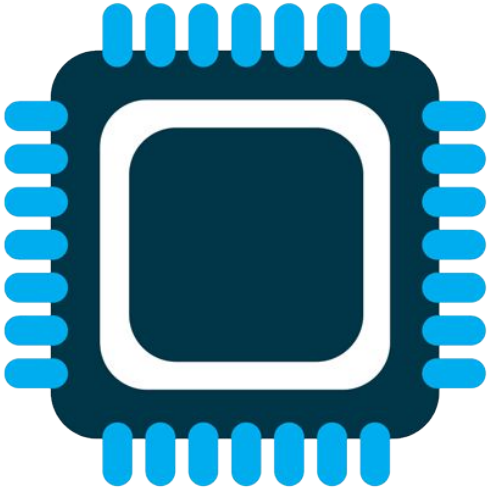
A microcontroller can be called small computer because on a single integrated circuit contains a processors core, a memory and programmable input and output peripherals.

A **microprocessor** incorporates the functions of a computer's central processing unit (CPU) on a single integrated circuit.

The IoT MCUs: How choose

There are a lot of metrics that must be considered:

- **Power Consumption**
- Clock Frequency
- **I/O Pins**
- **Memory**
- **Processor**
- Integrated Networks Interfaces
- Integrated Sensors
- Internal Functions
- **Architecture**
- **Size**
- Others..



The IoT MCUs: Components



- **Processor**
  - Function
    - Fetch and execute machine code from memory
    - Efficiency is determined by its architecture
  - Architecture Type
    - Bus: 4, 8, 16, 32 Bit
    - Contains ALU
    - General Registers
    - Special Registers
    - Instruction Set
    - Frequency
    - Volgate

The IoT MCUs: Components



- **Clock**
  - MCU needs a clock input to operate
  - It can have two clock source
    - Main processor clock input
      This clock may be divided or multiply to allow the processor to run at a slower or higher speed
    - Sub clock input
      Usually used to generate “second” via a 16 bit counter

The IoT MCUs: Components



- **Memory**
  - ROM
    - Program and fixed data storage
    - Size: from 1 KB to some MB
    - Example
      Masked ROM
      Flash
      PROM
  - RAM
    - For data storage
    - Size from 256 Byte to 1 MB

The IoT MCUs: Components



- **Peripheral**
  - ADC
    - Interface between MCU and analogue sensors
    - Converts Analogue signal to Digital Signal
  - DAC
    - Interface between MCU and analogue actuators
    - Converts Digital signal to Analogue Signal
  - I/O Ports
    - Interface between MCU and I/O peripheral
    - Modules, keyboard, display, buttons, etc.
    - Timers/Counters

The IoT MCUs: Components



- **Watchdog Timer**
  - A specialised program found as part of the microcontroller designer to prevent the microcontroller from halting or "locking up" because of a user-written program, since the instructions are processed step-by-step
  - Uses a routine that is based on timing. If a program has not been completed or repeated as a loop within a certain amount of time, the watchdog time issues a reset command
  - **A system reset sets all the register values to zero**
  - The reset feature allows the controller to recover from the crash.

The IoT MCUs: Components

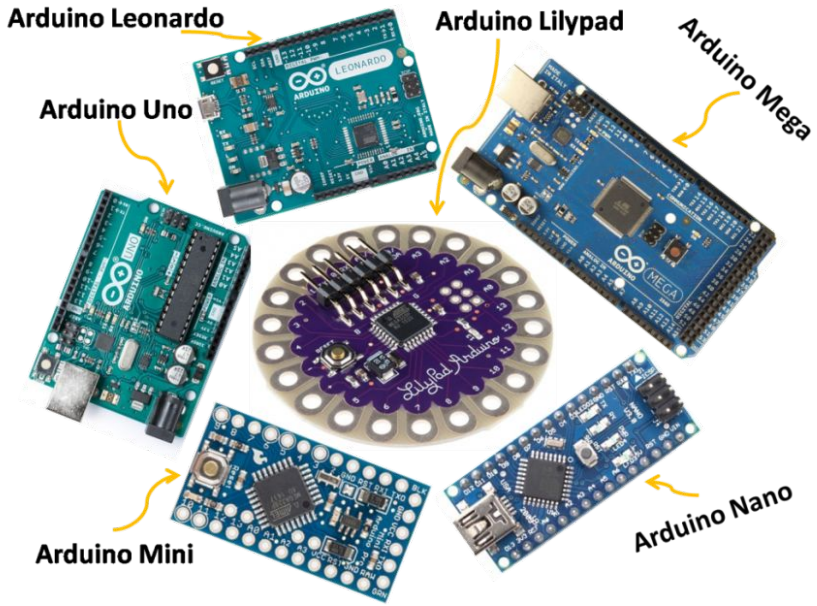


- **Serial Interface**
  - There are different serial interfaces
    - Asynchronous mode (2400 Baud Rate…)
    - Synchronous mode (at higher transfer rate)
    - USB, I2C, SPI, CAN, UART, etc. (communication protocols)
  - Used for:
    - Communication with other devices, such as external RF module, PC , etc..
  - **Generally used only 3 pins**
    - Serial In, Serial Out and Clock

The IoT MCUs : Arduino

Arduino is an open source project that create microcontroller kits for building digital devices and interactive objects that can sense and control physical devices.

These systems provide sets of digital and analog I/O pins that can interface to various expansion boards (called shields) and other circuits.



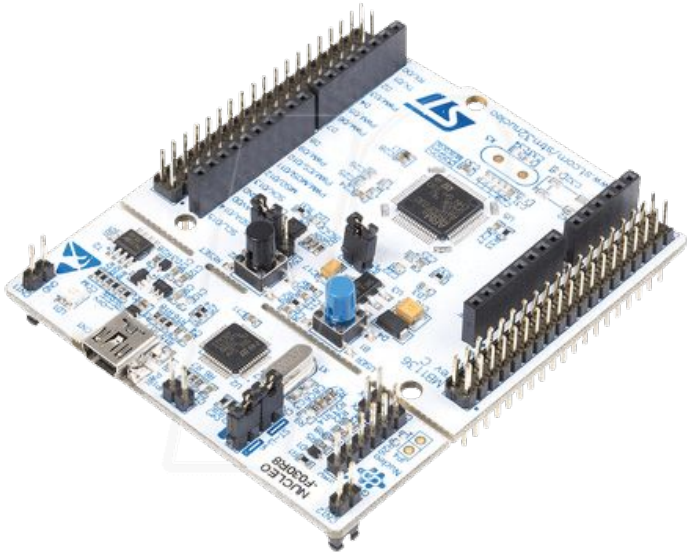
The IoT MCUs : Arduino (UNO)

- **Microcontroller:** ATmega328P
- **Voltage:** 3,3V to 12V
- **Digital I/O pins:** 14 (of which 6 provide PWM output)
- **Analog Input pins:** 6
- **Flash Memory:** 32 KB of which 0.5 KB used by bootloader
- **RAM:** 2 KB
- **EEPROM:** 1 KB
- **Clock Speed:** 16 MHz
- **Built-in:** One LED
- **Power Supply:** DC Jack and USB



The IoT MCUs : STM32

- **Microcontroller:** ARM Cortex
- **Voltage:** 3,3V to 12V
- **Digital I/O pins:** 50 (also Arduino pins)
- **Analog Input pins:** 16 (also Arduino pins)
- **Flash Memory:** 512 KB
- **RAM:** 96 KB
- **Clock Speed:** 84 MHz
- **Built-in:** One LED shared with Arduino, Reset Button and User Button
- **Power Supply:** ST-Link USB or External



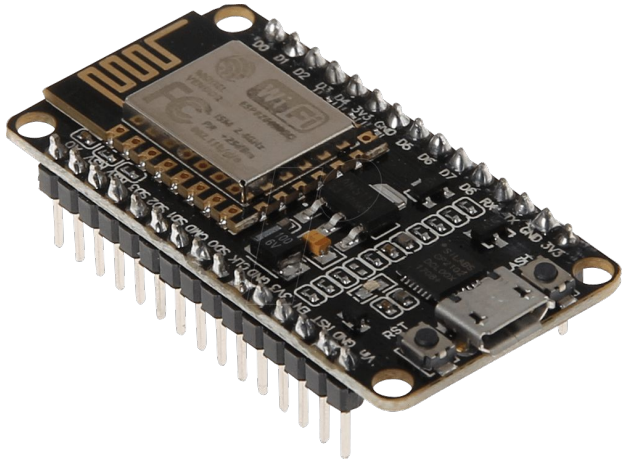
The IoT MCUs : Raspberry Pico

- **Microcontroller:** Dual Core ARM Cortex
- **Voltage:** 1,8V to 5,5V
- **Digital I/O pins:** 23
- **Analog Input pins:** 3
- **Flash Memory:** 2 MB
- **RAM:** 264 KB
- **Clock Speed:** 133 MHz
- **Built-in:** 2.4 GHz Wi-Fi 802.11, Temperature Sensor
- **Power Supply:** USB or External



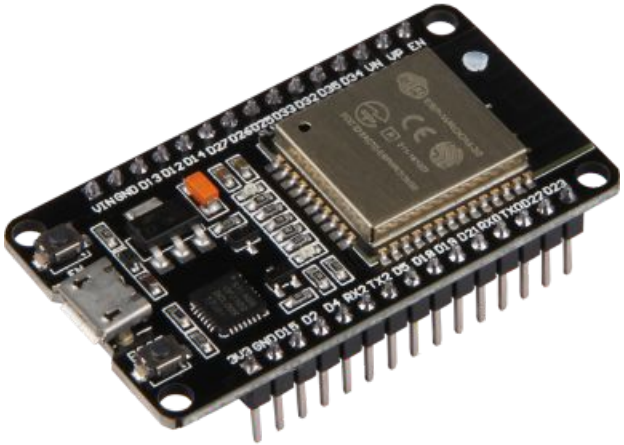
The IoT MCUs : ESP8266

- **Microcontroller:** Tensilica 32-bit RISC CPU Xtensa LX106
- **Voltage:** 3,3V to 10V
- **Digital I/O pins:** 16
- **Analog Input pins:** 1
- **Flash Memory:** 4 MB
- **RAM:** 64 KB
- **Clock Speed:** 160 MHz
- **Built-in:** 2.4 GHz Wi-Fi 802.11, one LED, reset button, flash/user button, TCP Stack
- **Power Supply:** USB or External



The IoT MCUs : ESP32

- **Microcontroller:**
  - Dual Core Tensilica Xtensa LX6 microprocessor
  - Ultra low power (ULP) co-processor
- **Voltage:** 3,3V to 10V
- **Digital I/O pins:** 34
- **Analog Input pins:** 4
- **Flash Memory:** 4 - 16 MB
- **RAM:** 512 KB and a pseudoRAM about 4-8 MB
- **Clock Speed:** 240 MHz
- **Built-in:** 2.4 GHz Wi-Fi 802.11, Bluetooth 4, BLE, TCP Stack, hall and touch sensor
- **Power Supply:** USB or External



The IoT Single Boards



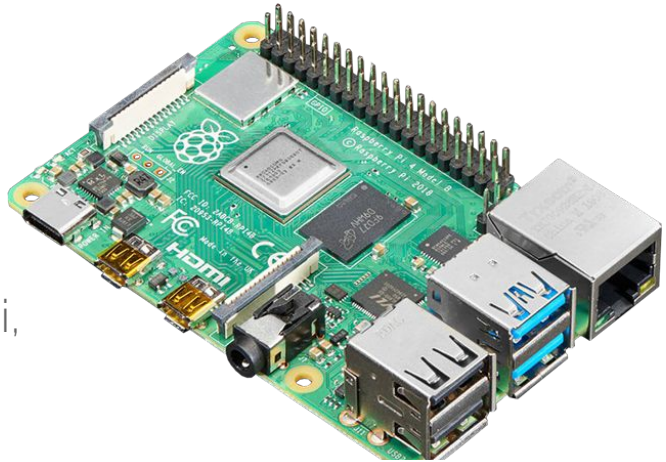
A **single-board computer** (**SBC**) is a complete computer built on a single circuit board, with microprocessor(s), memory, input/output (I/O) and other features required of a functional computer.

Single-board computers are commonly made as demonstration or development systems, for educational systems, or for use as embedded computer controllers.

Unlike a desktop personal computer, single board computers often do not rely on expansion slots for peripheral functions or expansion. Single board computers have been built using a wide range of microprocessors. Simple designs, such as those built by computer hobbyists, often use static RAM and low-cost 32- or 64-bit processors like ARM.

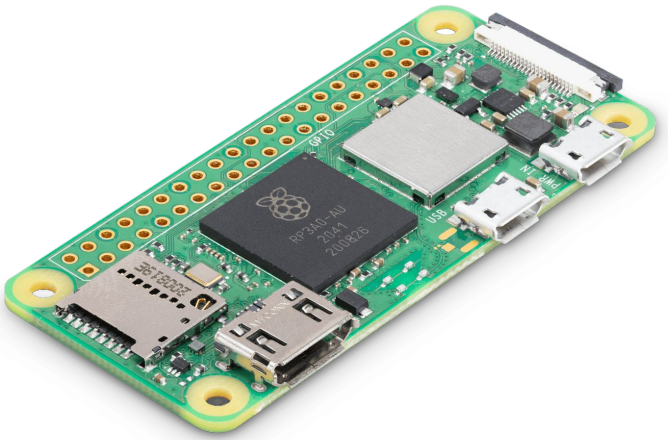
The IoT SBC: Raspberry Pi

- **CPU:** Quad core Cortex-A72 (ARM v8) 64-bit
- **Voltage:** 5V, 3A
- **Digital I/O pins:** 40
- **Storage:** Micro-SD card slot for loading operating system and data storage
- **RAM:** 1 GB to 8 GB
- **Clock Speed:** 1.5GHz
- **Built-in:** 2.4 GHz and 5.0 GHz 802.11ac Wi-Fi, Bluetooth 5.0, BLE, Gigabit Ethernet, 2 USB 3.0 ports; 2 USB 2.0 ports
- **Power Supply:** USB or External
- **OS**: Typically Linux, Android and Windows IoT Compatible



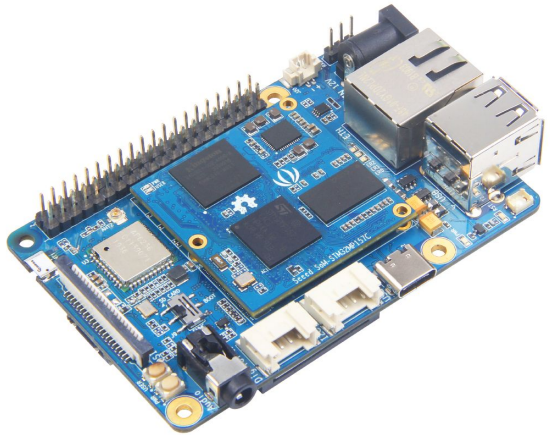
The IoT SBC: Rasp Pi Zero

- **CPU:** Quad core Cortex-A53 (ARMv7) 64-bit
- **Voltage:** 5V, 2.5A
- **Digital I/O pins:** 40
- **Storage:** Micro-SD card slot for loading operating system and data storage
- **RAM:** 512 MB
- **Clock Speed:** 1 GHz
- **Built-in:** 2.4 GHz 802.11b/g/n Wi-Fi, Bluetooth 4.0, BLE
- **Power Supply:** USB or External
- **OS**: Typically Linux



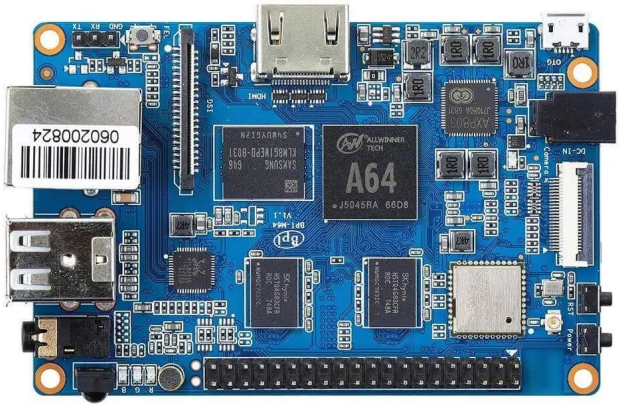
The IoT SBC: STM32 ODYSSEY

- **CPU:** Dual-core Arm Cortex-A7 (ARMv7) 32-bit
- **Voltage:** 12V, 2.5A
- **Digital I/O pins:** 40
- **Storage:** Micro-SD card slot for loading operating system and data storage and 4GB EMMC
- **RAM:** 512 MB
- **Clock Speed:** 650MHz
- **Built-in:** 2.4 GHz 802.11b/g/n Wi-Fi, Bluetooth 4.0, BLE
- **Power Supply:** Jack or USB-C
- **OS**: Typically Linux



The IoT SBC: Banana Pi

- **CPU:** Quad-core ARM Cortex A53 64-Bit
- **Voltage:** 5-12V, 2.5A
- **Digital I/O pins:** 40
- **Storage:** Micro-SD card slot for loading operating system and data storage and 8GB EMMC
- **RAM:** 2 GB
- **Clock Speed:** 1.2 GHz
- **Built-in:** 2.4 GHz 802.11b/g/n Wi-Fi, Bluetooth 4.0, BLE
- **Power Supply:** Jack or USB-C
- **OS**: Typically Linux



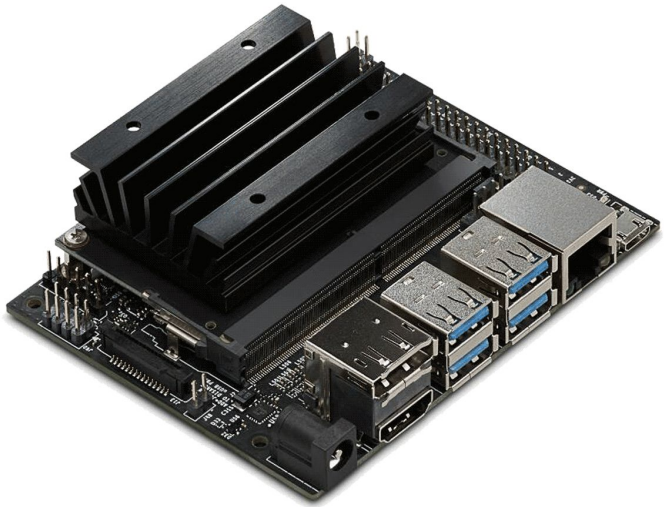
The IoT SBC: Orange Pi

- **CPU:** Rockchip RK3399 6-core ARM 64-bit
- **Voltage:** 5V, 3A
- **Digital I/O pins:** 40
- **Storage:** 16GB EMMC
- **RAM:** 4 GB
- **Clock Speed:** 2 GHz
- **Built-in:** 2.4 GHz 802.11b/g/n Wi-Fi, Bluetooth 4.0, BLE
- **Power Supply:** Jack or USB-C
- **OS**: Typically Linux



The IoT SBC: NVIDIA Jetson

- **CPU / GPU:** 128-core NVIDIA Maxwell™ GPU, Quad-core ARM® A57 CPU
- **Voltage:** 5V, 4A
- **Digital I/O pins:** 56
- **Storage:** SD Slot
- **RAM:** 4 GB
- **Clock Speed:** 2 GHz
- **Built-in:** Ethernet
- **Power Supply:** Jack or USB-C
- **OS**: Typically Linux



IoT Frameworks



How things could be programmed!

The IoT Frameworks/API



A framework is a collection (tools, libraries, etc.) that is meant to simplify software development and helps to provide a common look and feel for programs or operating systems.

There are now multiple frameworks and libraries to simplify development on microcontrollers to create a **Firmware**.

Clearly it is necessary that this framework supports the microcontroller you want to use.

The IoT Fr: Arduino API



The **Arduino API**, aka the "Arduino Programming Language", consists of several functions, variables and structures based on the C/C++ language.

Due to their simplicity, the programs you write using the Arduino API are called **sketches**. In their essence, they are text files written in Arduino language. To save and upload them to your Arduino board, you will need to use the **.ino** extension.

There are two functions that every sketch written in Arduino language contains. Those are **setup()** and **loop()**.

The IoT Fr: Arduino API



The Arduino API can be divided into three main parts: functions, variables and structure:

- **Functions**
  For controlling the Arduino board and performing computations.
- **Variables**
  The Arduino constants, data types and conversions.
- **Structure**
  The elements of the Arduino (C++) code

The Arduino API can be described as a simplification of the C++ programming language, with a lot of additions for controlling the Arduino hardware.

The IoT Fr: Arduino API



Program Structure

The absolute minimum requirement of an Arduino program is the use of two functions: `void setup()` and `void loop()`. The "`void`" indicates that nothing is returned on execution.

- **`void setup()`**
  This function executes only once, when the Arduino is powered on. Here we define things such as the mode of a pin (input or output), the baud rate of serial communication or the initialization of a library.
- **`void loop()`**
  This is where we write the code that we want to execute over and over again, such as turning on/off a lamp based on an input, or to conduct a sensor reading every X second.

The IoT Fr: Arduino API



In the Arduino project, a program is referred to as a "**sketch**". A sketch is a file that you write your program inside. It has typically the **.ino** extension, and is always stored in a folder of the same name.

The folder can include other files, such as a **header** file, that can be included in your sketch.

```
int sensorPin = A1; //define pin A1 (analog pin)
int ledPin = 2; //define pin 2 (digital pin)
int sensorValue; //create variable for storing readings

//void setup is for configurations on start up
void setup() {
    Serial.begin(9600); //initialize serial communication
    pinMode(ledPin, OUTPUT); //define ledPin as an output
}
```

```
void loop() {
    sensorValue = analogRead(sensorPin); // do a sensor reading

    Serial.print("Sensor value is: "); //print a message to the serial
monitor
    Serial.println(sensorValue); //print the value to the serial monitor

    //check if sensorValue is below 200
    if(sensorValue < 200) {
        digitalWrite(ledPin, HIGH); //if it is, turn on the LED on pin 2.
    }
    //if sensorValue is above 200, turn off the LED
    else{
        digitalWrite(ledPin, LOW);
    }
}
```

The IoT Fr: Arduino API



Arduino libraries are an extension of the standard Arduino API, and consists of thousands of libraries, both official and contributed by the community.

Libraries simplifies the use of otherwise complex code, such as reading a specific sensor, controlling a motor or connecting to the Internet.

Instead of having to write all of this code yourself, you can just install a library, include it at the top of your code, and use any of the available functionalities of it.

All Arduino libraries are open source and free to use by **anyone**.

```
#include <Library.h>
```

The Platform IO



PlatformIO is a cross-platform, cross-architecture, multiple framework, professional tool for embedded systems engineers and for software developers who write applications for embedded products.

With PlatformIO, users can be hobbyists or professionals. They can import the classic Arduino “Blink” sketch or develop a sophisticated low-level embedded C program for a commercial product. Example code for any supported framework can be compiled and uploaded to a target platform in minutes.

The Platform IO - Install



https://platformio.org/install

https://docs.platformio.org/en/latest/integration/ide/vscode.html#ide-vscode

The IoT Board - Install

Arduino Nano ESP32

ESP32-S3 dual core 240 MHz, Wi-Fi 4 + BLE 5, 16 MB flash, 8 MB PSRAM.

OLED display

SSD1306 128×64 on I^2C (A4/A5): the `display` pointer, Adafruit GFX API.

Buttons and LEDs

BTN1 D7, BTN2 D9; red A2, yellow A1, green A0 LEDs; RGB LED.

802.15.4 radio

MRF24J40MA on HSPI (CS D6): the ZigBee chapter.

LoRa radio

RFM95 (SX1276) at 866 MHz on HSPI (CS D10): LoRa and LoRaWAN.

Software

PlatformIO + **IoTBoard-Library**; start from IoTBoard-EmptyProject.

github.com/UniCT-Internet-of-Things/IoTBoard-Library
github.com/UniCT-Internet-of-Things/IoTBoard-EmptyProject

3

IoT Connectivity

Ping Pong!

There are a lot of connectivity ways for IoT!

Nowadays there are various ways to connect with the IoT devices that surround us and to let them connect with the outside or with other devices!



IoT Communication

	Standard	Range	Bandwidth
Bluetooth	**Bluetooth >=3**	**10 - 50 m**	**1 Mbps**
Bluetooth Low Energy	**Bluetooth >= 4.x**	**10 - 100 m**	**1 Mbps**
Zigbee	**IEEE 802.15.4**	**10 - 100 m**	**250 kbps**
6LowPAN	**RFC6282 & IEEE 802.15.4**	**10 - 100 m**	**250 kbps**
Thread	**6LowPAN & IEEE 802.15.4**	**10 - 100 m**	**250 kbps**
Wi-Fi	**IEEE 802.11**	**~60 m**	**54 - 1200 Mbps**
Cellular	**UMTS/HSPA (3G), LTE (4G)**	**~200 Km**	**3 - 50 Mbps**
LoRa	**LoRaWAN & IEEE 802.15.4**	**15 Km**	**50 kbps**
NFC	**ISO/IEC 18000-3**	**6 cm**	**100 – 420kbps**

The Wireless Networks

In the IoT field we can define three types and categories of Wireless Networks:

- **Wireless Local Area Network** (WLAN)
- **Wireless Sensors Area Network** (WSAN)
- **Low Power Wireless Personal Area Network** (LPWAN)

These differ in network bandwidth, coverage range and power consumption.

Each of these categories has its advantages and disadvantages, also considering the challenges of operation and security.

One of the firstly Wireless protocol for the IoT was the **RFID**!

The RFID Technology

RFID (Radio Frequency IDentification) lets a reader **identify and read data from a tag through radio waves**: no contact and no line of sight, unlike barcodes and QR codes.



Tag

Transponder: microchip + antenna. Stores an ID and maybe data. Usually no battery.



Reader

Interrogator: generates the RF field, powers and queries the tags, decodes replies.



Middleware

Filters duplicates, handles many readers, turns raw reads into events.



Application

Access control, inventory, payments, IoT cloud backend.

Key idea: a passive tag has no battery and no transmitter. The reader provides energy, clock and the communication channel.

The RFID Technology

1

Power up

The reader emits a continuous carrier. The tag antenna harvests energy (rectifier + capacitor) and the chip wakes up.

2

Command

The reader modulates the carrier with short pauses (ASK) to send commands: **Reader Talks First**.

3

Reply

The tag **never transmits**: it changes its own load (HF) or how much signal it reflects (UHF backscatter).

4

Decode

The reader detects these tiny variations, demodulates, checks the CRC and passes the data to the host.

Because the tag only answers when asked, it needs **no oscillator, no transmitter and no battery**: that is why a passive tag costs a few cents.

The RFID Technology





Passive

- Energy harvested from the reader field
- Unlimited life, a few cents each
- Range: cm (LF/HF) up to ~10 m (UHF)
- Badges, NFC stickers, retail labels



Semi-passive (BAP)

- Battery-Assisted Passive: the battery powers **chip and sensors** only
- Still answers with backscatter: no transmitter
- Better sensitivity, longer range
- Cold-chain temperature loggers



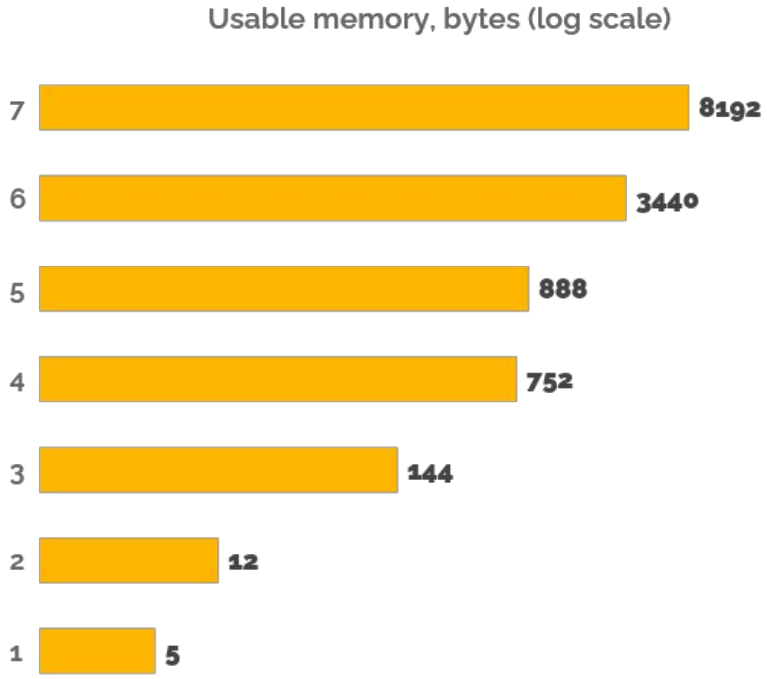
Active

- Battery + **its own radio transmitter**
- Beacons periodically or on request
- Range 100 m and more (433 MHz, 2.45 GHz)
- Containers, vehicles, RTLS

Cost, size and range grow from left to right. The "200 m" figures you find online are for **active** tags only.

The RFID Technology

- **Read-only**: ID written at the factory, e.g. EM4100 (40-bit ID).
- **WORM**: Write Once, Read Many, then locked.
- **Read/Write**: EEPROM, ~100k write cycles, ~10 years retention, often protected by keys or passwords.
- Every tag has a **UID**: it is the anti-collision handle, **not** a security credential.

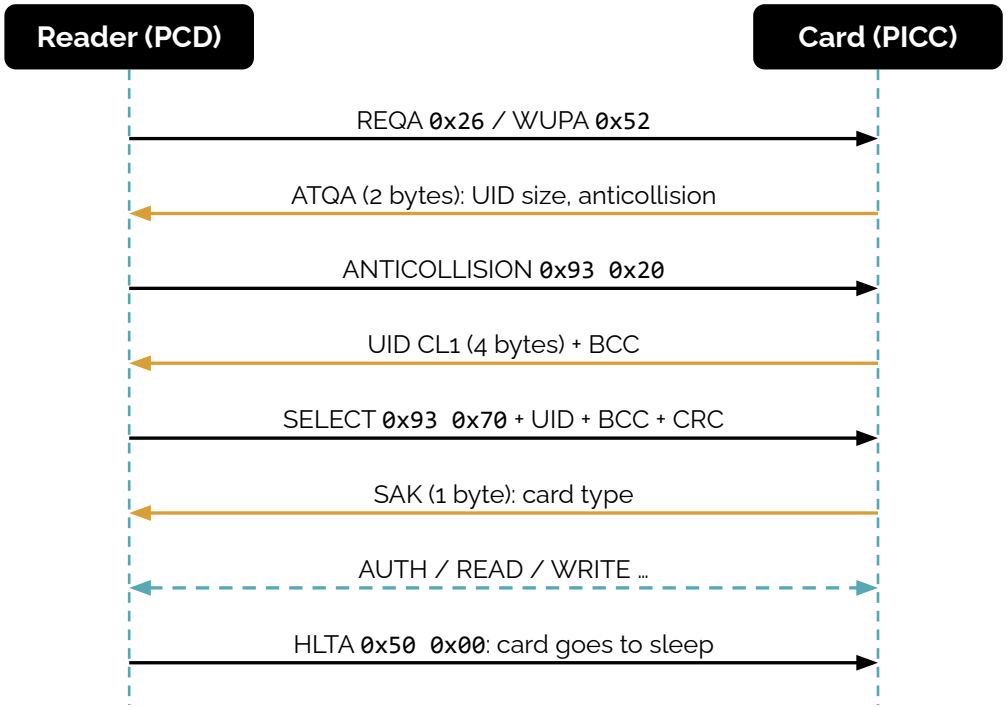


The RFID Technology

Band	Frequency	Passive range	Data rate	Main standards	Typical uses
LF	125–134.2 kHz	< 10 cm	~1–8 kbps	ISO 11784/11785, ISO 14223, EM4100	Animal ID, car immobilizers, legacy badges
HF	13.56 MHz	1–10 cm (14443), up to ~1 m (15693)	106–848 kbps	ISO 14443 A/B, ISO 15693, ISO 18092 (NFC)	Payments, transit, ID cards, libraries, NFC
UHF	860–960 MHz (EU 865–868, US 902–928)	up to ~10 m	40–640 kbps	ISO 18000-63 / EPC Gen2 (RAIN RFID)	Retail, logistics, supply chain, race timing
Microwave	2.45 / 5.8 GHz	1–10 m, often active	varies	ISO 18000-4, DSRC	Tolling, vehicle access, RTLS

- Lower frequency: better through water and near metal, but slow and short range.
- Higher frequency: faster and longer range, but UHF struggles with liquids and metal (special on-metal tags exist).

The RFID Technology



SAK tells you the card

SAK	Card
0x08	MIFARE Classic 1K
0x18	MIFARE Classic 4K
0x00	MIFARE Ultralight / NTAG
0x20	ISO 14443-4: DESFire, bank cards, phones (HCE)

- UID: 4, 7 or 10 bytes (cascade levels 1–3)
- Cascade tag `0x88` means "more UID bytes follow"
- BCC = XOR of the 4 UID bytes

The RFID Technology: MIFARE

	Block 0	Block 1	Block 2	Block 3
Sector 0	**0** manufacturer	**1** data	**2** data	**3** trailer
Sector 1	**4** data	**5** data	**6** data	**7** trailer
Sector 2	**8** data	**9** data	**10** data	**11** trailer
Sector 3	**12** data	**13** data	**14** data	**15** trailer
⋮	sectors 4–14 (blocks 16–59)			
Sector 15	**60** data	**61** data	**62** data	**63** trailer

Manufacturer (read-only) Data block Sector trailer

- **1024 B** = 16 sectors × 4 blocks × 16 B
- **Block** (16 B): unit of read/write
- **Sector**: unit of security, with its own Key A, Key B and access bits
- Absolute block = sector × 4 + block
- Usable: 47 blocks = **752 B**
- **Classic 4K**: 32 sectors × 4 blocks + 8 sectors × 16 blocks

The RFID Technology: MIFARE

Block 0: manufacturer block (read-only)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
UID (NUID)				BCC	SAK	ATQA		Manufacturer data							
DE	AD	BE	EF	22	08	04	00	..	..	..	..	..	..	..	..

Sector trailer: last block of every sector (factory values below)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Key A						Access bits			GPB	Key B					
FF	FF	FF	FF	FF	FF	FF	07	80	69	FF	FF	FF	FF	FF	FF

Data block

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
16 bytes of application data															
48	65	6C	6C	6F	00	00	00	00	00	00	00	00	00	00	00

Key A is never readable (reads back as 00). **Key B** is readable in the factory configuration: then it is just data, not a key.
BCC = DE ⊕ AD ⊕ BE ⊕ EF = 22.

The RFID Technology: MIFARE

	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0
Byte 6	$\neg C2_3$	$\neg C2_2$	$\neg C2_1$	$\neg C2_0$	$\neg C1_3$	$\neg C1_2$	$\neg C1_1$	$\neg C1_0$
Byte 7	$C1_3$	$C1_2$	$C1_1$	$C1_0$	$\neg C3_3$	$\neg C3_2$	$\neg C3_1$	$\neg C3_0$
Byte 8	$C3_3$	$C3_2$	$C3_1$	$C3_0$	$C2_3$	$C2_2$	$C2_1$	$C2_0$

Each block **b** of the sector gets 3 bits **C1b C2b C3b** (b = 3 is the trailer). Every bit is stored twice, plain and inverted (¬): if the copies do not match, the **sector is locked forever**.

Decoding FF 07 80 (factory)

```
B6 FF = 1111 1111  ¬C2=1111 ¬C1=1111
B7 07 = 0000 0111  C1=0000  ¬C3=0111
B8 80 = 1000 0000  C3=1000  C2=0000
```

- Blocks 0–2: C1C2C3 = **000** (read/write with A or B)
- Trailer: **001** (Key A rules everything)
- **78 77 88**: data **100** (read A|B, write B), trailer **011** (only Key B changes keys)

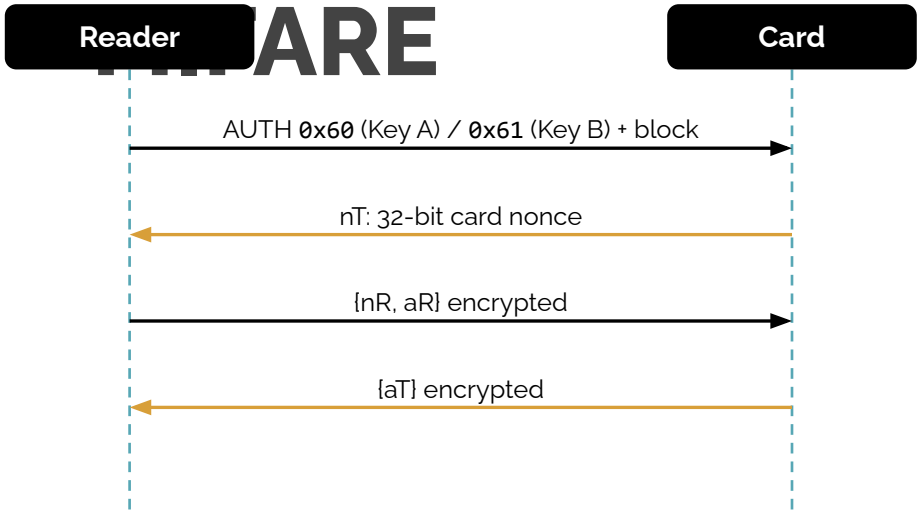
Compute access bytes with a tool and test on a **spare card**: one wrong write of a sector trailer can brick that sector forever.

The RFID Technology:

C1 C2 C3	Read	Write	Increment	Dec / Transfer / Restore	Typical use
0 0 0	A|B	A|B	A|B	A|B	Transport (factory) configuration
0 1 0	A|B	never	never	never	Read-only data
1 0 0	A|B	B	never	never	Read/write data
1 1 0	A|B	B	B	A|B	Value block, rechargeable with B
0 0 1	A|B	never	never	A|B	Value block, non-rechargeable
0 1 1	B	B	never	never	Read/write data, Key B only
1 0 1	B	never	never	never	Read-only, Key B only
1 1 1	never	never	never	never	Locked

- Table for **data blocks**. The **trailer** has its own table: who can read/write Key A, the access bits and Key B.
- If Key B is readable (factory config), every "B" above becomes **never**.
- Common pattern: read with Key A (readers in the field), write and manage keys with Key B (back office).

The RFID Technology: [illegible]ARE



Mutual 3-pass authentication, then the session is encrypted with **Crypto1** (48-bit key). Auth is **per sector**: to access another sector, authenticate again.

Command	Code	Notes
AUTH Key A / B	0x60 / 0x61	block address
READ	0x30	returns 16 B
WRITE	0xA0	16 B, 2-step
INCREMENT	0xC1	into buffer
DECREMENT	0xC0	into buffer
RESTORE	0xC2	copy to buffer
TRANSFER	0xB0	buffer to block
HALT	0x50 0x00	sleep

The RFID | Read - Write



```
#include <iot_board.h>
#include <Adafruit_PN532.h>
Adafruit_PN532 nfc(-1, -1, &Wire);    // I2C, IRQ and RST not wired
void setup() {
  IoTBoard::init_serial();
  IoTBoard::init_display();           // Wire.begin(A4, A5)
  nfc.begin();
  uint32_t v = nfc.getFirmwareVersion();       // cmd 0x02
  if (!v) { Serial.println("No PN532"); while (true) delay(1000); }
  Serial.printf("PN5%02X fw %u.%u\n", (v >> 24) & 0xFF,
                (v >> 16) & 0xFF, (v >> 8) & 0xFF);
  nfc.SAMConfig();                             // cmd 0x14
  nfc.setPassiveActivationRetries(0x10);       // do not wait forever
}
void loop() {
  uint8_t uid[7], len;
  if (!nfc.readPassiveTargetID(PN532_MIFARE_ISO14443A, uid, &len))
    return;                                    // cmd 0x4A
  Serial.print("UID:");
  for (uint8_t i = 0; i < len; i++) Serial.printf(" %02X", uid[i]);
  Serial.println(len == 4 ? "  Classic?" : "  NTAG?");
  delay(1000);
}
```

- `readPassiveTargetID()` = InListPassiveTarget: REQA, anticollision and SELECT run **inside the PN532**
- UID length hints the type: **4 B** Classic, **7 B** NTAG / Ultralight
- `0x10` activation retries: the chip gives up after 16 tries, so `loop()` keeps running (`0xFF` = wait forever)
- Init the display first: it starts the shared I^2C bus

The RFID | Read - Write



```
uint8_t keyA[6] = { 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF };  // factory
const uint8_t BLOCK = 4;                 // sector 1, block 0

void loop() {
  uint8_t uid[7], len, buf[16];
  if (!nfc.readPassiveTargetID(PN532_MIFARE_ISO14443A, uid, &len))
    return;
  if (len != 4) return;                  // Classic: 4-byte UID

  // 1) authenticate the sector: keyNumber 0 = Key A, 1 = Key B
  if (!nfc.mifareclassic_AuthenticateBlock(uid, len, BLOCK, 0, keyA)) {
    Serial.println("Auth failed: wrong key?");
    return;
  }
  // 2) write 16 bytes, then read them back
  uint8_t data[16] = "Hello UNICT IoT";  // 15 chars + '\0'
  if (nfc.mifareclassic_WriteDataBlock(BLOCK, data) &&
      nfc.mifareclassic_ReadDataBlock(BLOCK, buf)) {
    for (uint8_t i = 0; i < 16; i++) Serial.printf("%02X ", buf[i]);
    Serial.println();
  }
  delay(2000);
}
```

- Auth = InDataExchange **60** + block + key + UID: **Crypto1 runs in the PN532**
- Write **data blocks** only: never 0, 3, 7, 11… until you master access bits
- A library helper (IsTrailerBlock) tells you which blocks to skip
- After a failed auth the card stops answering: read the UID again

The NFC Technology

- **NFC** (Near Field Communication) is the evolution of **HF RFID at 13.56 MHz** (Sony + Philips, 2002; NFC Forum since 2004).
- Standardized as **ISO/IEC 18092** (NFCIP-1) and **ISO/IEC 21481** (NFCIP-2); compatible with ISO/IEC 14443 A/B, FeliCa and ISO/IEC 15693.
- Unlike classic RFID, an NFC device can be **reader, tag, or both**.
- The very short range is a **feature**: communication needs a deliberate tap.
- Tag data is formatted as **NDEF**: URL, Wi-Fi credentials, Bluetooth pairing, vCard, text…

13.56 MHz carrier frequency	**≤ 4 cm** typical range (10 cm max)
106–424 kbps data rate	**< 0.1 s** connection setup

The NFC Technology





Reader / Writer

The phone reads or writes a **passive tag**.

- Smart posters, product info, museum labels
- Write Wi-Fi or BT config to a sticker
- Android: Ndef, NfcA, MifareUltralight; iOS: Core NFC



Card Emulation

The phone or watch **behaves like a contactless card**, read by an external reader.

- Payments (EMV), transit, badges
- Credentials in a **Secure Element** or via **HCE** (Host Card Emulation, Android 4.4+)
- ISO 14443-4 APDUs routed by AID



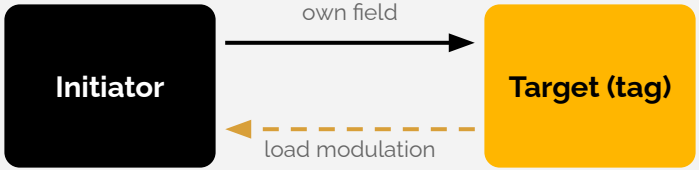
Peer-to-Peer

Two powered NFC devices **exchange data**.

- ISO 18092 NFC-DEP + LLCP + SNEP
- Android Beam (removed in Android 10)
- Today: **connection handover**, tap to pair, then BT/Wi-Fi carries the data

The NFC Technology

Passive communication



Only the **initiator** generates the RF field. The target answers with **load modulation**, so it can be unpowered (a tag).

Active communication



Initiator and target **alternately** generate their own field and switch it off while listening. Both need power.

Polling loop: an NFC controller cycles through NFC-A, NFC-B, NFC-F and NFC-V polling and listening phases to discover what is in front of it: a tag, a card reader or another phone.

The NFC Technology

Type	Based on	Example chips	Memory	Speed	Notes
Type 1	ISO 14443-A	Broadcom Topaz	96 B – 2 KB	106 kbps	Legacy, dropped from new specs
Type 2	ISO 14443-A	NTAG21x, MIFARE Ultralight	48 B – 2 KB	106 kbps	Cheapest and most common: stickers
Type 3	FeliCa (JIS X 6319-4)	Sony FeliCa Lite-S	up to 1 MB (spec)	212/424 kbps	Japan: transit (Suica), payments
Type 4	ISO 14443-A/B + ISO 7816-4 APDU	DESFire, NTAG 424 DNA, HCE	up to 32 KB	106–424 kbps	Secure (AES), smart cards
Type 5	ISO 15693 (NFC-V)	ICODE SLIX, ST25DV	up to 8 KB (typ.)	26.5 kbps	Longer range: libraries, industry

MIFARE Classic is not an NFC Forum type: its proprietary Crypto1 is supported only by NXP NFC controllers, so many Android phones cannot read it and iPhones do not either.

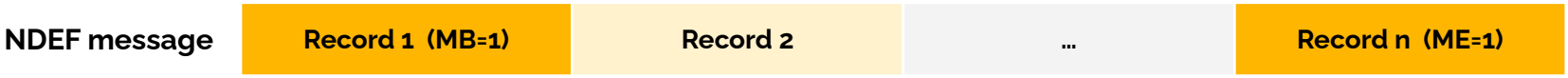
The NFC Technology

Reader / Writer	Peer-to-Peer	Card Emulation
NDEF + RTD	SNEP / Handover	Apps (EMV, transit…)
	LLCP	APDU (ISO 7816-4)
Tag Type 1–5 platform	NFC-DEP (ISO 18092)	ISO-DEP (ISO 14443-4)

Digital protocol NFC-A · NFC-B · NFC-F · NFC-V: framing, CRC, anti-collision, activation

RF / Analog 13.56 MHz, ASK modulation, Miller / Manchester coding (ISO 14443-2, ISO 18092, ISO 15693)

The NFC Technology



NDEF record





- **MB / ME**: Message Begin / End, first and last record
- **CF**: Chunk Flag, payload split over several records
- **SR**: Short Record, payload < 256 B, 1-byte length

- **IL**: the ID LENGTH and ID fields are present
- **TNF**: Type Name Format, how to read TYPE (next slide)
- A message is just records back to back: the phone dispatches on the **first** record

The NFC Technology

TNF	Meaning	Example
0x00	Empty	–
0x01	NFC Forum well-known (RTD)	"U" URI, "T" Text, "Sp" Smart Poster
0x02	Media type (RFC 2046)	application/vnd.wfa.wsc (Wi-Fi), text/vcard
0x03	Absolute URI	type is a URI
0x04	NFC Forum external type	android.com:pkg (Android App Record)
0x05	Unknown	opaque payload
0x06	Unchanged	middle/last chunks
0x07	Reserved	–

Code	URI prefix
0x00	(none)
0x01	http://www.
0x02	https://www.
0x03	http://
0x04	https://
0x05	tel:
0x06	mailto:

URI "U": 1 prefix byte saves space. **Text "T"**: status byte (bit 7 UTF-8/16, bits 5–0 language length) + "en" + text.

The NFC Technology

Goal: a phone tap opens **https://www.unict.it**. Here is the NTAG213 memory, byte by byte:

Page	Byte 0	Byte 1	Byte 2	Byte 3
3	E1	10	12	00
4	03	0D	D1	01
5	09	55 'U'	02 https://www.	75 'u'
6	6E 'n'	69 'i'	63 'c'	74 't'
7	2E '.'	69 'i'	74 't'	FE
8	00	00	00	00

- **E1 10 12 00**: Capability Container, NDEF tag v1.0, 144 B
- **03 0D**: NDEF Message TLV, length 13 bytes
- **D1**: header 1101 0001: MB ME SR set, TNF = 1 (well-known)
- **01** type length · **09** payload length · **55** type "U"
- **02**: URI prefix "https://www."
- **75 … 74**: "unict.it" in ASCII
- **FE**: Terminator TLV, end of NDEF data

The NFC | Write - Read



```
// NDEF for https://www.unict.it
uint8_t ndef[16] = {
  0x03, 0x0D, 0xD1, 0x01,  // p.4  TLV, len 13, header, type len
  0x09, 0x55, 0x02, 0x75,  // p.5  payload len, U, https://www., u
  0x6E, 0x69, 0x63, 0x74,  // p.6  n i c t
  0x2E, 0x69, 0x74, 0xFE   // p.7  . i t, Terminator TLV
};

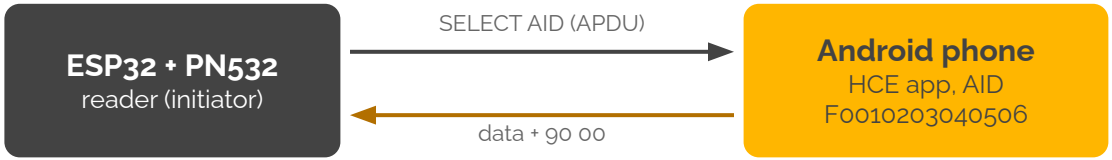
void loop() {
  uint8_t uid[7], len;
  if (!nfc.readPassiveTargetID(PN532_MIFARE_ISO14443A, uid, &len))
    return;
  if (len != 7) return;                    // NTAG21x: 7-byte UID

  for (uint8_t p = 0; p < 4; p++)          // 4 pages x 4 bytes
    nfc.ntag2xx_WritePage(4 + p, &ndef[p * 4]);

  Serial.println("Done: tap the tag with your phone!");
  delay(2000);
}
```

- Assumes a **factory-formatted** NTAG213: CC `E1 10 12 00` already in page 3
- `ntag2xx_WritePage()` = InDataExchange `A2` + page + 4 bytes
- **Never** write pages 2–3 or 40–44 by accident: lock bits and CC are one-time programmable
- Shortcut: `ntag2xx_WriteNDEFURI()` builds the same bytes from a prefix code and a string

ESP32 to Phone




```
uint8_t selectAid[] = {
  0x00, 0xA4, 0x04, 0x00,                 // SELECT by name
  0x07,                                   // Lc: AID length
  0xF0, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06,
  0x00 };                                 // Le

void loop() {
  if (!nfc.inListPassiveTarget()) return; // ISO 14443-4 target
  uint8_t resp[64], rlen = sizeof(resp);
  if (nfc.inDataExchange(selectAid, sizeof(selectAid), resp, &rlen))
    for (int i = 0; i < rlen; i++) Serial.printf("%02X ", resp[i]);
  delay(1000);
}
```

- An **APDU** is CLA INS P1 P2 [Lc data] [Le]; the answer ends with a status word, `90 00` = OK
- The phone side is an Android **HostApduService** registered for the same AID
- This is card emulation from the phone point of view: the MFRC522 cannot do it with its usual library

The RFID/NFC Security

Eavesdropping

Sniffing the radio exchange from meters away.

Fix: encrypted sessions (AES), no secrets in clear.

Skimming

Reading a card in a pocket or bag without consent.

Fix: mutual authentication, shielding sleeves.

Cloning & replay

Copying UID or data onto a magic card or emulator.

Fix: challenge-response crypto, dynamic data (SUN).

Relay attack

Two devices bridge a real card and a distant reader.

Fix: distance bounding, strict timing, limits.

Tracking

Static UIDs and EPCs let anyone follow a person.

Fix: random UIDs, EPC kill, no personal data on tag.

Tampering

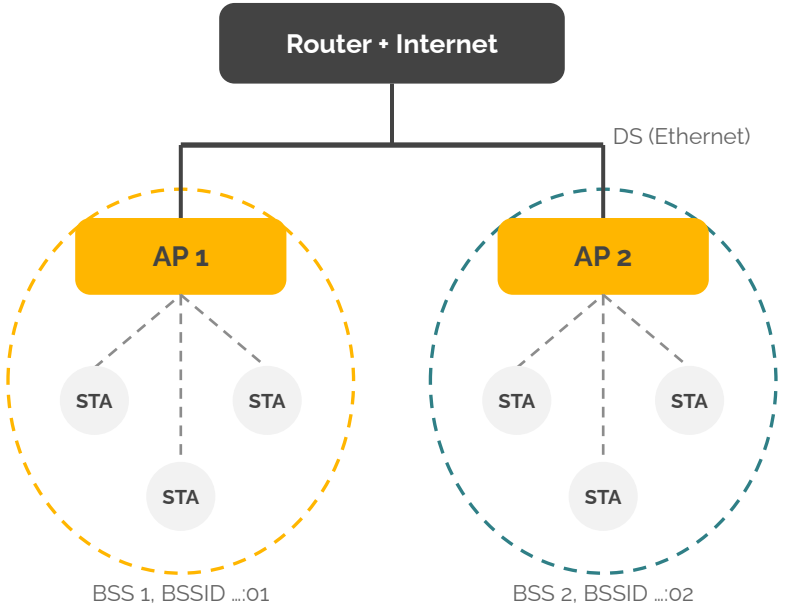
Overwriting unlocked NDEF tags (malicious URLs).

Fix: lock bits or password, signed content.

The WLAN Networks

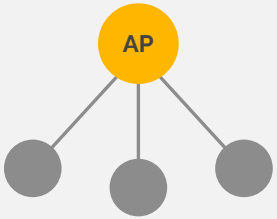
A **WLAN** (Wireless LAN) links devices by radio inside a limited area (home, campus, office) and, through a gateway, to the Internet. **IEEE 802.11** defines its PHY and MAC; **Wi-Fi** is the Wi-Fi Alliance brand for certified 802.11 products.

Term	Meaning
STA	station: any 802.11 client
AP	access point: bridges radio and wired network
BSS / BSSID	one AP and its stations / the AP radio MAC
ESS / SSID	several BSSs sharing one network name
DS	distribution system linking the APs (Ethernet)



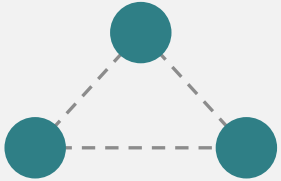
ESS: both APs announce the SSID **"WLUCTSTUD"**

The WLAN Networks



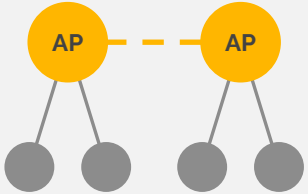
Infrastructure

Clients talk only through the **AP**. APs with the same SSID form an **ESS**; roaming picks the strongest. Homes, campuses, offices.



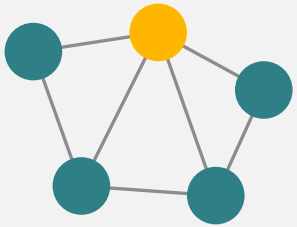
Ad hoc (IBSS) / P2P

Stations talk **directly**, no AP. Quick links between two devices; no roaming, little management.



Bridge / WDS

APs linked **by radio** instead of cable. WDS keeps client MAC addresses using the **4-address** frame format.



Mesh (802.11s)

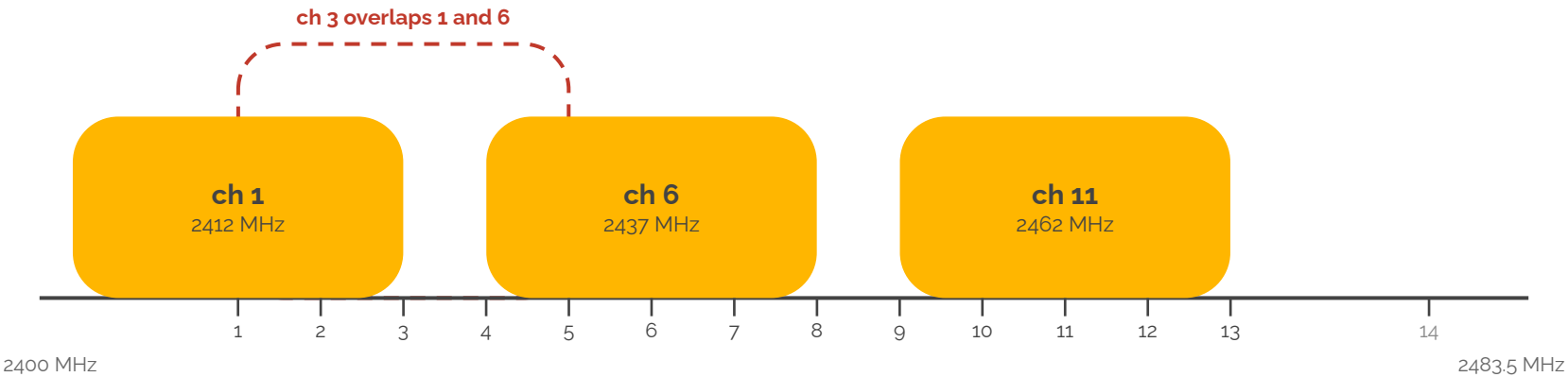
Every node **forwards** frames for the others (HWMP or DV routing). Vendors also have their own mesh, e.g. ESP-MESH.

The Wi-Fi Family

Name	Standard	Year	Band	Max rate	What it added
–	802.11b	1999	2.4 GHz	11 Mbps	DSSS/CCK: the first mass-market Wi-Fi
–	802.11a	1999	5 GHz	54 Mbps	OFDM, less crowded band
–	802.11g	2003	2.4 GHz	54 Mbps	OFDM in 2.4 GHz, backward compatible with b
Wi-Fi 4	802.11n	2009	2.4 / 5 GHz	600 Mbps	MIMO, 40 MHz channels, frame aggregation
Wi-Fi 5	802.11ac	2013	5 GHz	6.9 Gbps	MU-MIMO (downlink), 80/160 MHz channels
Wi-Fi 6/6E	802.11ax	2019/20	2.4 / 5 / 6 GHz	9.6 Gbps	OFDMA, **TWT** sleep scheduling: good for IoT
Wi-Fi 7	802.11be	2024	2.4 / 5 / 6 GHz	~46 Gbps	320 MHz channels, multi-link operation
HaLow	802.11ah	2017	sub-1 GHz	kbps–Mbps	IoT: about 1 km range, low power

Our ESP32-S3 is Wi-Fi 4: 802.11 b/g/n, 2.4 GHz only, up to 150 Mbps. The ESP32-C6 adds Wi-Fi 6 with TWT, the ESP32-C5 also 5 GHz.

The Wi-Fi - 2.4 GHz Band



13
channels in Europe (14: Japan, b only)

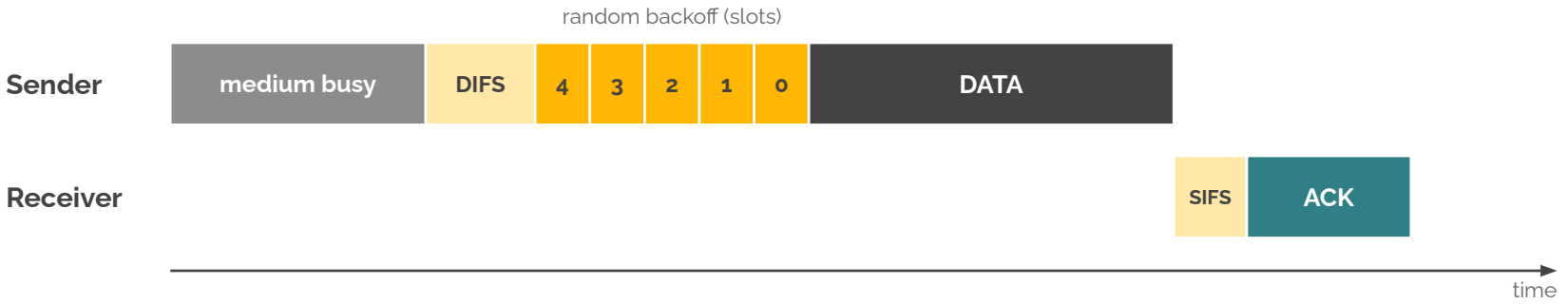
5 MHz
between channel centers

20–22 MHz
width of one channel

3
non-overlapping: 1, 6, 11

The band is shared with **Bluetooth**, **802.15.4** (ZigBee) and microwave ovens: interference is the norm, pick the channel after a scan.

The Wi-Fi - CSMA/CA



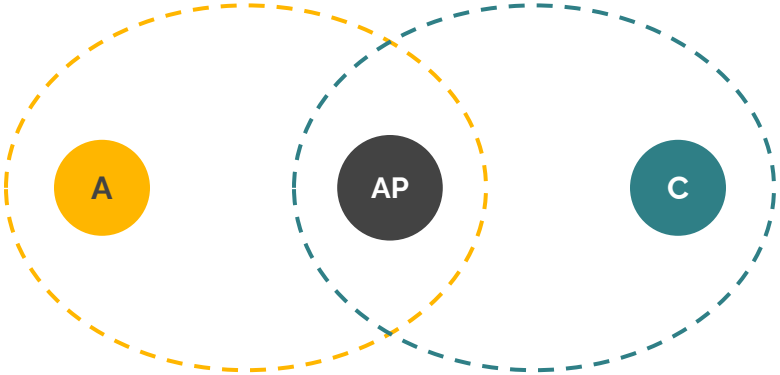
Avoid, because you cannot detect

- A radio cannot hear while it transmits: collisions are invisible (Ethernet uses CSMA/**CD**).
- Listen first (**carrier sense**), wait **DIFS** plus a **random backoff**, then send.
- Every unicast frame is **ACKed** after **SIFS**: no ACK = retry.

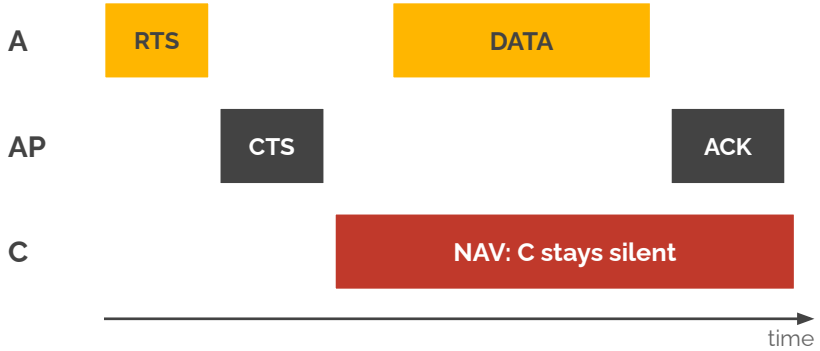
Backoff rules

- Random number of slots in [0, CW]; the counter **freezes** while the medium is busy.
- After a failure CW doubles (15, 31, … up to 1023): **binary exponential backoff**.
- SIFS < DIFS, so an ACK always wins against new transmissions.

The Wi-Fi - Hidden Node



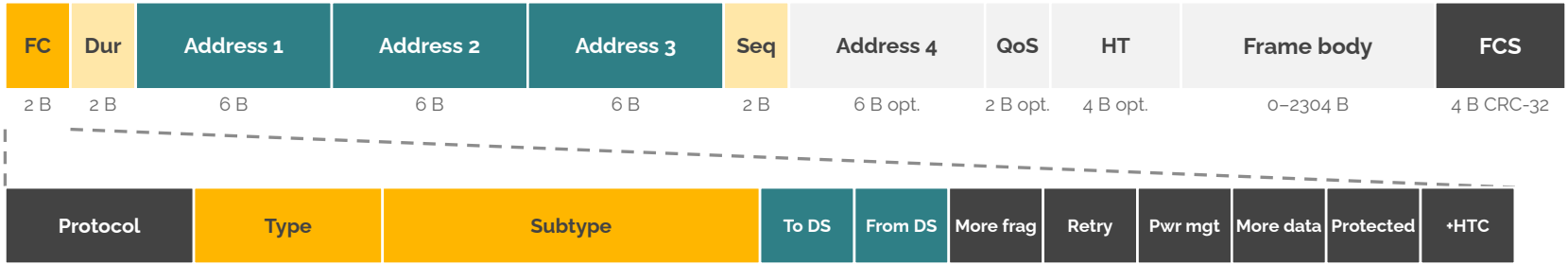
A and C both reach the AP, but cannot hear each other



- Both sense an idle channel and transmit: **collision at the AP**, and neither knows why.
- **RTS (Request to Send)/CTS (Clear to Send)**: the sender reserves the medium; the **CTS** from the AP is heard by everyone, C included.
- Every frame carries a **Duration** field: listeners set their **NAV (Network Allocation Vector)** timer (virtual carrier sense).
- RTS/CTS costs airtime: used only above the **RTS threshold** (large frames).

The Wi-Fi Frame

MAC header: 24–36 B



Frame Control: 16 bits

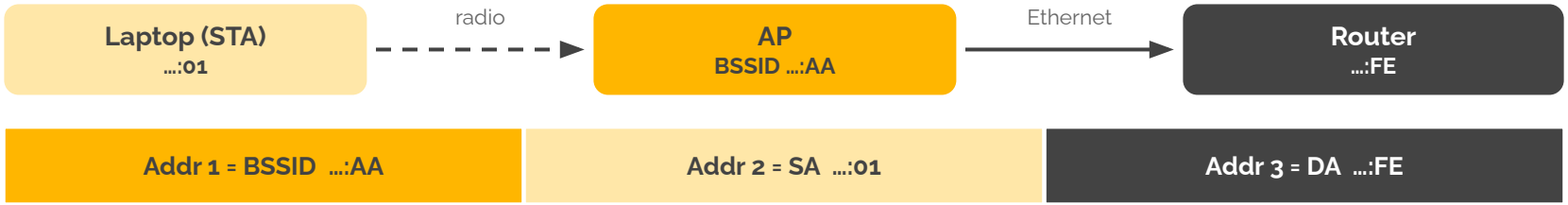
Type	Bits	Subtypes (examples)
Management	00	Beacon, Probe Request/Response, Authentication, Association, Deauthentication
Control	01	RTS, CTS, ACK, Block Ack
Data	10	Data, QoS Data, Null (power-save signalling)

Like Ethernet, but with up to **4 addresses**, a **Duration** for the NAV and per-frame **Retry** and **Protected** flags.

The Wi-Fi Frame - Addresses

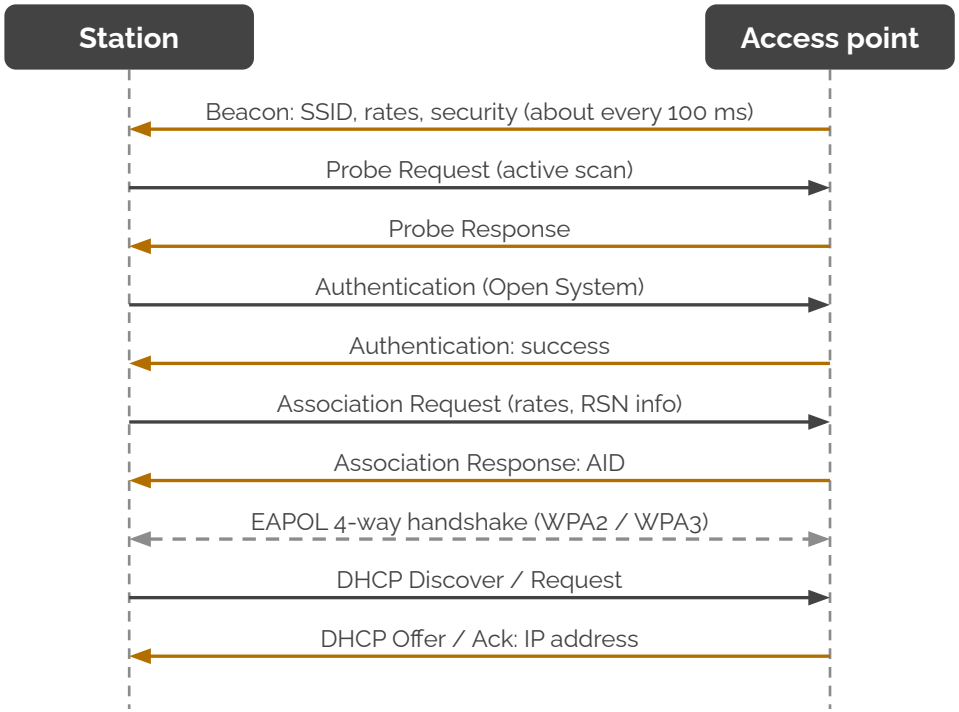
To DS	From DS	Address 1	Address 2	Address 3	Address 4	Used for
0	0	DA	SA	BSSID	–	ad hoc, management frames
1	0	BSSID	SA	DA	–	station to AP
0	1	DA	BSSID	SA	–	AP to station
1	1	RA	TA	DA	SA	WDS / mesh link between APs

Example: a laptop sends a packet to the Internet



To DS = 1, From DS = 0: the radio receiver is the AP, the final destination is the router. The AP rewrites it as an Ethernet frame SA → DA.

The Wi-Fi Joining



1 Discovery

Passive (listen to beacons) or active (probe) scan, channel by channel.

2 Auth + association

States: unauthenticated, authenticated, associated. The AP assigns an AID.

3 Key handshake

Derives the session keys; only now does the AP forward data frames.

4 IP

DHCP gives address, gateway, DNS: TCP/IP finally works.

The Wi-Fi Security



WEP
1997

RC4 + static key. Broken in 2001, crackable in a minute.

WPA
2003

TKIP: a firmware patch on WEP hardware, now deprecated.

WPA2
2004

802.11i: **AES-CCMP**, 4-way handshake. Still the most deployed.

WPA3
2018

SAE instead of PSK, PMF mandatory, Enhanced Open.

Not security, even if often sold as such

Hidden SSID

The SSID still travels in clear in probe requests and association frames.

MAC filtering

MAC addresses are sent in clear and can be spoofed in seconds.

Open + captive portal

Login page, but no encryption: everybody on the air reads the traffic.

The Wi-Fi for the IoT



Strengths

- Native **IP**: talk to the cloud, MQTT and HTTP directly
- Infrastructure is already everywhere
- Throughput for images, audio and OTA updates

Weaknesses

- **Power**: the radio draws hundreds of mA while sending; joining takes seconds
- One AP serves a limited number of clients
- The device needs credentials: **provisioning**

Provisioning: how does a new device learn the password?

- **SoftAP + web portal**
- **BLE provisioning**
- **SmartConfig / ESP-Touch**
- **WPS button**
- **Easy Connect (QR)**

Saving power: **modem sleep** between DTIM beacons, **deep sleep** between readings, Wi-Fi 6 **TWT** to schedule wake-ups.

The Wi-Fi on ESP32

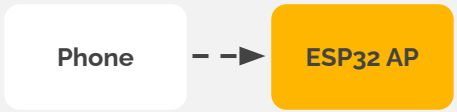
`WiFi.mode(WIFI_STA)`



Station

The ESP32 joins an existing network and gets an IP from its DHCP server. The normal IoT case.

`WiFi.mode(WIFI_AP)`



Access point

The ESP32 creates its own network (SoftAP, 192.168.4.1) with a DHCP server. Up to a few clients.

`WiFi.mode(WIFI_AP_STA)`

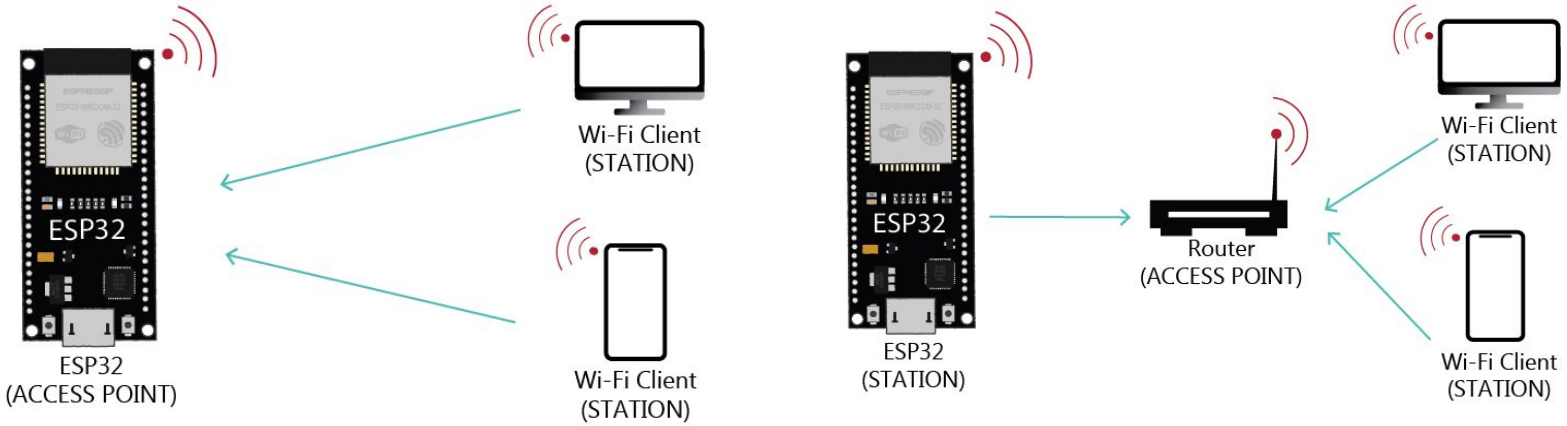


Both

Joined to the router and serving its own network: configuration portals, gateways, ESP-NOW + Wi-Fi.

One radio, **one channel**: in AP+STA mode the SoftAP must follow the channel of the router the station is connected to.

The Wi-Fi on ESP32



The Wi-Fi on ESP32 : Station



```
#include <iot_board.h>
#include <WiFi.h>
const char *SSID = "IoT-Lab";
const char *PASS = "password123456";

void setup() {
  IoTBoard::init_serial();
  IoTBoard::init_display();
  WiFi.mode(WIFI_STA);
  WiFi.begin(SSID, PASS);          // scan, assoc, 4-way, DHCP
  unsigned long t0 = millis();
  while (WiFi.status() != WL_CONNECTED && millis() - t0 < 15000) delay(250);// never wait
forever
  display->clearDisplay();
  display->setCursor(0, 0);
  if (WiFi.status() == WL_CONNECTED) {
    display->printf("IP   %s\n", WiFi.localIP().toString().c_str());
    display->printf("RSSI %d ch %d\n", WiFi.RSSI(), WiFi.channel());
  } else {
    display->println("Wi-Fi: no connection");
  }
  display->display();
}
```

- `WiFi.begin()` starts the whole join of the previous slides and returns at once
- `WL_CONNECTED` = associated **and** IP obtained
- Always use a **timeout**: a wrong password must not freeze the board
- Credentials in the code are a lab shortcut: real products use provisioning

The Wi-Fi on ESP32 : Events



```
#include <WiFi.h>
void onWiFi(WiFiEvent_t event, WiFiEventInfo_t info) {
  switch (event) {
    case ARDUINO_EVENT_WIFI_STA_CONNECTED:
      Serial.println("Associated to the AP");
      break;
    case ARDUINO_EVENT_WIFI_STA_GOT_IP:
      Serial.print("IP: ");
      Serial.println(IPAddress(info.got_ip.ip_info.ip.addr));
      break;
    case ARDUINO_EVENT_WIFI_STA_DISCONNECTED:
      Serial.printf("Link lost, reason %d\n",
                    info.wifi_sta_disconnected.reason);
      break;                        // auto-reconnect is on
    default:
      break;
  }
}
void setup() {
  Serial.begin(115200);
  WiFi.onEvent(onWiFi);             // one handler for every event
  WiFi.begin(SSID, PASS);           // no busy wait: events tell us
}
```

ARDUINO_EVENT_WIFI_...

Event	When
STA_START	radio started
STA_CONNECTED	associated
STA_GOT_IP	DHCP done
STA_DISCONNECTED	link lost (+ reason)
AP_STACONNECTED	client joined our AP
SCAN_DONE	async scan finished

The handler runs in the Wi-Fi event task: keep it short, set flags, do heavy work in `loop()`.

The Wi-Fi on ESP32

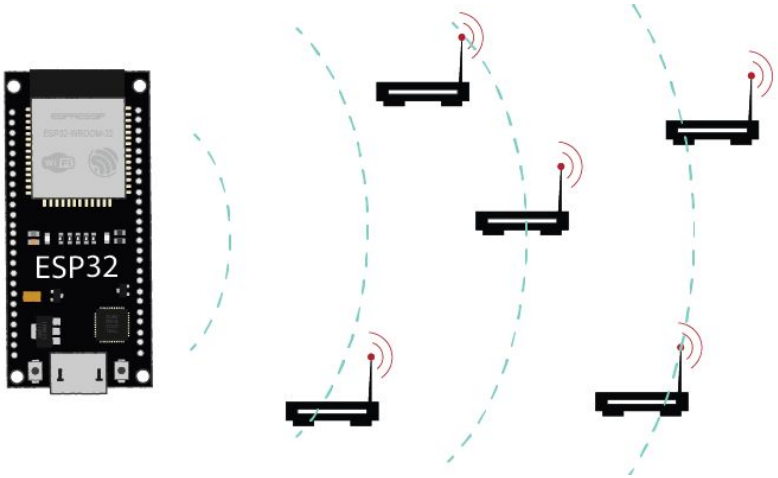


The ESP32 can handle all the following Wi-Fi events via **`onEvent()`**.

```
0  ARDUINO_EVENT_WIFI_READY               < ESP32 WiFi ready
1  ARDUINO_EVENT_WIFI_SCAN_DONE           < ESP32 finish scanning AP
2  ARDUINO_EVENT_WIFI_STA_START           < ESP32 station start
3  ARDUINO_EVENT_WIFI_STA_STOP            < ESP32 station stop
4  ARDUINO_EVENT_WIFI_STA_CONNECTED       < ESP32 station connected to AP
5  ARDUINO_EVENT_WIFI_STA_DISCONNECTED    < ESP32 station disconnected from AP
6  ARDUINO_EVENT_WIFI_STA_AUTHMODE_CHANGE < the auth mode of AP connected by ESP32 station changed
7  ARDUINO_EVENT_WIFI_STA_GOT_IP          < ESP32 station got IP from connected AP
8  ARDUINO_EVENT_WIFI_STA_LOST_IP         < ESP32 station lost IP and the IP is reset to 0
9  ARDUINO_EVENT_WPS_ER_SUCCESS       < ESP32 station wps succeeds in enrollee mode
10 ARDUINO_EVENT_WPS_ER_FAILED        < ESP32 station wps fails in enrollee mode
11 ARDUINO_EVENT_WPS_ER_TIMEOUT       < ESP32 station wps timeout in enrollee mode
12 ARDUINO_EVENT_WPS_ER_PIN           < ESP32 station wps pin code in enrollee mode
13 ARDUINO_EVENT_WIFI_AP_START             < ESP32 soft-AP start
14 ARDUINO_EVENT_WIFI_AP_STOP              < ESP32 soft-AP stop
15 ARDUINO_EVENT_WIFI_AP_STACONNECTED      < a station connected to ESP32 soft-AP
16 ARDUINO_EVENT_WIFI_AP_STADISCONNECTED   < a station disconnected from ESP32 soft-AP
17 ARDUINO_EVENT_WIFI_AP_STAIPASSIGNED     < ESP32 soft-AP assign an IP to a connected station
18 ARDUINO_EVENT_WIFI_AP_PROBEREQRECVED    < Receive probe request packet in soft-AP interface
19 ARDUINO_EVENT_WIFI_AP_GOT_IP6           < ESP32 ap interface
v6IP addr is preferred
19 ARDUINO_EVENT_WIFI_STA_GOT_IP6          < ESP32 station
interface v6IP addr is preferred
20 ARDUINO_EVENT_ETH_START             < ESP32 ethernet start
21 ARDUINO_EVENT_ETH_STOP              < ESP32 ethernet stop
22 ARDUINO_EVENT_ETH_CONNECTED         < ESP32 ethernet phy link
up
23 ARDUINO_EVENT_ETH_DISCONNECTED      < ESP32 ethernet phy link
down
24 ARDUINO_EVENT_ETH_GOT_IP            < ESP32 ethernet got IP
from connected AP
19 ARDUINO_EVENT_ETH_GOT_IP6           < ESP32 ethernet
interface v6IP addr is preferred
25 ARDUINO_EVENT_MAX
```

The Wi-Fi on ESP32

The ESP32 can scan nearby Wi-Fi networks within its Wi-Fi range.



This can be useful to check if the Wi-Fi network you're trying to connect is within the range of your board or other applications. Your Wi-Fi project may not often work because it may not be able to connect to your router due to insufficient Wi-Fi strength.

The Wi-Fi on ESP32 : Scan



```ino
#include <WiFi.h>

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  WiFi.disconnect();               // scan only, do not join
}

void loop() {
  int n = WiFi.scanNetworks();     // active scan, all channels
  Serial.printf("%d networks\n", n);
  for (int i = 0; i < n; i++) {
    Serial.printf("%-20s ch %2d %4d dBm %s\n",
      WiFi.SSID(i).c_str(), (int)WiFi.channel(i),
      (int)WiFi.RSSI(i),
      WiFi.encryptionType(i) == WIFI_AUTH_OPEN ? "open" : "secured");
  }
  WiFi.scanDelete();               // free the result list
  delay(10000);
}
```

Reading the RSSI

-30 dBm	excellent, next to the AP
-67 dBm	good: video, voice
-70 dBm	fine for IoT telemetry
-80 dBm	unstable, retries
-90 dBm	unusable

dBm is logarithmic: **-3 dB = half the power**, -10 dB = one tenth.

The Wi-Fi on ESP32 : softAP



```
#include <WiFi.h>

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_AP);
  // ssid, password, channel, hidden, max clients
  WiFi.softAP("IoTBoard-42", "iot-lab-2026", 6, 0, 4);
  Serial.print("AP address: ");
  Serial.println(WiFi.softAPIP());   // 192.168.4.1
}

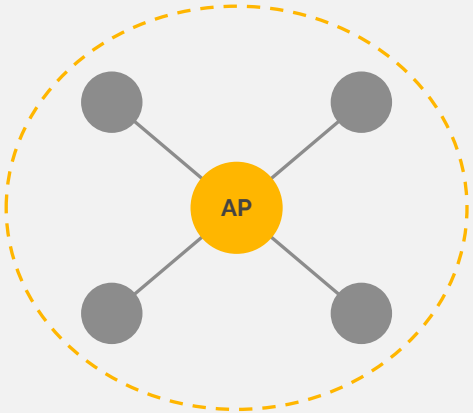
void loop() {
  Serial.printf("Clients: %d\n", WiFi.softAPgetStationNum());
  delay(5000);
}
```

Parameter	Meaning
ssid	network name, max 32 bytes
password	8–63 chars (WPA2), NULL = open
channel	1–13 in Europe
ssid_hidden	1 = no SSID in beacons
max_connection	max clients, default 4

Add a web server on port 80 and you have a **configuration portal**: the classic way to give an IoT device the home Wi-Fi password (did you followed the Web Development course?).

The Wi-Fi Mesh on ESP32

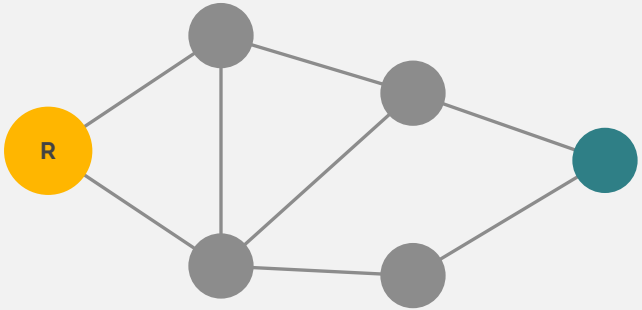
Traditional: a star around the AP



X

out of range

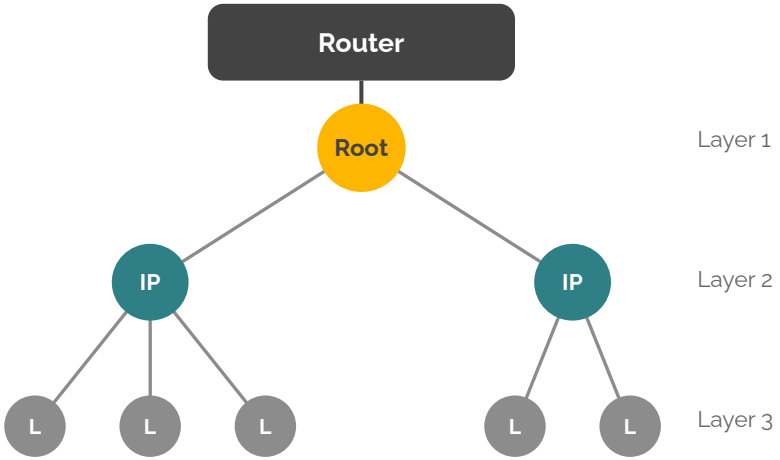
Mesh: nodes relay for each other



far node reached in 3 hops

A mesh is **self-organizing** (nodes pick their parents on their own) and **self-healing** (a broken link is routed around).

The ESP-Mesh on ESP32



IP = intermediate parent, L = leaf

- Only the **root** talks to the router: the gateway to IP
- Every node is a **station** upstream and a **SoftAP** downstream
- Parent choice from beacons: RSSI threshold, then the **shallowest** layer with fewest children
- Root lost: layer-2 nodes **elect a new root**; parent lost: pick another one
- Same mesh ID, router settings and **channel** on every node

The ESP-MESH protocol was defined, designed and developed by **ARSLab of University of Catania** and implemented by Espressif for ESP devices. M. Buffa, F. Messina, C. Santoro, F. F. Santoro, "Design of self-organizing protocol for LoWPAN networks", IDCS 2019, Naples.

The ESP-Mesh on ESP32



Your sketch
`onMesh(event), espmesh->sendToRoot(), ...`

↓

EspMesh (IoTBoard-Library)
Wi-Fi + mesh set-up, receive tasks, event queue

↓

ESP-IDF: esp_wifi + esp_mesh

↓

Radio: station upstream, SoftAP downstream

Build flag `-DESPMESH_ENABLED`. The mesh uses the Wi-Fi radio: no `WiFi.begin()` in the same sketch.

EspMeshEventType	Meaning
Connected	joined the mesh (has a parent)
Disconnected	lost its parent
BecameRoot	elected root of the mesh
GotIP	the root got an IP from the router
ChildJoined / Left	a child attached or left (`from` = MAC)
Message	data received: `from`, `data`, `len`

Events are queued by the mesh tasks and delivered by `espmesh->update()` in `loop()`: inside the callback the display and the LEDs are safe.

The ESP-Mesh on ESP32



```
#include <iot_board.h>   // + -DESPMESH_ENABLED
const char *SSID = "IoT-Lab";
const char *PASS = "router-passphrase";
int sent = 0;

void onMesh(const EspMeshEvent &e) {
  switch (e.type) {
  case EspMeshEventType::Connected:
    digitalWrite(LED_GREEN, HIGH); break;
  case EspMeshEventType::Disconnected:
    digitalWrite(LED_GREEN, LOW);
    digitalWrite(LED_YELLOW, LOW); break;
  case EspMeshEventType::BecameRoot:
    digitalWrite(LED_YELLOW, HIGH); break;
  case EspMeshEventType::Message:
    display->clearDisplay();
    display->setCursor(0, 0);
    display->printf("From " MACSTR "\n",
                    MAC2STR(e.from));
    display->println((char *)e.data);
    display->display(); break;
  default: break;
  }
}
```

```
void setup() {
  IoTBoard::init_serial();
  IoTBoard::init_leds();
  IoTBoard::init_display();
  if (IoTBoard::init_espmesh(SSID, PASS, onMesh))
    display->println("EspMesh enabled");
  else
    display->println("EspMesh error");
  display->display();
}

void loop() {
  espmesh->update();           // runs the callback
  if (!IoTBoard::runEvery(5000)) return;
  if (espmesh->isRoot() || !espmesh->isConnected())
    return;                    // the root only listens
  String s = "Hello " + String(sent++) +
             " layer " + String(espmesh->getLayer());
  espmesh->sendToRoot(s.c_str(), s.length());
}
```

The ESP-Mesh on ESP32

espmesh->…	Delivers to	How
`sendToRoot(data, len)`	the root, from any node	routed up the tree
`sendTo(mac, data, len)`	one node, by MAC address	point to point (P2P)
`broadcast(data, len)`	every node of the mesh	P2P to FF:FF:FF:FF:FF:FF
`sendToGroup(group, data, len)`	the nodes that called `joinGroup(group)`	mesh group (multicast)
`sendToServer(ip, port, data, len)`	a server outside the mesh	through the root, UDP

Ask the mesh

`isRoot()`, `isConnected()`, `hasIP()`, `getLayer()`, `getMac(mac)`, `getNodes(nodes, max)`: on the root, the MACs of all nodes.

Rules

- At most **1472 B** per message
- Group ID = a multicast MAC, e.g. 01:00:5E:00:00:01

The ESP-Mesh on ESP32

```
void setup() {
  // ... init_serial(), init_leds(), init_display()
  EspMeshConfig config;
  config.router_ssid  = "IoT-Lab";
  config.router_pass  = "router-passphrase";
  config.mesh_id[5]   = 0x01;              // default 77:77:77:77:77:77
  config.mesh_pass    = "team-1-pass";     // default iot-lab-board-pass
  config.channel      = 0;                 // 0 = scan for the router
  config.max_layer    = 4;                 // default 6
  config.max_children = 4;                 // default 6
  config.role         = EspMeshRole::Auto; // Auto, Root or Node
  config.on_event     = onMesh;
  IoTBoard::init_espmesh(config);
}
```

- `role`: **Auto** = election; **Root** / **Node** fix the roles
- `max_layer` × `max_children` limit the size of the tree
- `channel = 0` scans every channel: a fixed one starts faster
- Mesh password: 8+ chars, it protects the links between nodes

Change **`mesh_id`** and **`mesh_pass`** per team: one lab, many independent meshes on the same router.

The ESP-Mesh on ESP32




```
IPAddress server = IPAddress("192.168.1.5");          // the PC running nc
const uint16_t PORT = 5000;

void onBtn2(uint8_t pin) {                  // works on any node
  const char *msg = "button pressed";
  espmesh->sendToServer(server, PORT, msg, strlen(msg));
}

void setup() {
  // ... init_serial(), init_buttons(), onBtn2Release(onBtn2)
  EspMeshConfig config;
  config.router_ssid  = SSID;
  config.router_pass  = PASS;
  config.forward_ip   = server;             // the root copies every
  config.forward_port = PORT;               // message to this server
  IoTBoard::init_espmesh(config);
}
```

- **forward_ip/port**: the root sends every message it receives to the server
- Still works after a **new root** is elected
- `sendToServer()` reaches the server from any node, through the root

The painlessMesh on ESP32

An open-source **Arduino library** for ESP8266 and ESP32 meshes. It is **not** Espressif's ESP-MESH: it builds its own mesh on top of ordinary Wi-Fi.

	ESP-MESH	painlessMesh
Made by	ARSLab, Espressif, in ESP-IDF	open-source community
Topology	tree with one root	self-organizing mesh
Messages	binary, IP via the root	JSON over TCP
Addressing	MAC addresses	32-bit node ID
Programming	ESP-IDF API	Arduino (C++)

Install `painlessMesh` from the library manager: it pulls in ArduinoJson, TaskScheduler and AsyncTCP.

AP + STA

Every node is access point and station; node links are TCP connections.

No IP for you

Nodes have a 32-bit ID from the chip MAC; no TCP/IP network to the app.

Time sync

All nodes share a mesh clock (useful for coordinated actions).

Send

`sendBroadcast()` to all or `sendSingle(id)` to one node.

The painlessMesh on ESP32



```
#include <painlessMesh.h>
#define MESH_PREFIX   "iotMesh"     // mesh SSID, same on all nodes
#define MESH_PASSWORD "meshPass123" // WPA2 key of the mesh links
#define MESH_PORT     5555          // TCP port between nodes

Scheduler userScheduler;            // TaskScheduler, shared with mesh
painlessMesh mesh;

void sendMessage();
Task taskSendMessage(TASK_SECOND * 1, TASK_FOREVER, &sendMessage);

void sendMessage() {
  String msg = "Hi from node " + String(mesh.getNodeId());
  mesh.sendBroadcast(msg);
  taskSendMessage.setInterval(random(TASK_SECOND, TASK_SECOND * 5));
}

void receivedCallback(uint32_t from, String &msg) {
  Serial.printf("From %u: %s\n", from, msg.c_str());
}
```

- Same **prefix**, **password** and **port** on every node, nothing else to configure
- A **Task** runs `sendMessage()` every 1–5 s (random, to avoid collisions)
- `getNodeId()`: the 32-bit ID of this node
- Messages are Strings: put JSON inside for structured data

The painlessMesh on ESP32



```
void newConnectionCallback(uint32_t nodeId) {
  Serial.printf("New connection: %u\n", nodeId);
}
void changedConnectionCallback() {
  Serial.println("Topology changed");
}
void nodeTimeAdjustedCallback(int32_t offset) {
  Serial.printf("Time %u, offset %d\n", mesh.getNodeTime(), offset);
}

void setup() {
  Serial.begin(115200);
  mesh.setDebugMsgTypes(ERROR | STARTUP);    // before init()
  mesh.init(MESH_PREFIX, MESH_PASSWORD, &userScheduler, MESH_PORT);
  mesh.onReceive(&receivedCallback);
  mesh.onNewConnection(&newConnectionCallback);
  mesh.onChangedConnections(&changedConnectionCallback);
  mesh.onNodeTimeAdjusted(&nodeTimeAdjustedCallback);
  userScheduler.addTask(taskSendMessage);
  taskSendMessage.enable();
}
void loop() { mesh.update(); }               // also runs the tasks
```

- Callbacks for messages, new links, topology changes and **clock sync**
- `mesh.update()` must run often: **never** use long `delay()` calls
- Flash the same sketch on 3+ boards and watch the topology form

The ESP-NOW on ESP32

- Espressif protocol on the **Wi-Fi radio**, without AP, association, DHCP or IP
- Data travels in **vendor-specific action frames**, addressed by **MAC**
- **Connectionless**: a peer is just a registered MAC (+ optional key), no handshake
- Unicast frames are **ACKed by the MAC layer**: success does not mean the app processed it
- Optional encryption: **CCMP** with a per-peer key (LMK, itself encrypted by the PMK)
- Sender and receiver must be on the **same channel**

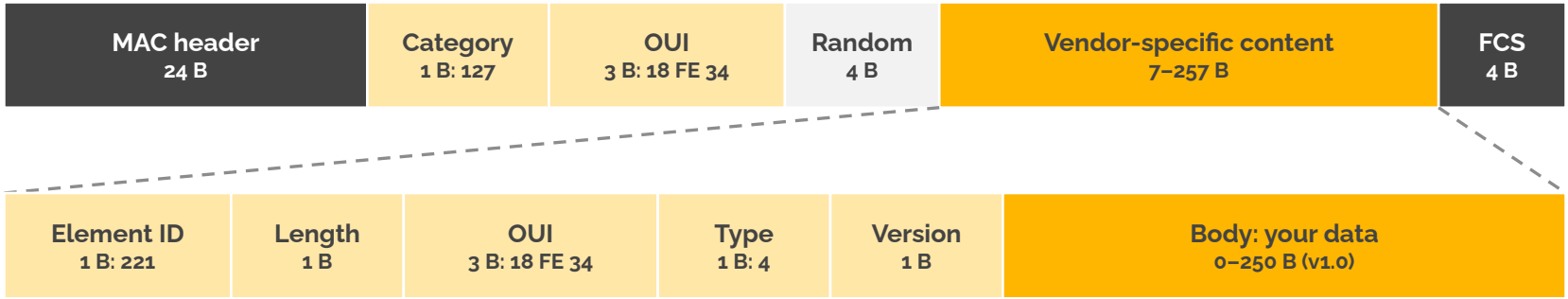
250 B payload, ESP-NOW v1.0	**1470 B** payload, v2.0 (recent ESP-IDF)
20 peers at most	**17** encrypted peers max (default 7)
1 Mbps default bit rate	**0** handshakes before sending

The ESP-NOW on ESP32



The ESP-NOW Frame

An 802.11 **action frame** (management type), sent as a normal Wi-Fi frame:



Why it is fast

- No scan, no association, no keys exchange, no DHCP
- One frame out, one MAC-level ACK back

Addressing

- Destination MAC in the header; `FF:FF:FF:FF:FF:FF` = broadcast (**no ACK**)
- An AP ignores these frames; any ESP on the same channel can receive them

The ESP-NOW : Sender



```
#include <iot_board.h>
#include <WiFi.h>
#include <esp_now.h>
// receiver MAC; FF:..:FF = broadcast (no ACK)
uint8_t peer[6] = {0xFF,0xFF,0xFF,0xFF,0xFF,0xFF};
typedef struct __attribute__((packed)) {
  uint8_t  board;          // who is sending
  uint32_t presses;        // the payload
} Msg;                     // same on the receiver!
Msg msg = {1, 0};
volatile bool pressed = false;

void onBtn1(uint8_t pin) { pressed = true; }

#if ESP_IDF_VERSION >= ESP_IDF_VERSION_VAL(5, 5, 0)
void onSent(const esp_now_send_info_t *info,
            esp_now_send_status_t st) {
#else
void onSent(const uint8_t *mac,
            esp_now_send_status_t st) {
#endif
  digitalWrite(LED_RED, st != ESP_NOW_SEND_SUCCESS);
}
```

```
void setup() {
  IoTBoard::init_serial();
  IoTBoard::init_leds();
  IoTBoard::init_buttons();
  buttons->onBtn1Release(onBtn1);

  WiFi.mode(WIFI_STA);       // radio on, no AP
  if (esp_now_init() != ESP_OK) return;
  esp_now_register_send_cb(onSent);

  esp_now_peer_info_t p = {};
  memcpy(p.peer_addr, peer, 6);
  p.channel = 0;             // = current channel
  p.encrypt = false;
  esp_now_add_peer(&p);
}

void loop() {
  buttons->update();         // may call onBtn1()
  if (!pressed) return;
  pressed = false;
  msg.presses++;
  esp_now_send(peer, (uint8_t *)&msg, sizeof(msg));
}
```

The ESP-NOW : Receiver



```
#include <iot_board.h>
#include <WiFi.h>
#include <esp_now.h>

typedef struct __attribute__((packed)) {
  uint8_t  board;
  uint32_t presses;
} Msg;                      // identical to the sender
Msg last;
uint8_t from[6];
volatile bool fresh = false;

// runs in the Wi-Fi task: copy and leave
void onRecv(const esp_now_recv_info_t *info,
            const uint8_t *data, int len) {
  if (len != sizeof(Msg)) return;   // not ours
  memcpy(&last, data, sizeof(Msg));
  memcpy(from, info->src_addr, 6);
  fresh = true;
}
```

```
void setup() {
  IoTBoard::init_serial();
  IoTBoard::init_leds();
  IoTBoard::init_display();
  WiFi.mode(WIFI_STA);
  WiFi.STA.begin();
  Serial.println(WiFi.macAddress()); // to sender
  esp_now_init();
  esp_now_register_recv_cb(onRecv);
}

void loop() {
  if (!fresh) return;
  fresh = false;
  digitalWrite(LED_GREEN, !digitalRead(LED_GREEN));
  display->clearDisplay();
  display->setCursor(0, 0);
  display->printf("Board %u\n", last.board);
  display->print("Presses ");
  display->println(last.presses);
  display->printf("MAC ..%02X\n", from[5]);
  display->display();
}
```

Which one for your project?

	Wi-Fi (station)	ESP-MESH	painlessMesh	ESP-NOW
Topology	star around the AP	self-healing tree	self-organizing mesh	peer-to-peer, star
Needs an AP	yes	root joins the router	no	no
Network layer	IP	IP through the root	node IDs, no IP	MAC only
Payload	IP packets	IP packets	JSON strings	250 B (1470 B v2)
Setup time	seconds	seconds	seconds	none
Power	high	high: relays always on	high	low: wake, send, sleep
Typical use	cloud, MQTT, OTA	large installations	hobby meshes	remotes, sensors

Common IoT pattern: battery **ESP-NOW sensors** send to one mains-powered **gateway**, which is also a Wi-Fi station and forwards the data to **MQTT**.

The WPAN Networks

Wireless Personal Area Network in short known as WPAN. Actually WPAN is PAN (Personal Area Network) where the interconnected devices are centered around a person's workspace and connected through wireless medium. That's why it is also called as Person's centered short range wireless connectivity.

Typically the range is within about 10 meters means very short range.

IEEE 802.15 has produced standards for several types of PANs operating in the ISM band including **Bluetooth**. The Infrared Data Association (IrDA) has produced standards for WPANs that operate using infrared communications.

The Bluetooth Protocol



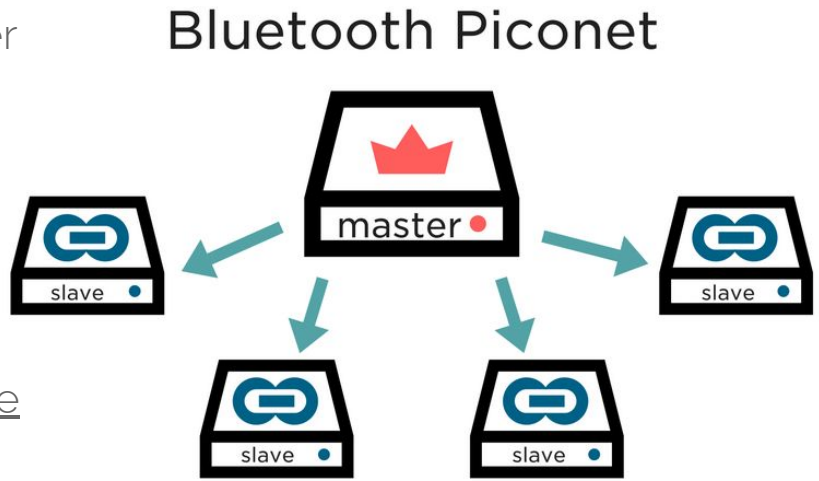
Bluetooth is a short-range wireless technology standard that is used for exchanging data between fixed and mobile devices over short distances. The IEEE standardized Bluetooth as **IEEE 802.15.1**, but no longer maintains the standard.

Traditional Bluetooth is divided up into **3 classes**; 1, 2, and 3 with Class 3 devices having a range of roughly 5m, Class 2 devices having a range of 20-30m, and Class 1 of around 100m. Bluetooth 4.0 simplified things by removing the range limitation from the specification and simply allowing for the signal strength and receiver sensitivity to determine the range, providing that the regulations for radio-frequency transmission power aren't exceeded.

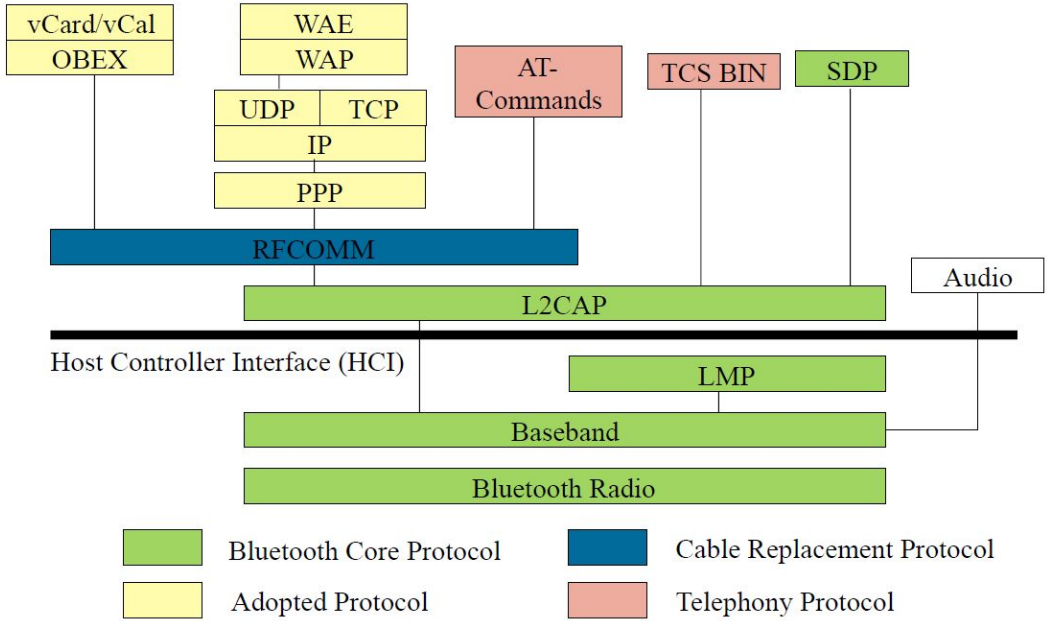
Bluetooth was originally designed to be used as a wireless solution for serial communication such as **UART** which makes it perfect for wireless communication between chips.

The Bluetooth Protocol

Bluetooth **is a packet based master/slave protocol** which allows for up to 7 slave devices to be connected to a single master device. This type of single master network is called a '**piconet**'. A slave device, also known as a peripheral device, **can only communicate with a single master device**, and cannot communicate with other slave devices on the piconet. If it wishes to pass data to and from other slave devices, it must go through the master. A set of Piconet is called **Scatternet** where masters can communicate.



The Bluetooth Protocol



- **Baseband**
  Concerned with connection establishment within a piconet, addressing, packet format, timing and power control.
- **Link Manager Protocol (LMP)**
  Setup links between devices and security aspects.
- **Link Logical Control Adaptation (L2CAP)**
  Adapts upper layer protocol and provides connectionless and connection oriented services.
- **RFCOMM**
  Simulate Serial over radio

The Bluetooth Security

Five basic security services are specified in the Bluetooth standard:

- **Authentication**
  Verifying the identity of communicating devices based on their Bluetooth address. Bluetooth does not provide native user authentication.
- **Confidentiality**
  Preventing information compromise caused by eavesdropping by ensuring that only authorized devices can access and view transmitted data.
- **Authorization**
  Allowing the control of resources by ensuring that a device is authorized to use a service before permitting it to do so.
- **Message Integrity**
  Verifying that a message sent between two Bluetooth devices has not been altered in transit.
- **Pairing/Bonding**
  Creating one or more shared secret keys and the storing of these keys for use in subsequent connections in order to form a trusted device pair.

The Bluetooth on ESP32 - M



```
#include "BluetoothSerial.h"

bool SlaveConnected;
int recatt = 0;
unsigned long previousMillisReconnect;
String myName = "ESP32-BT-Master";
String slaveName = "ESP32-BT-Slave";

String MACadd = "03:B4:16:72:36:9C";
uint8_t address[6] = { 0x03, 0xB4, 0x16, 0x72, 0x36, 0x9C };

BluetoothSerial SerialBT;
```

The Bluetooth on ESP32 - M



```
void Bt_Status(esp_spp_cb_event_t event, esp_spp_cb_param_t *param) {
 if (event == ESP_SPP_OPEN_EVT) {
   Serial.println ("Client Connected");
   SlaveConnected = true;
   recatt = 0;
 }
 else if (event == ESP_SPP_CLOSE_EVT) {
   Serial.println("Client Disconnected");
   SlaveConnected = false;
 }
}

void setup() {
 SlaveConnected = false;
 Serial.begin(115200);
 SerialBT.register_callback(Bt_Status);
 SerialBT.begin(myName, true);
 Serial.printf("The device \"%s\" started in master mode, make sure slave BT device is on!\n", myName.c_str());
 SlaveConnect();
}
```

The Bluetooth on ESP32 - M



```
void SlaveConnect () {
 Serial.println("Function BT connection executed" );
 SerialBT.connect(address);
}
void loop() {
  if (!SlaveConnected) {
   if (millis() - previousMillisReconnect  >= 10000) {
     previousMillisReconnect  = millis();
     recatt++;
     Serial.print("Trying to reconnect. Attempt No.: " );
     Serial.println(recatt);
     Serial.println("Stopping Bluetooth..." );
     SerialBT.end();
     Serial.println("Bluetooth stopped !" );
     Serial.println("Starting Bluetooth..." );
     SerialBT.begin(myName, true);
     Serial.printf("The device  \"%s\" started in master mode, make sure slave BT device is on!  \n", myName.c_str());
     SlaveConnect();
   }
 }
 if (Serial.available()) {
   SerialBT.write(Serial.read());
 }
 if (SerialBT.available()) {
   Serial.write(SerialBT.read());
 }
}
```

The Bluetooth on ESP32 - S



```
#include "BluetoothSerial.h"
unsigned long previousMillis;                                   // BT: Variable used to chage the indication LED state
bool MasterConnected;
String device_name = "ESP32-BT-Slave";
String MACadd = "03:B4:16:72:36:9C";

BluetoothSerial SerialBT;

void Bt_Status (esp_spp_cb_event_t event, esp_spp_cb_param_t *param) {

 if (event == ESP_SPP_SRV_OPEN_EVT) {
   Serial.println ("Client Connected");
   MasterConnected = true;
 }
 else if (event == ESP_SPP_CLOSE_EVT ) {
   Serial.println ("Client Disconnected");
   MasterConnected = false;
 }
}
```

The Bluetooth on ESP32 - S



```
void setup() {

 Serial.begin(115200);
 SerialBT.register_callback (Bt_Status);
 SerialBT.begin(device_name);
}

void loop() {

 if (!MasterConnected) {
   if (millis() - previousMillis >= 500) {
   previousMillis = millis();
   }
 }

 if (Serial.available()) {
   SerialBT.write(Serial.read());
 }
 if (SerialBT.available()) {
   Serial.write(SerialBT.read());
 }
}
```

The BLE Protocol

Bluetooth Low Energy (BLE), sometimes referred to as "**Bluetooth Smart**", is a light-weight subset of classic Bluetooth and was introduced as part of the Bluetooth 4.0 core specification. While there is some overlap with classic Bluetooth, BLE actually has a completely different lineage and was started by Nokia as an in-house project called 'Wibree'.

There are plenty of wireless protocols out there for engineers and product designers, but what makes BLE so interesting is that it's almost certainly the easiest way to design something that can talk to any modern mobile platform out there.

Compared to Classic Bluetooth, **Bluetooth Low Energy is intended to provide considerably reduced power consumption** and cost while maintaining a similar communication range.

The BLE Protocol



GAP is an acronym for the **Generic Access Profile**, and it controls connections and advertising in Bluetooth. GAP is what makes your device visible to the outside world, and determines how two devices can (or can not) interact with each other.

GAP defines various roles for devices, but the two key concepts to keep in mind are **Central** devices and **Peripheral** devices.

- **Peripheral** devices are small, low power, resource constrained devices that can connect to a much more powerful central device. Peripheral devices are things like a heart rate monitor, a BLE enabled proximity tag, etc.
- **Central** devices are usually the mobile phone or tablet that you connect to with far more processing power and memory.

There are two ways to send advertising out with GAP. The **Advertising Data** payload and the **Scan Response** payload.

The BLE Protocol

GATT is an acronym for the **Generic ATTribute Profile**, and it defines the way that two Bluetooth Low Energy devices transfer data back and forth using concepts called **Services** and **Characteristics**.

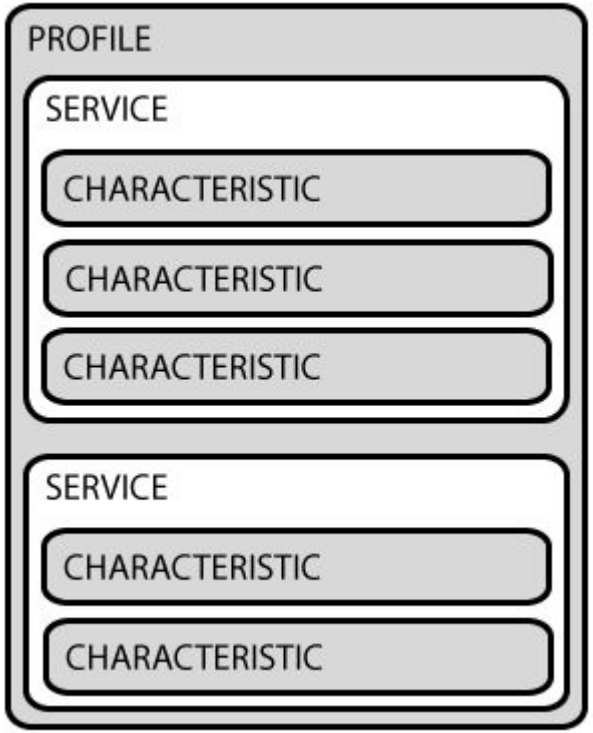
It makes use of a generic data protocol called the **Attribute Protocol (ATT)**, which is used to store Services, Characteristics and related data in a simple lookup table using 16-bit IDs for each entry in the table.

The most important thing to keep in mind with GATT and connections is that **connections are exclusive**. What is meant by that is that a **BLE peripheral can only be connected to one central device** at a time!

The peripheral is known as the GATT Server, which holds the ATT lookup data and service and characteristic definitions, and the **GATT Client** (the phone/tablet), which sends requests to this server.

The BLE Protocol

- A **Profile** does not actually exist on the BLE peripheral itself, it is simply a pre-defined collection of Services that has been compiled by the peripheral designers.
- **Services** are used to break data up into logical entities, and contain specific chunks of data called characteristics. A service can have one or more characteristics, and each service distinguishes itself from other services by means of a unique numeric ID called a UUID.
- **Characteristic** encapsulates a single data point. Similarly to Services, each Characteristic distinguishes itself via a pre-defined 16-bit or 128-bit UUID. Characteristics are the main point that you will interact with your BLE peripheral, so it is important to understand the concept.



The BLE Security

A BLE connection is said to **operate at a specific security mode** for which there are several security levels. The required security mode/level of a connection may change from time to time, leading to procedures to increase that level. Note that each connection starts its lifetime in Security Mode 1, Level 1..

When two devices, which initially do not have security, wish to do something that requires security, the devices must first pair. This process **is triggered by a Central device that is attempting to access a data value (a "characteristic") on a Peripheral device** that requires authenticated access.

Pairing involves authenticating the identity of two devices, encrypting the link using a short-term key (STK) and then distributing long-term keys (LTK) used for encryption. The LTK is saved for faster reconnection in the future, that is termed Bonding.

The encryption in Bluetooth LE is based on 128-bit Advanced Encryption Standard. LTK is used with this algorithm to create the 128-bit "shared secret" key. Authentication is provided in BLE by digitally signing the data using the connection Signature Resolving Key (CSRK). The sending device places a signature after the Data PDU. The receiver verifies the signature using the CSRK.

The BLE Security

For a BLE connection,the Generic Access Protocol (GAP) defines two security modes, along with several security levels per mode.

- **Security Mode 1**
  - Security Level 1: No Security (No authentication and no encryption)
  - Security Level 2: Unauthenticated pairing with encryption
  - Security Level 3: Authenticated pairing with AES-CCM encryption
  - Security Level 4: Authenticated LE Secure Connections pairing with encryption.
- **Security Mode 2**
  - Security Level 1: Unauthenticated pairing with data signing
  - Security Level 2: Authenticated pairing with data signing
- **Mixed Security Mode**
  Mixed Security Mode is when a device is required to support both Security Mode 1 and 2, (e.g. it needs to support signed and unsigned data)

The BLE on ESP32



The ESP32 can act as a BLE server or as a BLE client. For a brief introduction to the ESP32 with BLE on the Arduino IDE, we'll create an ESP32 BLE server, and then an ESP32 BLE scanner to find that server.

Libraries for Server

```
#include <BLEDevice.h>
#include <BLEUtils.h>
#include <BLEServer.h>
```

Service and **Characteristic** (you can generate your UUID from https://uuidgenerator.net)

```
#define SERVICE_UUID        "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define CHARACTERISTIC_UUID "beb5483e-36e1-4688-b7f5-ea07361b26a8"
```

The BLE on ESP32



```
void setup() {
  Serial.begin(115200);
  Serial.println("Starting BLE work!");
  BLEDevice::init("ServerName");
  BLEServer *pServer = BLEDevice::createServer();
  pServer->setCallbacks(new MyServerCallbacks());
    BLEService *pService = pServer->createService(SERVICE_UUID);
    BLECharacteristic *pCharacteristic = pService->createCharacteristic(
                                           CHARACTERISTIC_UUID,
                                           BLECharacteristic::PROPERTY_READ |
                                           BLECharacteristic::PROPERTY_WRITE);
   pCharacteristic->setValue("Value");
   pService->start();
```

The BLE on ESP32



```
  BLEAdvertising *pAdvertising = BLEDevice::getAdvertising();
  pAdvertising->addServiceUUID(SERVICE_UUID);
  pAdvertising->setScanResponse(true);
  pAdvertising->setMinPreferred(0x06);  // functions that help with iPhone
connections issue
  pAdvertising->setMinPreferred(0x12);
  BLEDevice::startAdvertising();
    Serial.println("Characteristic defined!");

}
```

The BLE on ESP32



```
void loop() {
    pCharacteristic->setValue("New Value");
    pCharacteristic->notify();
    delay(1000);
}
class MyServerCallbacks: public BLEServerCallbacks {
  void onConnect(BLEServer* pServer) {
    deviceConnected = true;
  };
  void onDisconnect(BLEServer* pServer) {
    deviceConnected = false;
  }
};
```

The BLE on ESP32

For creating a BLE server, the code should follow the next steps:

- Create a BLE Server. In this case, the ESP32 acts as a BLE server.
- Create a BLE Service.
- Create a BLE Characteristic on the Service.
- Create a BLE Descriptor on the Characteristic.
- Start the Service.
- Start advertising, so it can be found by other devices.

Now let is create a BLE scanner!

Libraries:

```
#include <BLEDevice.h>
#include <BLEUtils.h>
#include <BLEScan.h>
#include <BLEAdvertisedDevice.h>
```

The BLE on ESP32



```
void setup() {
  Serial.begin(115200);
  Serial.println("Scanning...");

  BLEDevice::init("");
  pBLEScan = BLEDevice::getScan(); //create new scan
  pBLEScan->setAdvertisedDeviceCallbacks(new MyAdvertisedDeviceCallbacks());
  pBLEScan->setActiveScan(true); //active scan uses more power, but get results faster
  pBLEScan->setInterval(100);
  pBLEScan->setWindow(99);  // less or equal setInterval value
}
```

The BLE on ESP32



```
void loop() {
  // put your main code here, to run repeatedly:
  BLEScanResults foundDevices = pBLEScan->start(scanTime, false);
  Serial.print("Devices found: ");
  Serial.println(foundDevices.getCount());
  Serial.println("Scan done!");
  pBLEScan->clearResults();   // delete results fromBLEScan buffer to release memory
  delay(2000);
  }

  class MyAdvertisedDeviceCallbacks: public BLEAdvertisedDeviceCallbacks {
      void onResult(BLEAdvertisedDevice advertisedDevice) {
        Serial.printf("Advertised Device: %s \n", advertisedDevice.toString().c_str());
      }};
```

The BLE on ESP32



```
#include "BLEDevice.h"
#define bleServerName "ServerName"
static BLEUUID serviceUUID("4fafc201-1fb5-459e-8fcc-c5c9c331914b");
static BLEUUID characteristicUUID("beb5483e-36e1-4688-b7f5-ea07361b26a8");
static BLERemoteCharacteristic* characteristic;
char* charValue;
static boolean doConnect = false;
static boolean connected = false;
static BLEAddress *pServerAddress;
```

The BLE on ESP32



```
void setup() {
      BLEDevice::init("");
      BLEScan* pBLEScan = BLEDevice::getScan();
      pBLEScan->setAdvertisedDeviceCallbacks(new MyAdvertisedDeviceCallbacks());
      pBLEScan->setActiveScan(true);
      pBLEScan->start(30);
}

class MyAdvertisedDeviceCallbacks: public BLEAdvertisedDeviceCallbacks {
  void onResult(BLEAdvertisedDevice advertisedDevice) {
    if (advertisedDevice.getName() == bleServerName) {
      advertisedDevice.getScan()->stop();
      pServerAddress = new BLEAddress(advertisedDevice.getAddress());
      doConnect = true;
      Serial.println("Device found. Connecting!");
      }
  }
};
```

The BLE on ESP32



```
void loop() {
  if (doConnect == true) {
    if (connectToServer(*pServerAddress)) {
      Serial.println("We are now connected to the BLE Server.");
      characteristic->getDescriptor(BLEUUID((uint16_t)0x2902))->writeValue((uint8_t*)notificationOn, 2,
true);
      connected = true;
    } else {
      Serial.println("We have failed to connect to the server.");
    }
    doConnect = false;
  }
  delay(1000);
}
```

The BLE on ESP32



```
bool connectToServer(BLEAddress pAddress) {
   BLEClient* pClient = BLEDevice::createClient();

  // Connect to the remove BLE Server.
  pClient->connect(pAddress);
  Serial.println(" - Connected to server");

  // Obtain a reference to the service we are after in the remote BLE server.
  BLERemoteService* pRemoteService = pClient->getService(serviceUUID);
  if (pRemoteService == nullptr) {
    Serial.print("Failed to find our service UUID: ");
    Serial.println(serviceUUID.toString().c_str());
    return (false);
  }

  // Obtain a reference to the characteristics in the service of the remote BLE server.
  characteristic = pRemoteService->getCharacteristic(characteristicUUID);

  if (characteristic == nullptr) {
    Serial.print("Failed to find the characteristic UUID");
    return false;
  }
  Serial.println(" - Found characteristic");

  //Assign callback functions for the Characteristic
  characteristic->registerForNotify(notifyCallback);
  return true;
}
```

```
static void notifyCallback(BLERemoteCharacteristic*
pBLERemoteCharacteristic, uint8_t* pData, size_t length, bool isNotify)
{
  //store value
  charValue = (char*)pData;
}
```

The 802.15.4 Protocol

IEEE 802.15.4 is a standard that was developed to provide a framework and the lower layers in the OSI model for low cost, low power wireless connectivity networks. IEEE 802.15.4 provides provides the MAC and PHY layers, leaving the upper layers to be developed for specific higher later standards.

IEEE 802.15.4 does not take the limelight in the way that other standards might, but nevertheless it forms the basis for a large number of standards and accordingly it is far more widely deployed than may be apparent at first sight.

The IEEE 802.15.4 standard is aimed at providing the essential lower network layers for a WPAN. The chief **requirements are low-cost, low-speed ubiquitous communication between devices**.

The concept of IEEE 802.15.4 is to provide communications over distances up to about 10 metres and with maximum transfer data rates of **250 kbps**.

The 802.15.4 Protocol

The purpose of the IEEE 802.15.4 MAC layer is to **provide an interface between the PHY or physical layer and the application layer**. The as IEEE 802.15.4 does not specify an application layer, this is generally an application system.

The IEEE 802.15.4 MAC provides the interface to the application layer using two elements:

- **MAC Management Service**: This is called the MAC Layer Management Entity, MLME. It provides the service interfaces through which layer management functions may be called or accessed.
- **MAC Data Service**: This is called the MAC Common Port Layer (MCPS). This entity provides data transport services between the peer MACs.

The 802.15.4 Protocol



There are different types of devices that can exist in a network. There are three types:

- **FFD**: Full Function Device - a node that has full levels of functionality. It can be used for sending and receiving data, but it can also route data from other nodes.
- **RFD** : Reduced Function Device - a device that has a reduced level of functionality. Typically it is an end node which may be typically a sensor or switch. RFDs can only talk to FFDs as they contain no routing functionality. These devices can be very low power devices because they do not need to route other traffic and they can be put into a sleep mode when they are not in use.
- **Coordinator**: This is the node that controls the IEEE 802.15.4 network. **This is a special form of FFD**. In addition to the normal FFD functions it also sets the IEEE 802.15.4 network up and acts as the coordinator or manager of the network.

These definitions were originally generated for use in **Zigbee**, but their use has now been introduced with IEEE 802.15.4 network terminology.

The 802.15.4 Protocol

There are two main forms of network topology that can be used within IEEE 802.15.4. These network topologies may be used for different applications and offer different advantages.

- **Star Topology**
  In the star topology, all the different nodes are required to talk only to the central PAN coordinator. Even if the nodes are FFDs and are within range of each other, in a star network topology, they are only allowed to communicate with the coordinator node. Having a star network topology does limit the overall distances that can be covered. It is limited to one hop.
- **P2P/ Mesh Topology**
  A p2p or mesh network topology provides a number of advantages over a star network topology. In addition to communication with the network coordinator, **devices are also able to communicate with each other**. FFDs are able to route data, while the RFDs are only able to provide simple communication.

The 802.15.4 Protocol

Several established frame types are used by the MAC layer to accomplish these functions. In 802.15.4, there are four different types of MAC frames.

Octets:2	1	0/2	0/2/8	0/2	0/2/8	variable	2
Frame control	Sequence number	Destination PAN identifier	Destination address	Source PAN identifier	Source address	Frame payload	Frame check sequence
		Addressing fields					
MAC header						MAC payload	MAC footer

Bits: 0-2	3	4	5	6	7-9	10-11	12-13	14-15
Frame type	Sequrity enabled	Frame pending	Ack. Req.	Intra PAN	Reserved	Dest. addressing mode	Reserved	Source addressing mode

The 802.15.4 Protocol

- **Beacon Frame**
  Used by a Coordinator

Octets:2	1	4 or 10	2	variable	variable	variable	2
Frame control	Beacon sequence number	Source address information	Superframe specification	GTS fields	Pending address fields	Beacon payload	Frame check sequence
MAC header			MAC payload				MAC footer

Bits: 0-3	4-7	8-11	12	13	14	15
Beacon order	Superframe order	Final CAP slot	Battery life extension	Reserved	PAN coordinator	Association permit

The 802.15.4 Protocol

- **MAC Command Frame**
  Used for handling all MAC peer entity control transfer (data request , etc)

Octets:2	1	4 to 20	1	variable	2
Frame control	Data sequence number	Address information	Command type	Command payload	Frame check sequence
MAC header			MAC payload		MAC footer

Command Types: Association Request and Response, Disassociation Notification, Data Request, PAN ID Conflict, Orphan Notification, Beacon Request, Coordinator realignment.

The 802.15.4 Protocol

- **Data Frame**
  Used for all transfer of data

Octets:2	1	4 to 20	variable	2
Frame control	Data sequence number	Address information	Data payload	Frame check sequence
MAC header			MAC Payload	MAC footer

The 802.15.4 Protocol

- **ACK Frame**
  Used for confirming successful frame reception

Octets:2	1	2
Frame control	Data sequence number	Frame check sequence
MAC header		MAC footer

The 802.15.4 Security

The physical medium is accessed through a **CSMA/CA** protocol. Networks which are not using beaconing mechanisms utilize an unslotted variation which is based on the listening of the medium, leveraged by a random exponential backoff algorithm.

if a device is unable to process a frame at a given time, it simply does not confirm its reception; timeout-based retransmission can be performed a number of times, following after that a decision of whether to abort or keep trying.

Regarding secure communications, the MAC sublayer offers facilities which can be harnessed by upper layers to achieve the desired level of security. Higher-layer processes may specify keys to perform symmetric cryptography to protect the payload and restrict it to a group of devices or just a point-to-point link; these groups of devices can be specified in access control lists. Furthermore, **MAC computes freshness checks between successive receptions to ensure that presumably old frames**, or data which is no longer considered valid, does not transcend to higher layers.

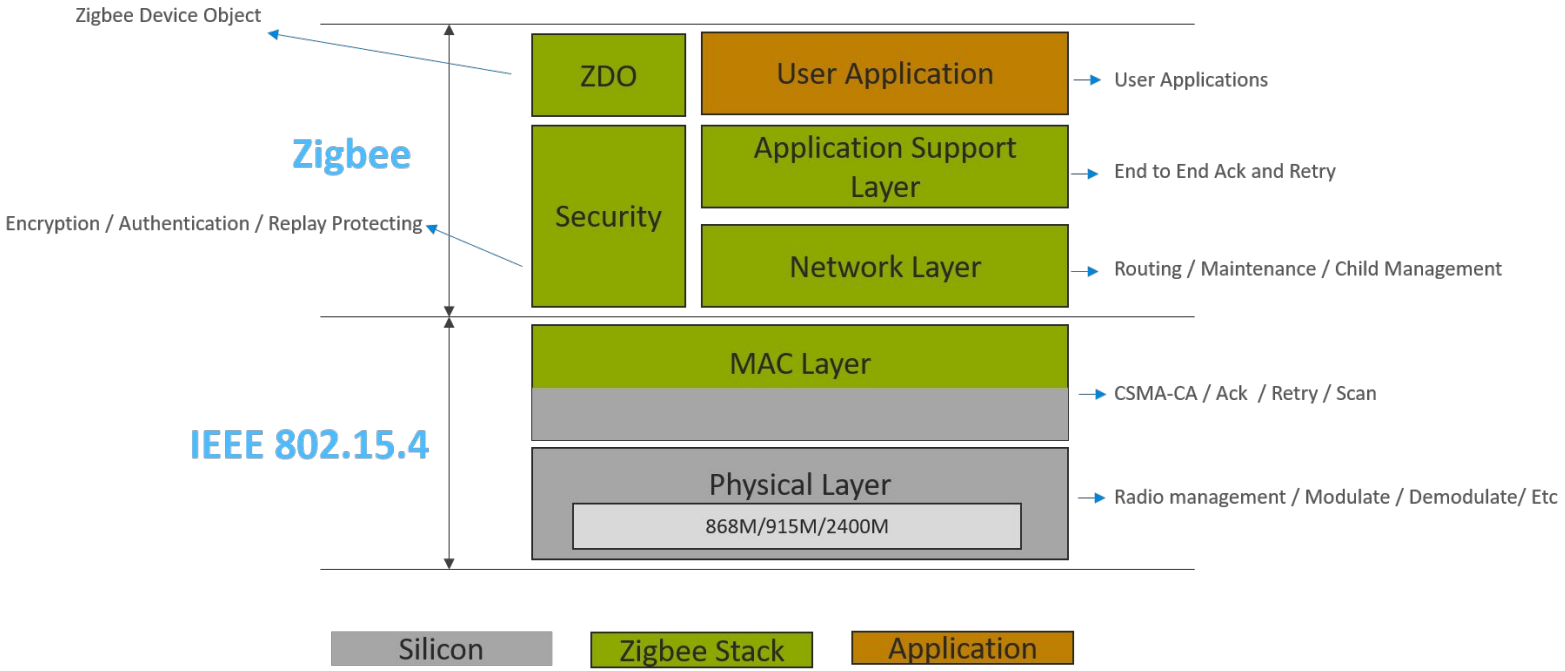
In addition to this secure mode, **there is another, insecure MAC mode**, which allows access control lists merely as a means to decide on the acceptance of frames according to their (presumed) source.

The ZigBee Protocol

The **Zigbee standard is a open standard**, developed and maintained by an industry alliance called the Zigbee Alliance, built on top of IEEE 802.15.4 which provides the upper layers for control and sensor applications. It has been designed to be very robust so that it can operate reliably in harsh radio environments, providing security and flexibility.



The ZigBee Protocol



The ZigBee Protocol



- **Physical layer** and **MAC layer** are defined by IEEE 802.15.4. Physical layer is responsible for radio management.
- **Network layer** is responsible for message transmitting and receiving, device maintenance, routing , etc.
- **Application Support Layer (APS)**, is in charges of end to end message transmitting.
- **Application layer** is left for user design. Each application instance is called an endpoint. A special endpoint, which is endpoint 0, is reserved for management functionalities. We also call this management functionalities model as **Zigbee Device Object (ZDO)**.
- In **APS layer** and **network layer**, there are some security features which are used to protect the network from being hacked.

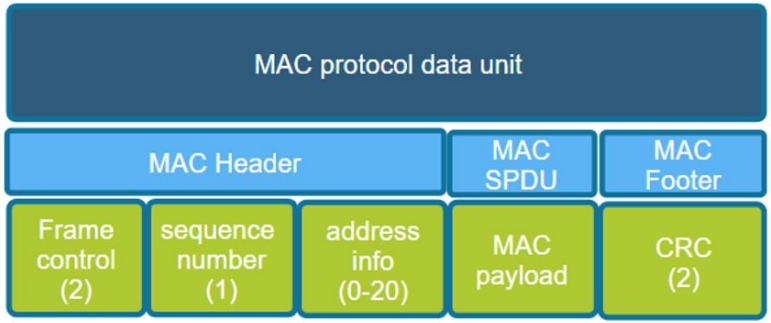
The ZigBee Protocol

- **MAC Layer**

At the end of each MAC frame, there are **two bytes CRC** used to verify the integrity of the packet. In MAC header, there is a 2-bytes field "Frame Control". Bit0~2 of field "Frame Control" indicates the frame type.



- 4 MAC Frames
  - Beacon
  - Data
  - ACK
  - MAC Command



The ZigBee Protocol

- **Network Layer**
  Here we will introduce some basic concepts of Zigbee network, including:
  - Device Type
    - Coordinator, Router **(FFD)**
    - End Device (non-sleepy end device and sleepy end device) (**RFD**)
  - Network Address
    - PAN ID (16 bit)
    - Extended PAN ID (64 bit)
  - Device Address
    - Short (Node ID 16 bit)
    - Long (MAC address 64 bit)

The ZigBee Protocol



- **Application Layer**
  In application layer, a physical device can be split to several logic devices by implementing multiple endpoints. The endpoint ID is a 8 bit value, ranging from 0 to 255.
  - Endpoint 0 is **reserved** for Zigbee Device Object, mainly used for management purpose.
  - Endpoint 1 to 239 can be used by user applications.
  - Endpoint 240 to 254 are **reserved** for special applications. Like Zigbee Green Power use dedicate endpoint 242.
  - Endpoint 255 is used for **broadcasting**.

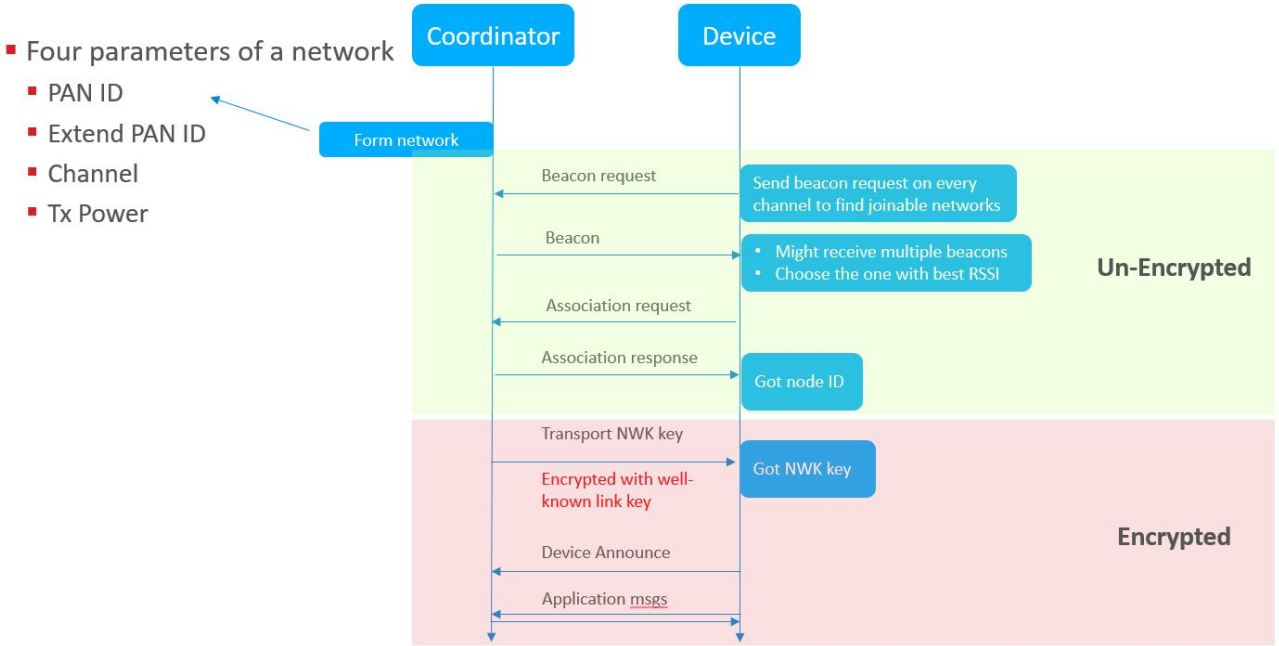
The ZigBee Protocol

- **Joining**
  First, the coordinator forms a network. To form a Zigbee network, you have to prepare 4 parameters:
  - **PAN ID**
  - **Extend PAN ID**
  - **The working channel**
  - **Transmit power**

  You need to specify these four parameters. If you don't, the coordinator will randomly choose a PAN ID and an extended PAN ID. If you don't specify a channel, the coordinator will scan and pick a relatively quiet channel to work on.

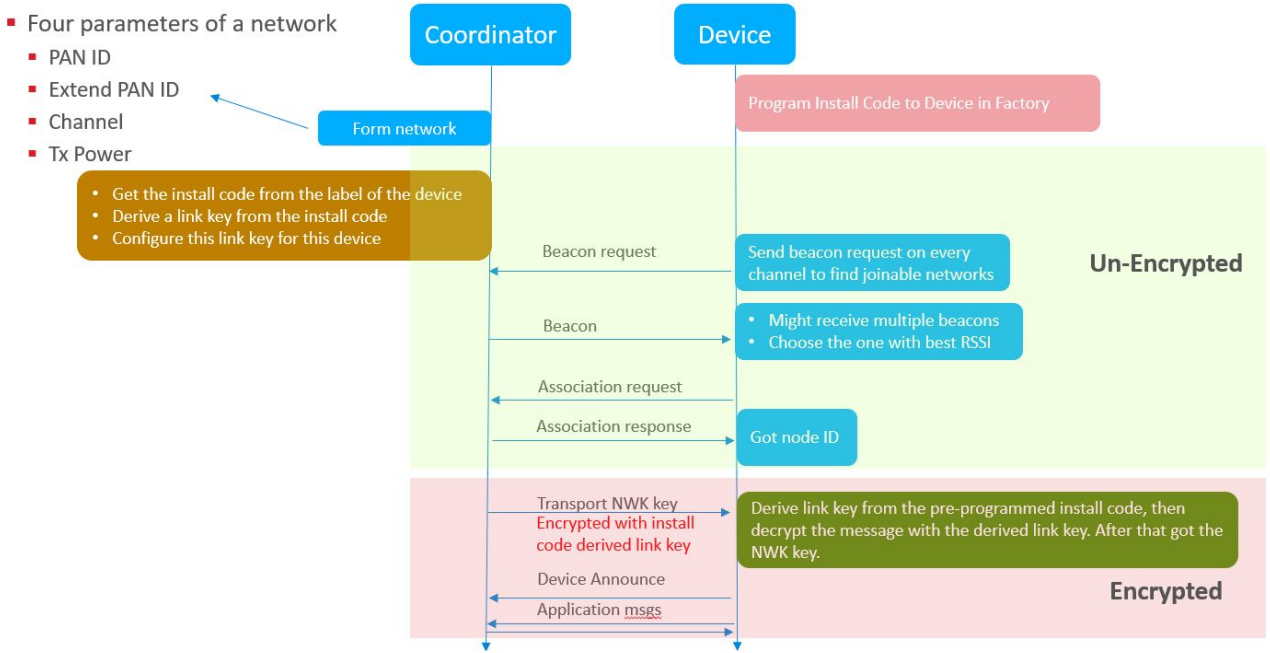
The ZigBee Protocol

- Joining (Well-Known Link Key)

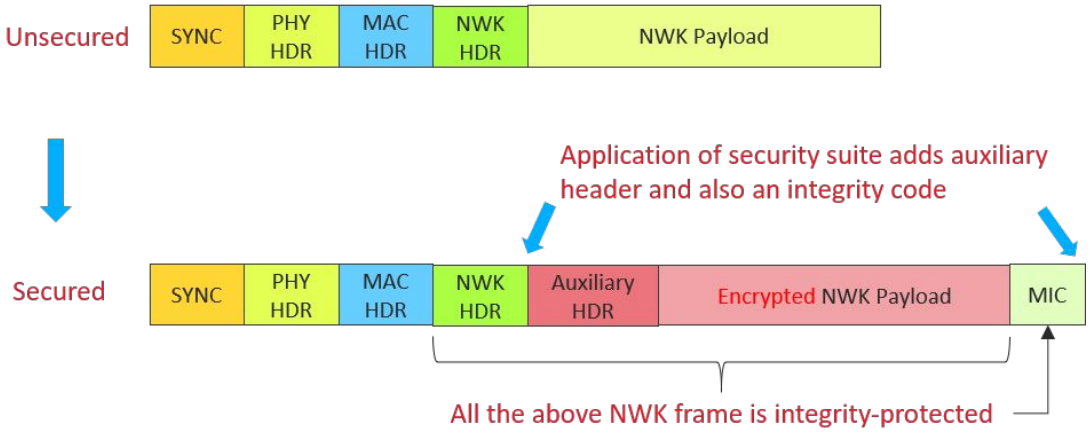


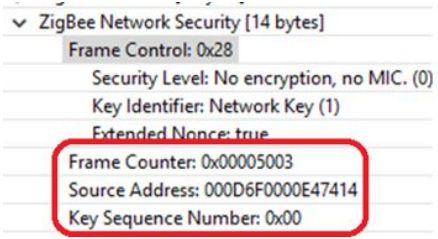
The ZigBee Protocol

- Joining (Install-Code Derived Link Key)



The ZigBee Security





The ZigBee Security

First, **the network payload will be encrypted**. After that, an security header will be added before the encrypted payload. Then calculate a hash value from the network header, security header and the encrypted payload. Finally appended the 32-bit hash value to the end of the frame. If any byte of the network header, security header and encrypted payload is changed, the hash value will be different. We call this value **MIC**, short for **message integrity check**.

The network encryption uses a symmetric encrypting algorithm (AES128), which means the **same key is used for encryption and decryption**. This key is called **network key**. As it's a symmetric encrypting algorithm, all devices in the same Zigbee network will use the same network key.

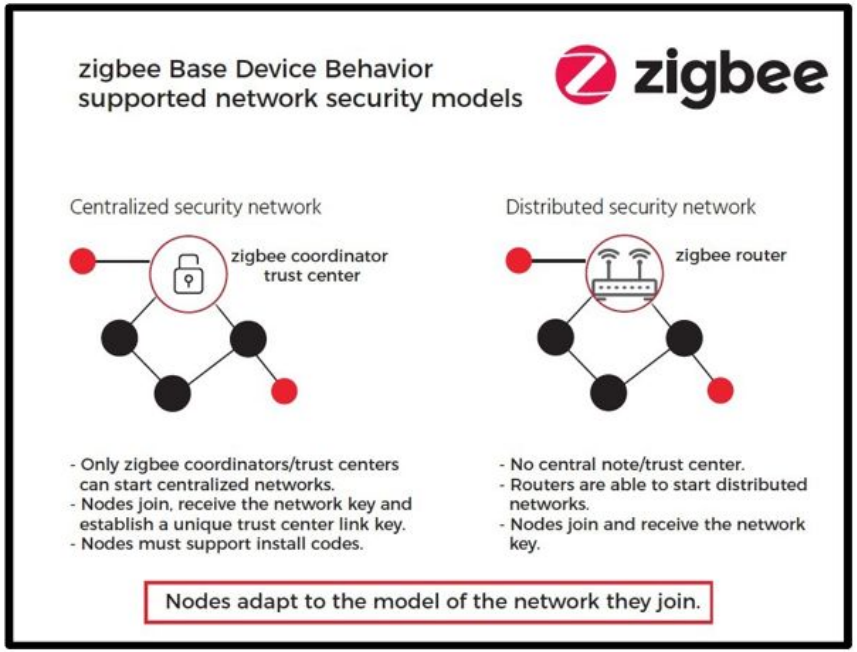
In the network security header, a field named "**frame counter**" and the source Eui64 of the node who encrypt the message are **added to protect replay attack**. A key sequence number is added to support network key updating.

Network layer security is a hop-by-hop security.

The ZigBee Security

Network key is a 16 bytes octets. Normally it's **randomly generated by coordinator** when the network is formed. When new devices join network, **they must get a copy of the network key.**

In Zigbee network, **the role who distribute network key to new devices is called trust center**. There are two typical security models, centralized security network and distributed security network.

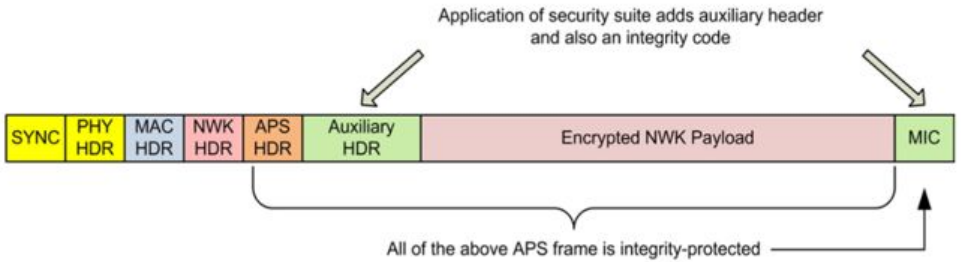


The ZigBee Security

In a centralized security network, **there is only one trust center**, normally it's the coordinator. All new devices will get network key from coordinator.

In a distributed security network, **every router is a trust center**. New devices can get network key from every router.

As the network key needs to be transported from one device to another, **the key value needs to be encrypted during transporting**. This encryption is done in application layer.



The ZigBee Security

It's quite similar to network security. **Also symmetric encrypting algorithm (AES128) is used**. The key is called **link key**. For most case, only the transporting network key message needs to be encrypted in application layer, and this only happens between trust center and the new device. So in this case, we also call it as trust center link key.

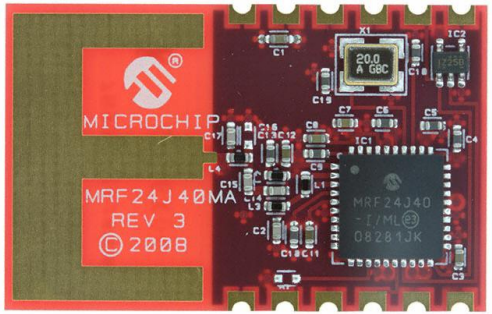
APS layer security is an end-to-end security because only the two peers which participate the communication know the link key. Devices in the network can use the same link key or different link key. If the devices use the same link key, this key is a global link key.

Zigbee defines an approach to configure the link key out of band. It's the **install code**.

The ZigBee on ESP32

There are several zigbee modules, in our case we will use the MRF24J40MA module.

You can check its documentation in this PDF: https://ww1.microchip.com/downloads/aemDocuments/documents/WSG/ProductDocuments/DataSheets/MRF24J40-Data-Sheet-DS30009776E.pdf



There are also several libraries for this module, we will use the implementation of Nanoshield Library but you can use any library.

The ZigBee on ESP32



```
#include <iot_board.h>
int count = 0;
int count_sent = 0;
const char *str;

void setup(){
   IoTBoard::init_serial();
   IoTBoard::init_buttons();
   IoTBoard::init_leds();
   IoTBoard::init_display();
   display->println("Display enabled");
   display->display();

   IoTBoard::init_spi();
   display->println("SPI (HSPI) enabled");
   display->display();
   ...
   ...
   IoTBoard::init_zigbee();
   zigbee->setAddress(0x1);
   zigbee->setPanId(0x8888);
   zigbee->setChannel(12);
   display->printf("Zigbee enabled (%04X)\n",
zigbee->getAddress());
   display->display();
   delay(500);
}
```

We need to initialise the SPI interface for the Zigbee module!

The ZigBee on ESP32



```
void loop()
{
    String s = "Hello " + String(count_sent);
    zigbee->startPacket();
    zigbee->writeString((char *)s.c_str());
    zigbee->sendPacket(0xffff, false);
    while (!zigbee->transmissionSuccess()){
        delay(1);
    }
    count_sent = count_sent +1;
    if (zigbee->receivePacket()){
        str = zigbee->readString();
        count++;
    }
...
```

```
...
    if (str != nullptr){
        display->clearDisplay();
        display->setCursor(0, 0);
        display->printf("[%04X] Received:\n", zigbee->getAddress());
        display->println(str);
        display->print("LQI/RSSI=");
        display->print(zigbee->getRXPacket()->lqi, DEC);
        display->print("/");
        display->println(int(zigbee->getRXPacket()->rssi), DEC);
        display->printf("Source: %04X\n", zigbee->getRXPacket()->src_addr);
        display->printf("Dest: %04X\n", zigbee->getRXPacket()->dest_addr);
        display->printf("Count: %d \n", count);
        display->printf("Sent Count: %d \n", count_sent);
        display->display();
    }
    delay(500);
}
```

The LWPAN Networks

Low Power Wide Area (LoWPAN or LPWA) Networks represent a novel communication paradigm, which will complement traditional cellular and short range wireless technologies in addressing diverse requirements of IoT applications. There are three technical features of LPWAN that meet the requirements of IoT, these are:

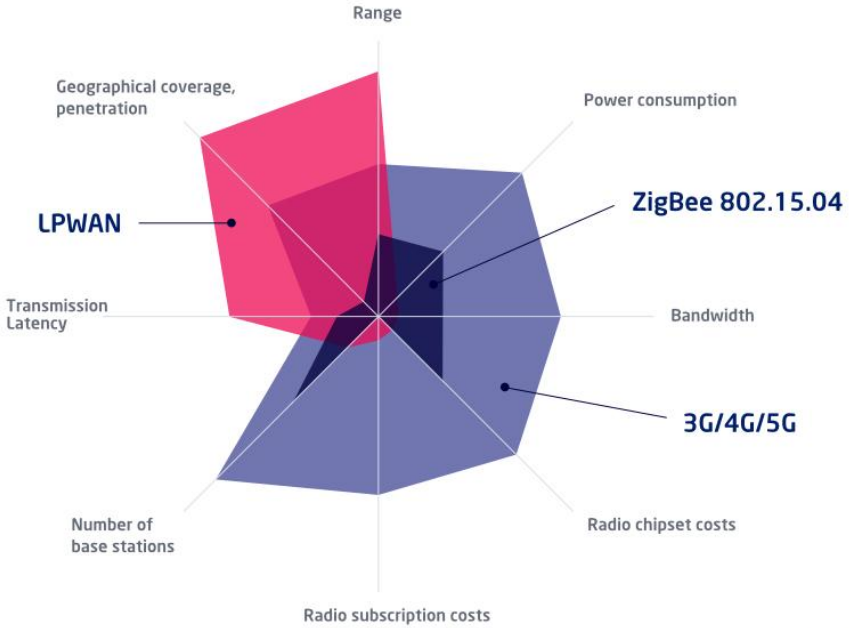
- **Geographical range:**
  LPWAN is designed for wireless data transport between devices separated by distances in the range of kilometers and not meters.
- **The amount of data transmitted:**
  LPWAN's idea is to regulate the non-constant transport of small amounts of data.
- **Low power consumption:**
  The protocol is based on the use of devices with batteries that last for years instead of weeks and months.

The LWPAN Networks

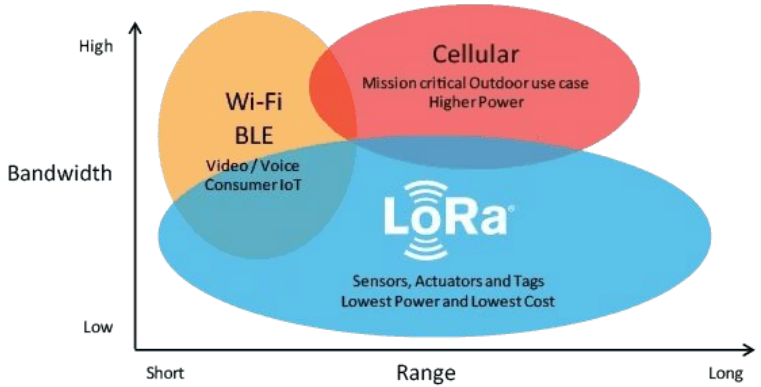
The disadvantages that can be pointed out are the following:

- The low transmission **speed does not allow handling large volumes of data**. LPWAN allows the creation of sensor and device networks, however, we must say that even in these networks the volumes of telemetry data that can be transported cannot be very large, which rules out very complex elements or those that require a lot of information.
- There are reports of signal attenuation problems when the LPWAN network includes devices located in buildings or separated by physical elements, operating more efficiently in open spaces with secured lines of sight.
- **Connectivity between the devices and its application or server is not constant and can be unidirectional**; from the device to a capture element, which makes it difficult to control the movement of objects in real time.However, it should be noted that most LPWAN implementations allow two-way communications.

The LPWAN Networks



source: techplayon



The LoRa Protocol

LoRa (which stands for Long Range) is a patented wireless communication technology which combines ultra-low power consumption with an effective long range. While range highly depends on the environment and possible obstructions (LOS or N-LOS),

LoRa typically has a range between 13- 15Km, which means a single LoRa gateway can provide coverage for an entire city, and with a couple more, a whole country.

The technology was developed by Cycleo in France and came to fore when the company was acquired by **Semtech** in 2012.

A LoRa radio comprise of a few features which help it achieve long-range effective power and Low cost.

The LoRa Protocol

- **Modulation**
  Lora radios **use the chirp spread spectrum modulation** technique to achieve a significantly high communication range while maintaining low power characteristics that are similar to the FSK modulation physical layer based radios. While chirp spread spectrum modulation has been around for a while with applications in military and space communications, LoRa presents the first, low-cost commercial application of the modulation technique.

- **Frequency**
  While the LoRa technology is frequency agnostic, Communication between LoRa radios happen via the use of unlicensed sub-GHz radio frequency bands that are available around the world. These frequencies vary from region to region and often also differ between countries. For instance the 868MHz is commonly used for LoRa communications in Europe, while the 915MHz is used in North America.

The LoRa Protocol

- **Adaptive Data Rate**
  LoRa uses a combination of variable bandwidth and **spreading factors (SF7-SF12)** to adapt the data rate in a trade-off with the range of the transmission. Higher spreading factor allows longer range at the expense of lower data rate, and vice versa. The combination of bandwidth and spreading factor can be chosen according to the link conditions and the level of data to be transmitted. Thus, a higher spreading factor improves transmission performance and sensitivity for a given bandwidth, but it also increases transmission time as a result of lower data rates.
- **Adaptive Power Level**
  The power level used by LoRa radios is adaptive. **It is dependent on factors like the data rate and link conditions among others**. When a fast transmission is required, the transmitted power is pushed closer to the maximum and vice versa. Thus, battery life is maximized and network capacity maintained. Power consumption also depends on the class of devices among several other factors.

The LoRa on ESP32



```
#include <iot_board.h>
int count = 0;
int count_sent = 0;
bool displayNeedUpdate =false;

uint16_t localAddress = 0x01;      // address of this device
uint16_t destination = 0xFF;       // destination to send to
void onLoRaSend() {
 Serial.printf("LoRa Send: %d \n", count_sent);
}

boolean runEvery(unsigned long interval){
 static unsigned long previousMillis = 0;
 unsigned long currentMillis = millis();
 if (currentMillis - previousMillis >= interval){
   previousMillis = currentMillis;
   return true;
 }
 return false;
}

void onBtn1Released(uint8_t pinBtn){
   digitalWrite(LED_GREEN, !digitalRead(LED_GREEN));
}
```

```
void onLoRaReceive(int packetSize){
   if (packetSize == 0)
       return; // if there's no packet, return
   uint16_t recipient = lora->read();       // recipient address
   uint16_t sender = lora->read();          // sender address
   byte incomingMsgId = lora->read();  // incoming msg ID
   byte incomingLength = lora->read(); // incoming msg length
   String incoming = ""; // payload of packet
   while (lora->available()){
       incoming += (char)lora->read(); // add bytes one by one
   }
   if (incomingLength != incoming.length()){ // check length for error
       Serial.println("error: message length does not match length");
       return; // skip rest of function
   }
   count++;
   display->clearDisplay();
   display->setCursor(0,0);
   display->printf("From: %04X\n", sender);
   display->printf("Sent to: %04X\n", recipient);
   display->printf("Message ID: %d\n", incomingMsgId);
   display->printf("Count: %d\n", count);
   display->printf("Message length: %d\n", incomingLength);
   display->printf("Message: %s\n", incoming);
   display->printf("RSSI: %d\n", lora->packetRssi());
   display->printf("Snr: %02f\n", lora->packetSnr());
   displayNeedUpdate = true;
}
```

The LoRa on ESP32



```
void setup(){
    IoTBoard::init_serial();
    IoTBoard::init_buttons();
    IoTBoard::init_leds();
    IoTBoard::init_display();
    display->println("Display enabled");
    display->display();

    IoTBoard::init_spi();
    display->println("SPI (HSPI) enabled");
    display->display();

    if (IoTBoard::init_lora()){
        lora->onReceive(onLoRaReceive);
        lora->receive();
        display->println("LoRa enabled");
    }
    else{
        display->printf("LoRa Error");
    }

    display->display();
    delay(500);
}
```

```
void loop()
{
    String s = "Hello " + String(count_sent);
    lora->beginPacket();

    lora->write(destination);              // add destination address
    lora->write(localAddress);             // add sender address
    lora->write(count_sent);                 // add message ID
    lora->write(s.length());        // add payload length
    lora->write((const uint8_t *)s.c_str(), s.length()); // add payload

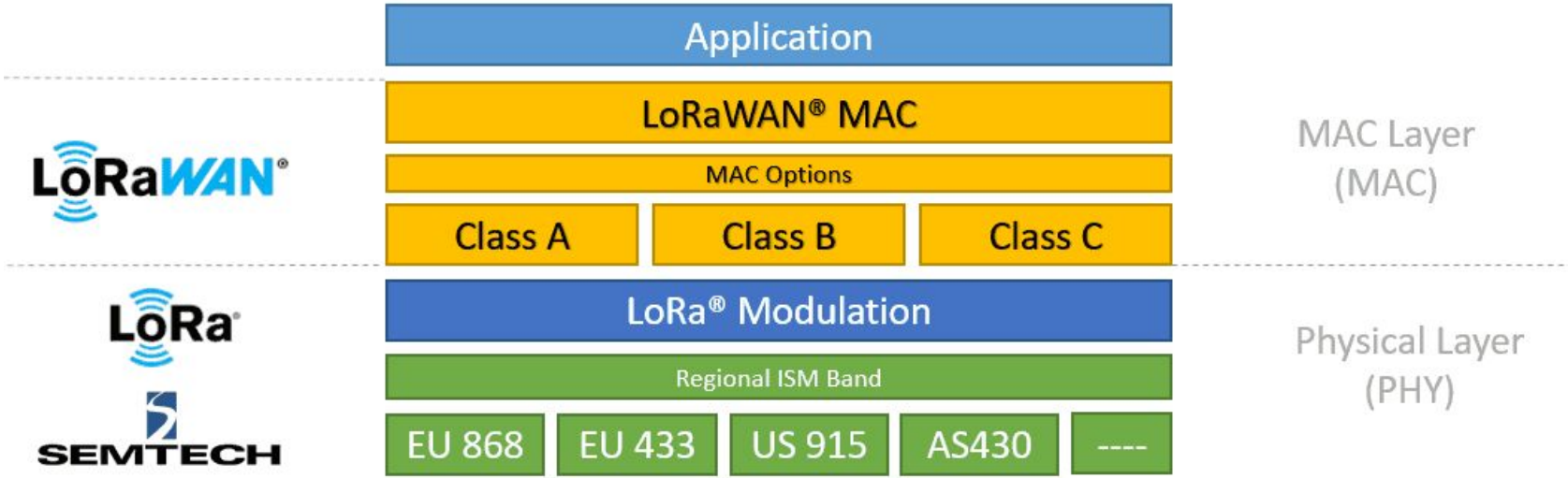
    lora->endPacket(); // true = async / non-blocking mode
    count_sent++;
    lora->receive();
    if(displayNeedUpdate){
        displayNeedUpdate =false;
        display->display();
    }
    delay(5000);
}
```

The LoRaWAN Protocol

LoRaWAN is a high capacity, Long Range, open, Low Power Wide Area Network (LPWAN) standard designed for LoRa Powered IoT Solutions by the **LoRa Alliance**. It is a bi-directional protocol which takes full advantage of all the features of the LoRa technology to deliver services including reliable message delivery, end to end security, location and multicast capabilities. **The standard ensures the interoperability of the various LoRaWAN networks world-wide**.

There is usually a mix up when people try to define LoRa and LoRaWAN which is probably best solved by examining the OSI reference stack Model.

The LoRaWAN Protocol



The LoRaWAN Protocol

LoRaWAN corresponds to Media Access protocol for the communication network while LoRa corresponds to the Physical layer. Thus LoRaWAN defines the communication protocol and system architecture for the network, while LoRa architecture enables the long-range communication link.

The two of them merged together to provide the functionality that determines **battery life of a node, the network capacity, the quality of service, the security and other applications served by the network**.

While LoRaWAN is the most popular MAC layer for LoRa other proprietary layers which are also built on the LoRa technology exists.

The LoRaWAN Protocol

LoRaWAN networks are usually deployed in a **star-of-stars topology.**

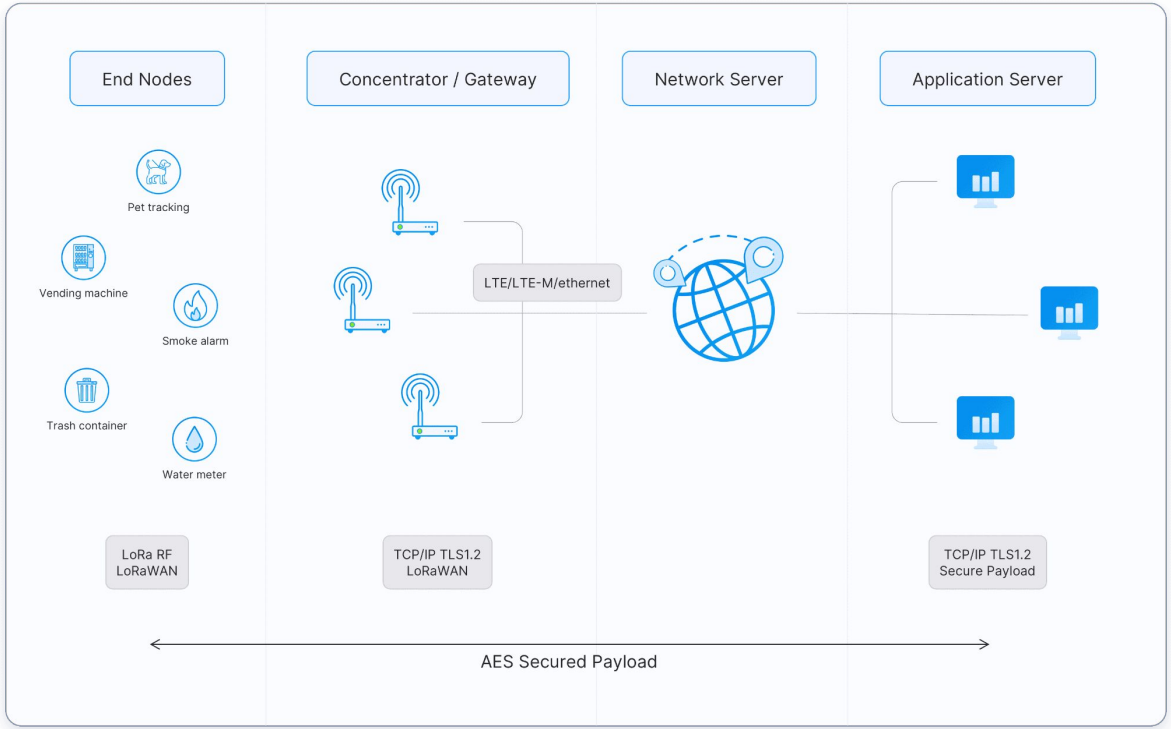
This means that gateways manage data between end-devices and a **network server**.

Gateways are connected to the central network server via the Internet, while **end-devices** use LoRa to send and receive data to and from the gateways

End-devices are not exclusively tied to a single gateway, end-devices broadcast information to all the gateways in range.

Communication in LoRaWAN networks is natively **bi-directional**, although uplink communication between end-devices and the central network server is expected to be predominant in the network.

The LoRaWAN Protocol



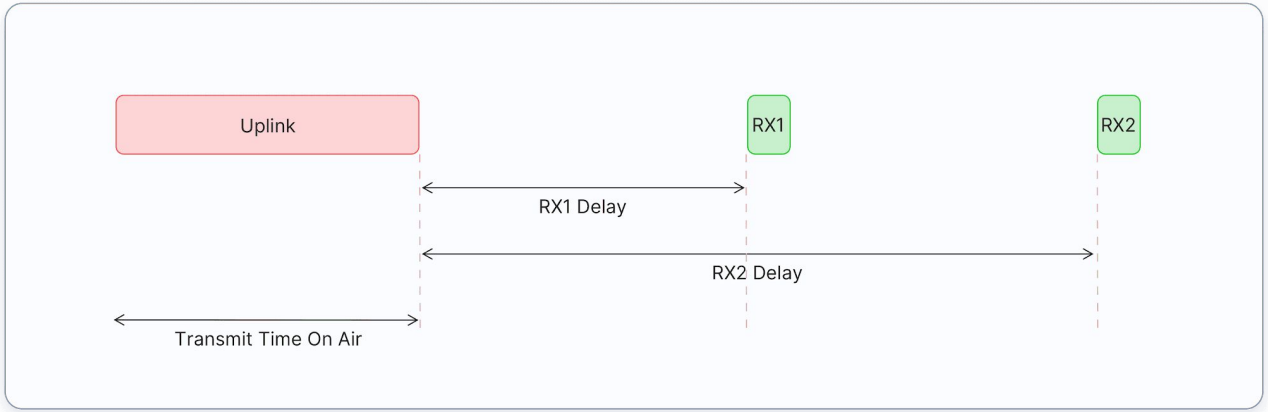
The LoRaWAN Protocol

- **End-Device**
  These are sensors or actuators at the network edge. End-devices serve different applications and have different requirements. In order to optimize a variety of end application profiles, **LoRaWAN utilizes three different device classes to which end-devices can be configured as**. The classes feature trade offs between downlink communication latency and battery life of the device.
  The three major classes are:

  - **Class A End-Devices**
  - **Class B End-Devices**
  - **Class C End-Devices**

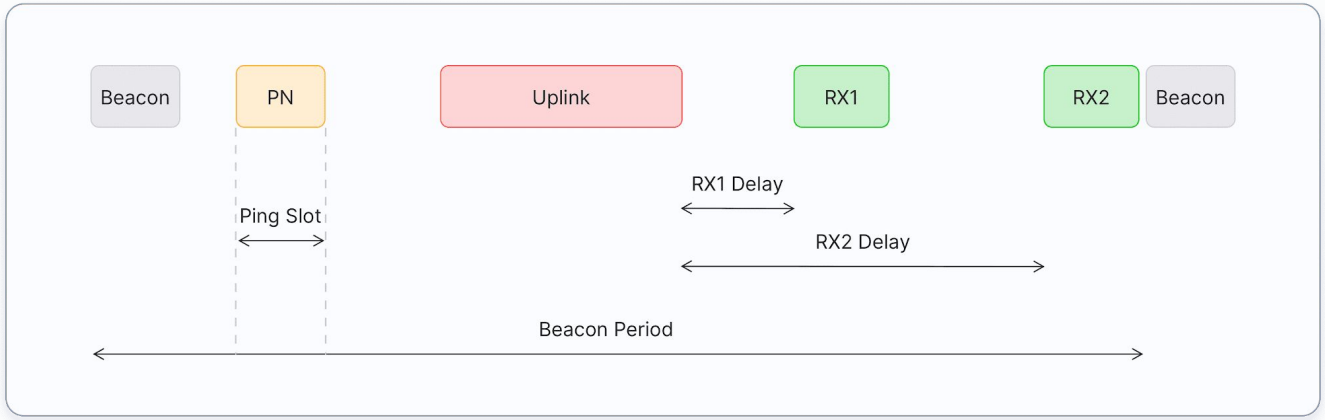
The LoRaWAN Protocol

- **Class A End-Devices (Aloha)**
  Class A devices implement a **bi-directional communication** profile where two short downlinks follow the end-device uplink transmission receive windows, usually referred to as **RX1** and **RX2**. If the server does not respond in either RX1 or RX2 windows, the next opportunity will be after the next uplink transmission. Class A devices are often battery-powered and spend most of the time in sleep mode.
  A good example of class A devices is a LoRa based Smart Energy Meter



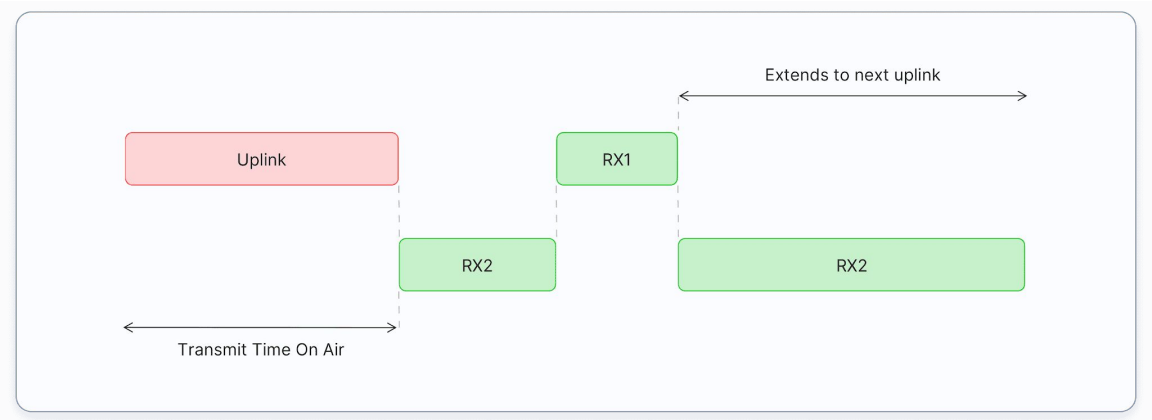
The LoRaWAN Protocol

- **Class B End-Devices (Beaconing)**
  Class B devices extend Class A devices by adding scheduled receive windows for downlinks, and, therefore, they **emulate a continuously receiving device by opening receive windows at fixed time intervals**. This class should be implemented when low latency of downlink communication while keeping the power consumption as low as possible is required.

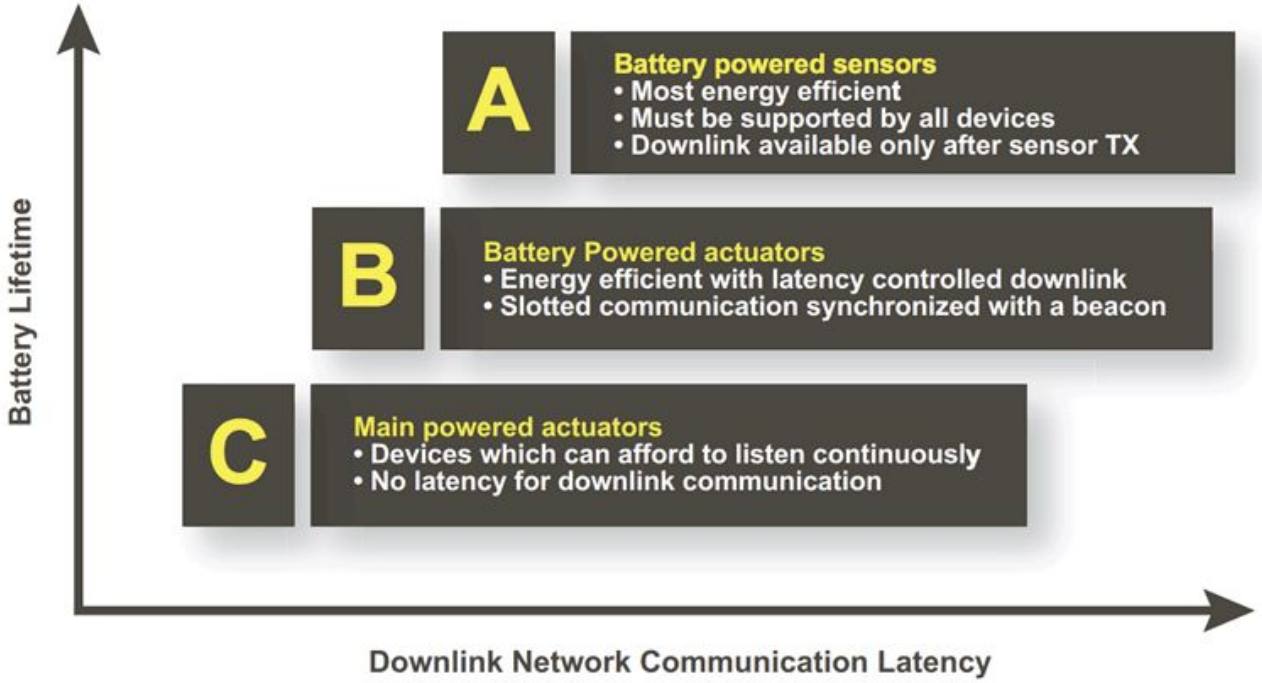


The LoRaWAN Protocol

- **Class C End-Devices (Continuous)**
  Class C communication profile is used in applications with enough power available, so there is no need to minimize the time of the reception windows; this is the case of most actuators (e.g., smart plugs, street lights, electrical meters, etc.) Class C devices always listen for downlinks messages unless they transmit an uplink message. This behavior results in the **lowest latency between the server and the end-device**.



The LoRaWAN Protocol



The LoRaWAN Protocol

- **Gateways**
  Gateways (or concentrators) are devices connected to the network server via standard IP connections that relay messages between the central network server backend and end-devices using single-hop wireless communication protocol. They are designed to support bi-directional communication and are equipped with multicast enabling the software to send mass distribution messages like over-the-air updates.
  At the heart of every **LoRa gateway is a multi-channel LoRa demodulator** able to decode all LoRa modulation variants on several frequencies in parallel.

The LoRaWAN Protocol

- **Gateways**
  LoRa spreads communication between end-devices and gateways across multiple frequency channels and data rates. The spread spectrum technology uses data rates ranging from 0.3 kbps to 50 kbps **to prevent communications from interfering with each other and creates a set of "virtual" channels** that increase the capacity of the gateway.

  To maximize both the battery life of the end-devices and the overall network capacity, the **LoRa network server manages the data rate and RF output for each end-device individually through an adaptive data rate (ADR) scheme**.

The LoRaWAN Protocol

- **Network Server**
  Lora Network server is **the interface between the Application server and the Gateways**. It relays commands from the Application server to the gateway while ferrying data from the gateways to the application server. It perform functions including ensuring there are no duplicate packets, Scheduling acknowledgements and managing the data rate and RF output for each end-device individually using a adaptive data rate (ADR) scheme.

- **Application Server**
  The application server determines what the data from the end devices are used for. Data Visualization etc are probably done here.

The LoRaWAN Data Rates

Communication between end-devices and gateways in LoRaWAN networks is spread out on different **frequency channels** and **data rates** (communications using different data rates do not interfere with each other).

To maximize the battery life of each end-device and the overall capacity available through the network, LoRaWAN uses an **Adaptive Data Rate (ADR)** mechanism for optimizing **data rates, airtime, and power consumption**. ADR controls the following transmission parameters on end-devices:

- **Spreading factor**: the speed of data transmission. Lower spreading factors mean a higher data transmission rate.
- **Bandwidth**: the amount of data that can be transmitted from one point to another within the network.
- **Transmission power**: the energy that the end-device transmitter produces at its output.

The LoRaWAN Data Rates

The table below shows compares spreading factor, data rate, and time on-air at a bandwidth of 125 kHz (range is an indicative value, it will depend on the propagation conditions):

Spreading Factor	Data Rate	Range	Time on-air
SF7	5470 bps	2 km	56 ms
SF8	3125 bps	4 km	100 ms
SF9	1760 bps	6 km	200 ms
SF10	980 bps	8 km	370 ms
SF11	440 bps	11 km	400 ms
SF12	290 bps	15 km	1400 ms

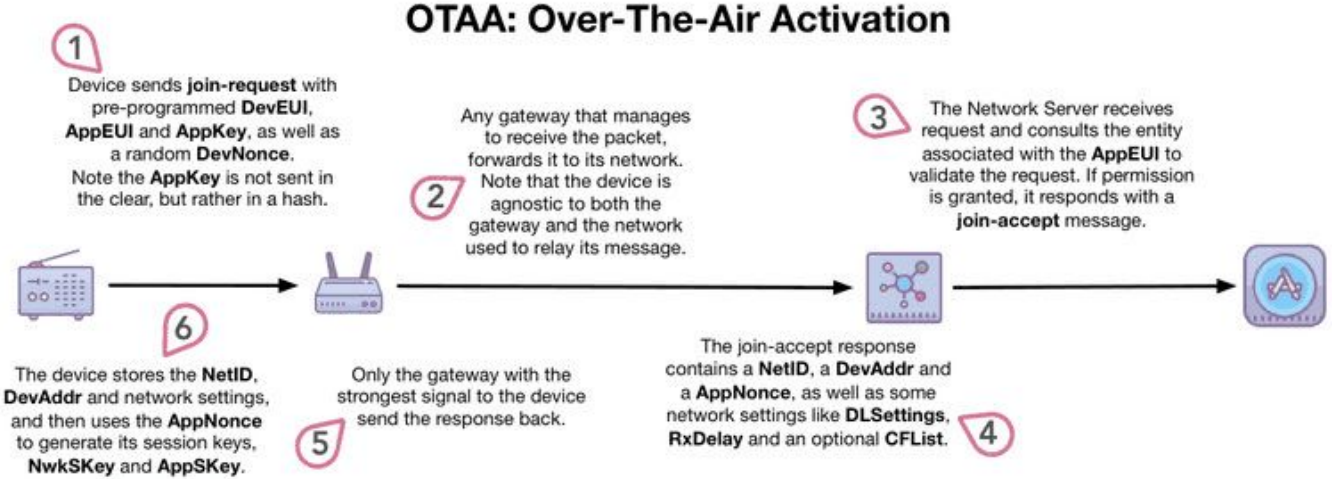
The LoRaWAN Authentication

Authentication and security are also important in LoRaWAN networks. Any LoRaWAN network has a baseline authentication and security framework based on the AES 128 encryption scheme. Compared to other LPWAN's, which rely on a single key for authentication and encryption, the LoRaWAN framework separates both. Authentication and integrity control use a **network session key** (NwkSKey) while user data encryption uses an **application session key** (AppSKey).

LoRaWAN supports two authentication and activation methods: **Over-The-Air-Activation (OTAA)** and **Activation by Personalization (ABP)**.

The LoRaWAN Authentication

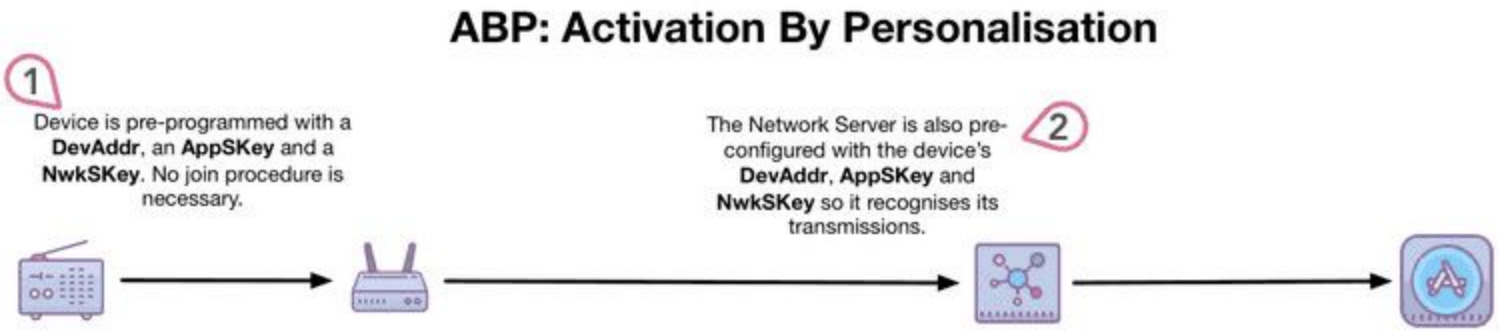
Over-The-Air Activation (OTAA): In this method, end-devices are not initialized for any particular network, they send a JOIN request to a specific LoRaWAN network and then receive a device address and an authorization token from which session keys are derived; NwkSKey and AppSKey are derived during this procedure from a root AppKey pre-provisioned in the end-devices by its manufacturer.



The LoRaWAN Authentication

Activation by Personalization (ABP): in this method, end-devices are personalized to work with a given LoRaWAN network. End devices are pre-provisioned with the NwkSKey and AppSKey and the 32-bits device network address.

The recommended authentication and activation method is **OTAA** since it provides a high level of security; ABP method should be used only for specific situations.



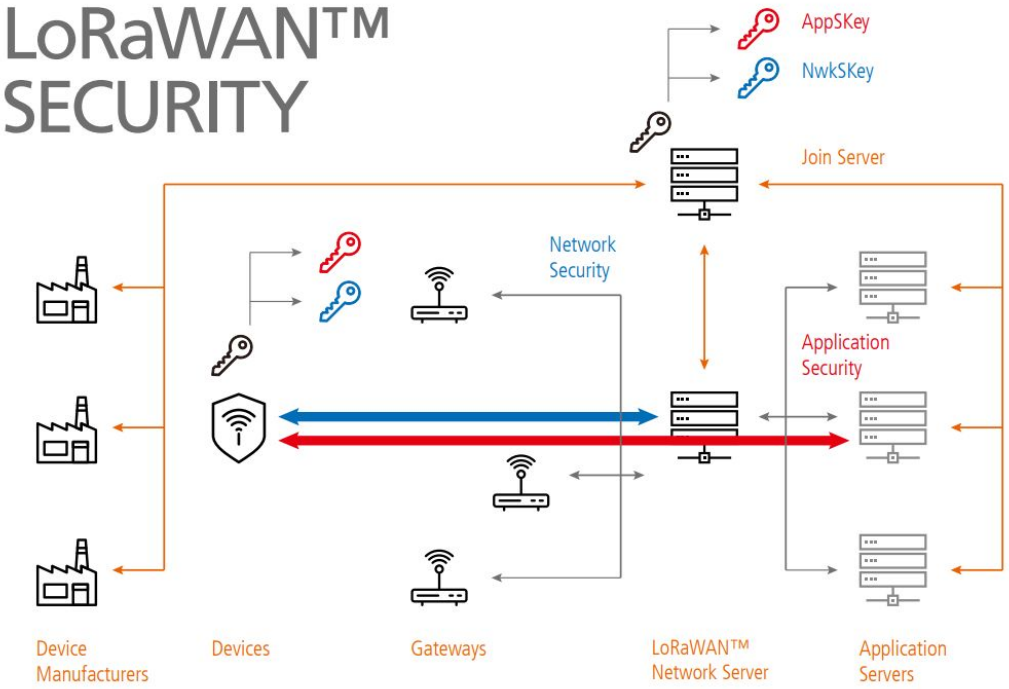
The LoRaWAN Security

The LoRaWAN protocol **specifies encryption to assure your data is secure**, concretely: per-device AES128 keys, instant regeneration/revocation of device keys, per-packet payload encryption for data privacy, protection against replay attacks and protection against man-in-the-middle attacks.

LoRaWAN **uses two keys**; **Network Session and Application Session keys,** both of which provide split, encrypted communication for network management and application communication.

The **network session key**, shared between the device and the network is responsible for authentication of the end node data while the application session key, shared between the application and the end node is responsible for guaranteeing the privacy of the device data.

The LoRaWAN Security



The ChirpStack Solution

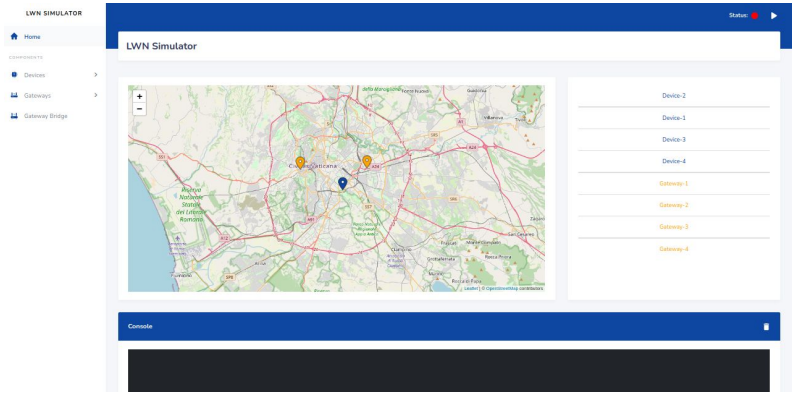


ChirpStack **provides open-source components for LoRaWAN networks**. Together they form a ready-to-use solution including an user-friendly web-interface for device management and APIs for integration. The modular architecture makes it possible to integrate within existing infrastructures. The following components are provided:

- **Gateway Bridge**: handles the communication with the LoRaWAN gateways.
- **Network Server**: a LoRaWAN Network Server implementation.
- **Application Server**: a LoRaWAN Application Server implementation
- **Gateway OS**: Linux-based OS to run the (full) ChirpStack stack on a Raspberry Pi based LoRa gateway

The LWN Simulator

LWN Simulator is a LoRaWAN nodes' simulator equipped with web interface. It allows to comunicate with a real infrastructure LoRaWAN or ad-hoc infrastructure, such as Chirpstack.



https://github.com/UniCT-ARSLab/LWN-Simulator

The LoRaWAN on ESP32



```
#include <iot_board.h>

int count_sent = 0;

const char *devAddr = "260B3E70";
const char *nwkSKey = "AF0D4340E12A7333A23BB42D239C2C5A ";
const char *appSKey = "909745BCD7A80913AAAD9138B8D7FAA7 ";

char myStr[50];
char outStr[255];

byte recvStatus = 0;

boolean runEvery(unsigned long interval)
{
 static unsigned long previousMillis = 0;
 unsigned long currentMillis = millis();
 if (currentMillis - previousMillis >= interval)
 {
   previousMillis = currentMillis;
   return true;
 }
 return false;
}
```

```
void onLoRaReceive (sBuffer *Data_Rx, bool isConfirmed, uint8_t fPort){
   String s = String((char *) Data_Rx->Data, HEX);
   Serial.println(s);
}

void setup(){
   IoTBoard::init_serial();
   IoTBoard::init_spi();

   if(IoTBoard::init_lorawan()){

       lorawan->setDeviceClass(CLASS_A);
       lorawan->setDataRate(SF9BW125);
       // set channel to random
       lorawan->setChannel(MULTI);
       // Put ABP Key and DevAddress here
       lorawan->setNwkSKey(nwkSKey);
       lorawan->setAppSKey(appSKey);
       lorawan->setDevAddr(devAddr);

       lorawan->onMessage(&onLoRaReceive);

   } else {
      Serial.println("Error on LoRawan");
   }
}
```

The LoRaWAN on ESP32



```
void setup(){
   IoTBoard::init_serial();
   IoTBoard::init_spi();
   if(IoTBoard::init_lorawan()){
       lorawan->setDeviceClass(CLASS_A);
       lorawan->setDataRate(SF9BW125);
       // set channel to random
       lorawan->setChannel(MULTI);
       lorawan->setDevEUI(devEui);
       lorawan->setAppEUI(appEui);
       lorawan->setAppKey(appKey);
       lorawan->onMessage(&onLoRaReceive);
         // Join procedure
       bool isJoined;
       do {
         Serial.println("Joining...");
         isJoined =  lorawan->join();

         //wait for 10s to try again
         delay(10000);
       }while(!isJoined);
       Serial.println("Joined to network");

   } else {
       Serial.println("Error");
   }
}
```

```
void loop(){

   if(runEvery(10000)){
       sprintf(myStr, "Hello-%d", count_sent);
       lorawan->sendUplink(myStr, strlen(myStr), 1, 2);
       count_sent++;
   }
   lorawan->readData(outStr);
 // Check Lora RX
   lorawan->update();
   delay(1);
}
```

4

IoT Communication

Well, now we talk the same language!

There are a lot of communication ways for IoT!

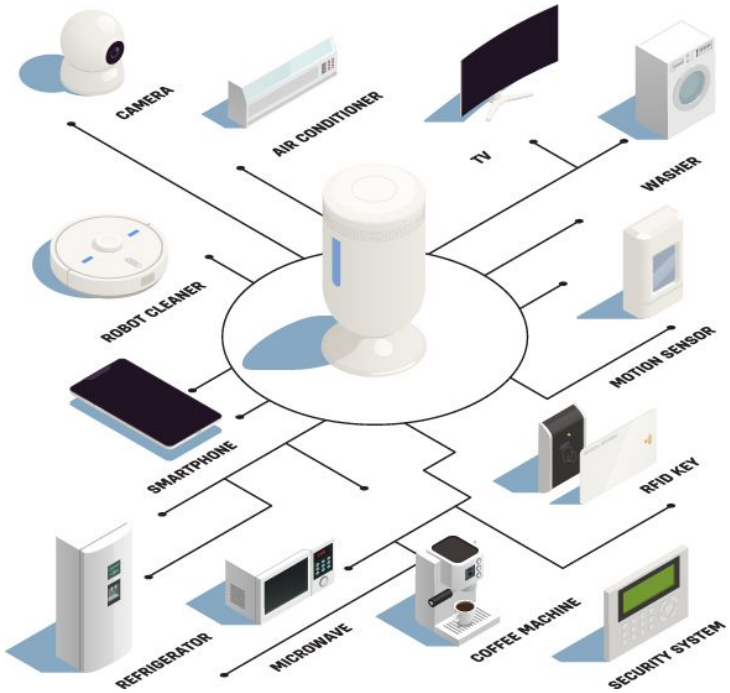
Nowadays there are various ways to communicate with the IoT devices that surround us and to let them communicate with the Internet!



IoT Data/Messaging protocols

	Type	Weight	Used for
MQTT	**Publisher/Subscribe**	**Light**	**Read or publish information**
CoAP	**HTTP RESTful**	**Light**	**Send commands**
XMPP	**XML message**	**Heavy**	**Exchange information**
AMQP	**Message queuing**	**Medium**	**Send commands**
HTTP	**HTTPv2**	**Light**	**Send and get information**
WebSocket	**Real time socket**	**Light**	**Real time communication**

The Gateway Infrastructure



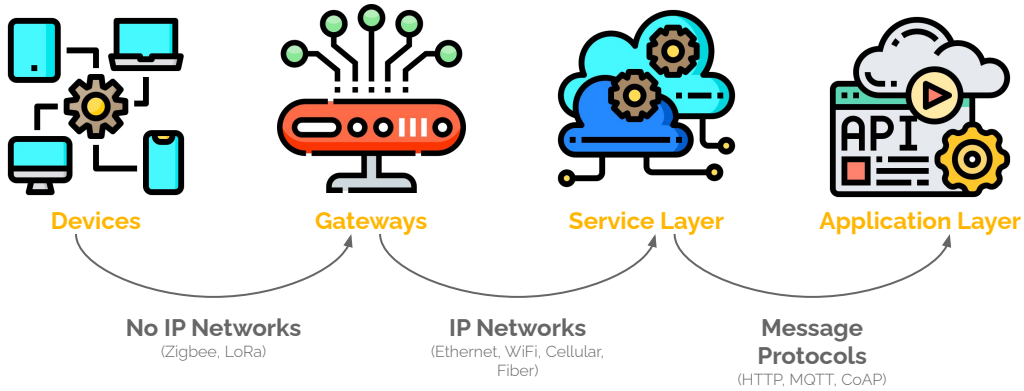
An **IoT Gateway** is a physical device or software program that serves as the connection point between the cloud and controllers, sensors and intelligent devices.

An IoT gateway **acts as a network router**, routing data between IoT devices and the cloud. Early on, most gateway devices only sent traffic in one direction: from the IoT devices to the cloud. Now, it's common for a gateway device to handle both inbound and outbound traffic.

A gateway device can sometimes be **used to preprocess that data locally at the edge** before sending it to the cloud.

The Gateway Infrastructure

A simple IoT gateway functions similarly to a Wi-Fi router. An IoT system connects to the gateway **using a IP connection** and then the gateway routes the IoT device data to the cloud. One reason why an IoT gateway tends to be more complex than a router is that **there are several different protocols used by IoT devices**. Some of these protocols include Bluetooth Low Energy, Zigbee and LoRa. As such, an IoT gateway might **need to support a variety of protocols to service all the IoT devices in an organization**.



Then a set of IoT Devices are connected to a Gateway that has an equal transceiver of same protocol. Then the Gateway sends these information through a IP protocol using some real-time protocol (MQTT or WebSocket). Finally the information are stored into cloud service and are available through same protocol or other API (HTTP) to users applications.

The HTTP Protocol



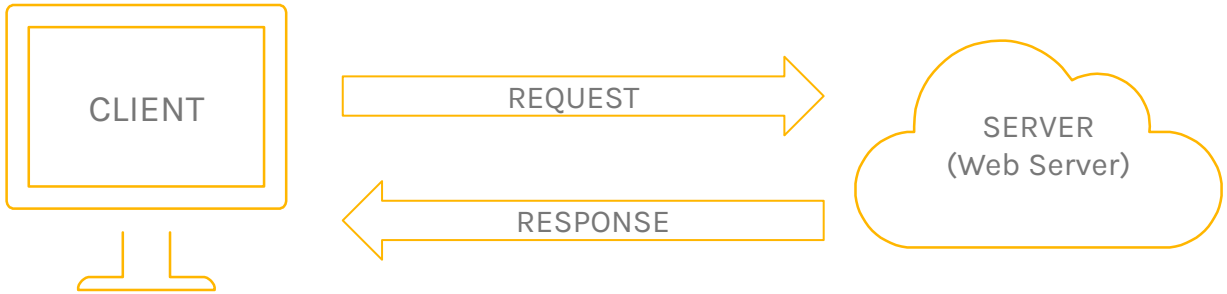
HTTP was developed by Tim Berners-Lee and his team in 1989, HTTP has gone through many changes that have helped maintain its simplicity while shaping its flexibility.

HTTP is a **stateless**, application-layer protocol to let communication possible between distributed systems, and is the foundation of the modern web.

HTTP allows communication between a variety of hosts and clients, and supports a mixture of network configurations.

HTTP evolved from a protocol designed to exchange files in a semi-trusted laboratory environment into a modern internet maze that carries images and videos in high resolution and 3D.

The HTTP Protocol



Communication between a host and a client occurs, via a request/response pair.
The client initiates an **HTTP request message**, which is serviced through a **HTTP response message** in return.

The HTTP Protocol



The heart of web communications is the request message, which is sent via **Uniform Resource Locators** (**URL**s).

[**PROTOCOL**]://[**DOMAIN**]:[**PORT**]/[**RESOURCE**]?[**QUERY**][#**FRAGMENT**]

E.G.
HTTP://PICSUM.PHOTOS/200/300/?RANDOM

DEFAULT PORT: 80 (**HTTP**) OR 443 (**HTTPS**)

The HTTP Protocol

URLs reveal the identity of a particular host with whom we want to communicate, but the action that should be performed on the host is specified via **HTTP verbs** (or **methods**).

- **GET**: fetch an existing resource.
- **POST**: create a new resource.
- PUT: update an existing resource.
- DELETE: delete an existing resource.

PUT and DELETE are sometimes considered specialized versions of the POST verb

The HTTP Protocol



With URLs and verbs, the client can initiate requests to the server.
In return, the server responds with status codes and message payloads.

- 1XX: **Info messages**
- 2XX: **Successful messages**
- 3XX: **Cache messages**
- 4XX: **Client error messages**
- 5XX: **Server error messages**

Never seen the error 404? Go to http://alexgorbatchev.com/wiki/

The HTTP(Client) on ESP32



```
#include <iot_board.h>
#include <WiFi.h>
#include <HTTPClient.h>
#include <WiFiClientSecure.h>

const char *ssid = "SSID";
const char *password = "PASSWORD";

String serverName =
"https://api.open-meteo.com/v1/forecast?latitude=37.525147802890295&longi
tude=15.070232376117882&current=temperature_2m,wind_speed_10m";
WiFiClientSecure client;
HTTPClient httpClient;
String response = "";

boolean runEvery(unsigned long interval){
   static unsigned long previousMillis = 0;
   unsigned long currentMillis = millis();
   if (currentMillis - previousMillis >= interval)
   {
       previousMillis = currentMillis;
       return true;
   }
   return false;
}
```

```
void setup(){
   IoTBoard::init_serial();
   IoTBoard::init_display();
   display->println("Display enabled");
   display->display();

   WiFi.begin(ssid, password);
   display->println("Connecting to WiFi");
   display->display();
   while (WiFi.status() != WL_CONNECTED){
       delay(500);
   }
   display->clearDisplay();
   display->setCursor(0, 0);
   display->println("Connected");
   display->println(WiFi.localIP());
   display->display();
   //to allow every tsl certificate for now
   client.setInsecure();
}
```

The HTTP(Client) on ESP32



```
void loop(){
   if (runEvery(2000))
   {
       if (WiFi.status() == WL_CONNECTED)
       {
           httpClient.begin(client, serverName);
           httpClient.addHeader("accept", "application/json");
           httpClient.addHeader("Content-Type", "application/json");
           int httpResponseCode = httpClient.GET();

           if (httpResponseCode > 0){
               Serial.print("HTTP Response code: ");
               Serial.println(httpResponseCode);
               response = httpClient.getString();
               Serial.println(response);

           }
           else{
               Serial.print("Error code: ");
               Serial.println(httpResponseCode);
           }
           httpClient.end();
       }
   }
}
```

The HTTP(Server) on ESP32



```
#include <iot_board.h>
#include <WiFi.h>
#include <AsyncTCP.h>
#include <ESPAsyncWebServer.h>

const char *ssid = "UniCT-Team-Public";
const char *password = "PASSWORD";
AsyncWebServer server(80);

void setup(){
   IoTBoard::init_serial();
   IoTBoard::init_display();
   IoTBoard::init_leds();
   display->println("Display enabled");
   display->display();

   WiFi.begin(ssid, password);
   display->println("Connecting to WiFi");
   display->display();
   while (WiFi.status() != WL_CONNECTED){
       delay(500);
   }

   //GET on root
   server.on("/", HTTP_GET, [](AsyncWebServerRequest *request){
       request->send(200, "text/plain", "Hello, world");
   });
   //GET with parameters
   server.on("/get", HTTP_GET, [] (AsyncWebServerRequest *request) {
       String message;
       if (request->hasParam("parametro")) {
           message = request->getParam("parametro")->value();
       } else {
           message = "No message sent";
       }
       request->send(200, "text/plain", "Hello, GET: " + message);
   });
   // POST with Body
   server.on("/post", HTTP_POST, [](AsyncWebServerRequest *request){
       String message;
       if (request->hasParam("parametro", true)) {
           message = request->getParam("parametro", true)->value();
       } else {
           message = "No message sent";
       }
       request->send(200, "text/plain", "Hello, POST: " + message);
   });
   display->clearDisplay();
   display->setCursor(0, 0);
   display->println("Connected");
   display->println(WiFi.localIP());
   display->println("Server actived on port 80");
   display->display();

   server.begin();
}
```

The WebSocket Protocol



When a user surfs the web, the browser (client) sends certain requests via REST routes ('GET', 'POST', etc) to the server that hosts the website they are trying to access. The server receives the requests and sends information as responses back to the client, which receives and renders the response information on the page. If you need to update some information, you must do several requests to the server in a interval time. This approach is called **Polling**.

WebSockets are different because they work by holding the connection from the client to the server open at all times. This way, the server can send information to the client, even in the absence of an explicit request from the client. Clients can still make HTTP requests to the server like normal; however, the WebSocket connection allows for constant communication in the case of new data being created and therefore needing to be rendered client-side.

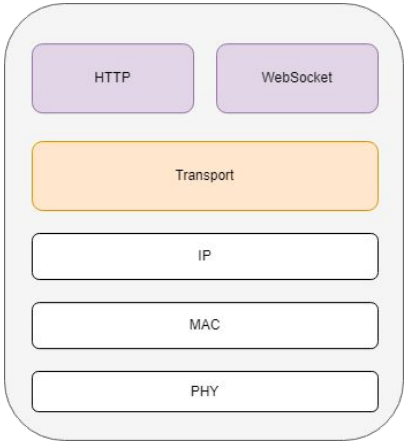
The WebSocket Protocol

A connection starts with a HTTP handshake that request an upgrade for the protocol, this is called WebSocket Handshake request. The handshake starts with an HTTP request/response, allowing servers to handle HTTP connections as well as WebSocket connections on the same port. Once the connection is established, communication switches to a bidirectional binary protocol which does not conform to the HTTP protocol.
In addition to Upgrade headers, the client sends a Sec-WebSocket-Key header containing base64-encoded random bytes, and the server replies with a hash of the key in the Sec-WebSocket-Accept header. **This is intended to prevent a caching proxy from re-sending a previous WebSocket conversation**, and does not provide any authentication, privacy, or integrity.
Once the connection is established, the client and server can send WebSocket data or text frames back and forth in full-duplex mode. The data is minimally framed, with a small header followed by payload. WebSocket transmissions are described as "messages", where a single message can optionally be split across several data frames.

The WebSocket Protocol

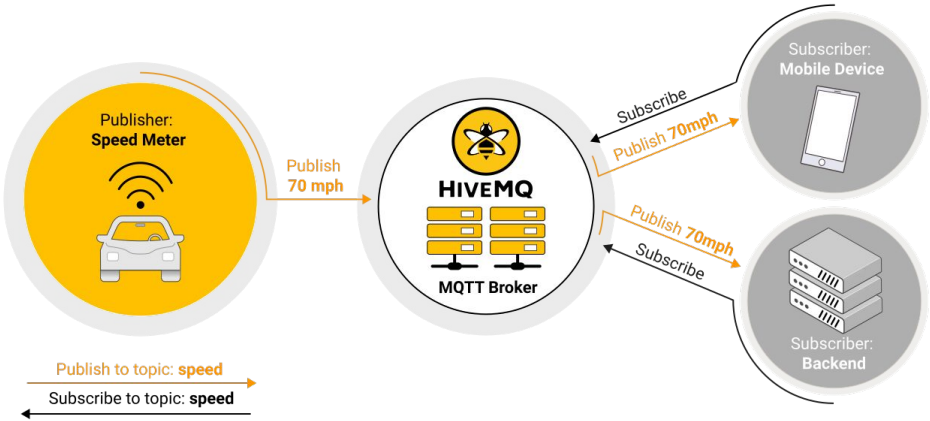
WebSockets work at Application layer and can work without a specific server. In fact, a WebSocket server can run using an existing HTTP server or standalone.
To define a WebSocket connection you have to define an URL using the **ws://** protocol, instead of http://. Like HTTP, WebSockets support the SSL encryption and you can specify that using the protocol wss://.

Attaching some event handlers immediately to the connection allows you to know when the connection is opened, received incoming messages, or there is an error.



The MQTT Protocol

MQTT (Message Queuing Telemetry Transport) is a **publish/subscribe protocol** that is lightweight and requires a minimal footprint and bandwidth to connect an IoT device. Unlike HTTP's, MQTT is event driven and enables messages to be pushed to clients. This type of architecture decouples the clients from each other to enable a highly scalable solution without dependencies between data producers and data consumers.



These characteristics make it ideal for use in many situations, including constrained environments such as for communication in M2M and IoT contexts where a small code footprint is required and/or network bandwidth is at a premium.

The MQTT Protocol

The most important aspect of **pub/sub** is the decoupling of the publisher of the message from the recipient (subscriber). This decoupling has several dimensions:

- **Space decoupling**
  Publisher and subscriber do not need to know each other (for example, no exchange of IP address and port).
- **Time decoupling**
  Publisher and subscriber do not need to run at the same time.
- **Synchronization decoupling**
  Operations on both components do not need to be interrupted during publishing or receiving.

The MQTT pub/sub model removes direct communication between the publisher of the message and the recipient/subscriber. The filtering activity of the broker makes it possible to control which client/subscriber receives which message. The decoupling has three dimensions: space, time, and synchronization.

The MQTT Protocol

Depending on what you want to achieve, MQTT embodies all the aspects of pub/sub that we have mentioned:

- MQTT **decouples the publisher and subscriber spatially**. To publish or receive messages, publishers and subscribers only need to know the hostname/IP and port of the broker.
- MQTT **decouples by time**. Although most MQTT use cases deliver messages in near-real time, if desired, the broker can store messages for clients that are not online.
- MQTT **works asynchronously**. Because most client libraries work asynchronously and are based on callbacks or a similar model, tasks are not blocked while waiting for a message or publishing a message.

MQTT **uses subject-based filtering of messages**. Every message contains a **topic** (**subject**) that the broker can use to determine whether a subscribing client gets the message or not.

The MQTT Protocol

There is a lot of confusion about the name MQTT and whether the protocol is implemented as a message queue or not. Regardless of where the name comes from, it's useful to understand the differences between MQTT and a traditional message queue:

- **A message queue stores message until they are consumed.** When you use a message queue, each incoming message is stored in the queue until it is picked up by a client (often called a consumer). If no client picks up the message, the message remains stuck in the queue and waits to be consumed.
- **A message is only consumed by one client.** Another big difference is that in a traditional message queue a message can be processed by one consumer only. The load is distributed between all consumers for a queue. In MQTT the behavior is that every subscriber that subscribes to the topic gets the message.
- **Queues are named and must be created explicitly.** A queue is far more rigid than a topic. Before a queue can be used, the queue must be created explicitly with a separate command. MQTT topics are extremely flexible and can be created on the fly.

The MQTT Protocol

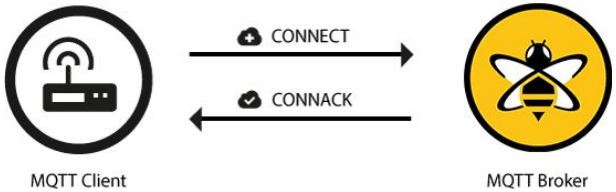
Because MQTT decouples the publisher from the subscriber, client connections are always handled by a broker.

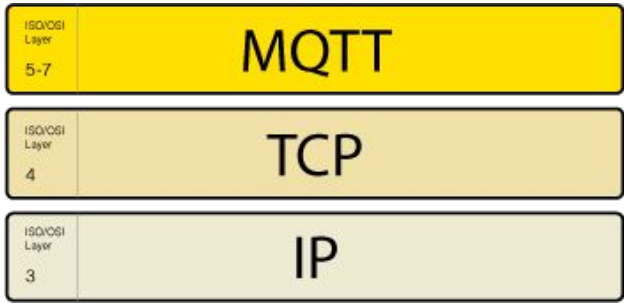
- **Client**
  When we talk about a client, we almost always mean an MQTT client. Both publishers and subscribers are MQTT clients. The publisher and subscriber labels refer to whether the client is currently publishing messages or subscribed to receive messages. **An MQTT client is any device (from a microcontroller up to a full-fledged server) that runs an MQTT library and connects to an MQTT broker** over a network.
- **Broker**
  The broker is responsible for receiving all messages, filtering the messages, determining who is subscribed to each message, and sending the message to these subscribed clients. The broker also holds the session data of all clients that have persistent sessions, including subscriptions and missed messages . Another responsibility of the broker is the authentication and authorization of clients.

The MQTT Protocol

The MQTT protocol is based on TCP/IP. Both the **client and the broker need to have a TCP/IP stack**.

The MQTT connection is always between one client and the broker. **Clients never connect to each other directly**. To initiate a connection, the client sends a **CONNECT** message to the broker.
The broker responds with a **CONNACK** message and a status code. Once the connection is established, the broker keeps it open until the client sends a disconnect command or the connection breaks.





MQTT-Packet:

CONNECT

contains:	Example
clientId	"client-1"
cleanSession	true
username (optional)	"hans"
password (optional)	"letmein"
lastWillTopic (optional)	"/hans/will"
lastWillQos (optional)	2
lastWillMessage (optional)	"unexpected exit"
lastWillRetain (optional)	false
keepAlive	60

The MQTT Protocol

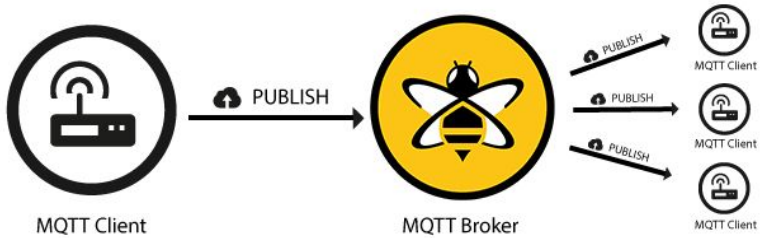
An MQTT client can publish messages as soon as it connects to a broker. MQTT utilizes topic-based filtering of the messages on the broker. **Each message must contain a topic that the broker can use to forward the message to interested clients**. Typically, each message has a payload which contains the data to transmit in byte format. **MQTT is data-agnostic**. The use case of the client determines how the payload is structured. The sending client (publisher) decides whether it wants to send binary data, text data, or even full-fledged XML or JSON.

MQTT-Packet:

PUBLISH

contains:	Example
packetId (always 0 for qos 0)	4314
topicName	"topic/1"
qos	1
retainFlag	false
payload	"temperature:32.5"
dupFlag	false

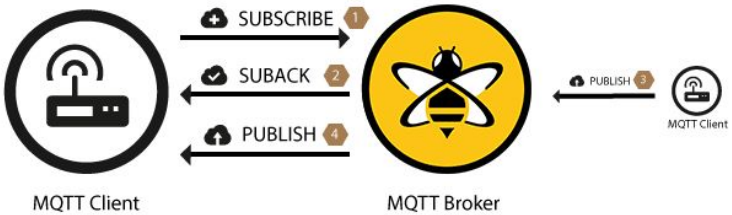
When a client sends a message to an MQTT broker for publication, **the broker reads the message, acknowledges the message and processes the message**.



The MQTT Protocol

Publishing a message doesn't make sense if no one ever receives it. In other words, if there are no clients to subscribe to the topics of the messages. To receive messages on topics of interest, the client sends a **SUBSCRIBE** message to the MQTT broker. This subscribe message is very simple, it contains a unique packet identifier and a list of subscriptions.

To confirm each subscription, the broker sends a **SUBACK** acknowledgement message to the client. This message contains the packet identifier of the original **Subscribe** message.



MQTT-Packet:

SUBSCRIBE

contains:		Example
packetId		4312
qos1	} (list of topic + qos)	1
topic1		"topic/1"
qos2	}	0
topic2		"topic/2"
...		...

The MQTT Protocol

In MQTT, the word **topic** refers to an UTF-8 string that the broker uses to filter messages for each connected client. The topic consists of one or more topic levels. Each topic level is separated by a forward slash (topic level separator).



In comparison to a message queue, MQTT topics are very lightweight. The client does not need to create the desired topic before they publish or subscribe to it. The broker accepts each valid topic without any prior initialization. Note that each topic must contain at least 1 character and that the topic string **permits empty spaces**. **Topics are case-sensitive**.

The MQTT Protocol

As the name suggests, **a single-level wildcard** replaces one topic level. **The plus symbol** represents a single-level wildcard in a topic.



- ✓ myhome / groundfloor / livingroom / temperature
- ✓ myhome / groundfloor / kitchen / temperature
- ✗ myhome / groundfloor / kitchen / brightness
- ✗ myhome / firstfloor / kitchen / temperature
- ✗ myhome / groundfloor / kitchen / fridge / temperature

The **multi-level wildcard** covers many topic levels. **The hash symbol** represents the multi-level wild card in the topic. For the broker to determine which topics match, the multi-level wildcard must be placed as the last character in the topic and preceded by a forward slash.



- ✓ myhome / groundfloor / livingroom / temperature
- ✓ myhome / groundfloor / kitchen / temperature
- ✓ myhome / groundfloor / kitchen / brightness
- ✗ myhome / firstfloor / kitchen / temperature

The MQTT on ESP32



```
#include <iot_board.h>
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <PubSubClient.h>

const char *ssid = "UniCT-Team-Public";
const char *password = "arslabiot";

const char* mqtt_server = "broker.hivemq.com";
uint16_t mqtt_port = 8883;
WiFiClientSecure client;
PubSubClient mqttClient(client);

boolean runEvery(unsigned long interval){
   static unsigned long previousMillis = 0;
   unsigned long currentMillis = millis();
   if (currentMillis - previousMillis >= interval)
   {
       previousMillis = currentMillis;
       return true;
   }
   return false;
}
```

```
void mqttCallback(char *topic, byte *payload, unsigned int
length) {
   Serial.print("Message arrived in topic: ");
   Serial.println(topic);
   Serial.print("Message:");
   for (int i = 0; i < length; i++) {
       Serial.print((char) payload[i]);
   }
   Serial.println();
   Serial.println("-----------------------");
}
```

The MQTT on ESP32



```
void setup(){

    IoTBoard::init_serial();
    IoTBoard::init_display();
    IoTBoard::init_leds();
    display->println("Display enabled");
    display->display();

    WiFi.begin(ssid, password);
    display->println("Connecting to WiFi");
    display->display();
    while (WiFi.status() != WL_CONNECTED){
        delay(500);
    }

    client.setInsecure();
    mqttClient.setServer(mqtt_server, mqtt_port);
    mqttClient.setCallback(mqttCallback);

    display->clearDisplay();
    display->setCursor(0, 0);
    display->println("Internet Connected");
    display->display();
    while (!mqttClient.connected()) {
        String client_id = "iotBoard-client-";
        client_id += String(WiFi.macAddress());

        if (mqttClient.connect(client_id.c_str())) {
            display->println("MQTT Connected");
        } else {
            display->println("MQTT error");
            delay(2000);
        }
    }
    display->display();
    mqttClient.subscribe("test/recv1");
}

void loop(){
    if(runEvery(4000)){
        mqttClient.publish("test/status", "Hi, I'm IoTBoard
^^");
    }
    mqttClient.loop();
}
```

5

IoT Artificial Intelligence

Oh no! My smartwatch is smart!

The IA for IoT

Artificial Intelligence (AI) in IoT combines data from sensors, devices, and machines with algorithms to enable decision-making without human intervention.
Unlike traditional IoT systems that rely heavily on the cloud for analysis, AI allows for smarter, real-time decision-making at the edge, enabling IoT systems to respond dynamically to changing conditions.

- **Edge Intelligence**
  Processes data where it is generated, minimizing delays and dependency on cloud services. Critical for applications like autonomous vehicles, industrial automation, and medical devices.

- **Cost Efficiency**
  Reduces bandwidth costs by limiting the amount of data sent to the cloud.

- **Enhanced Privacy**
  Keeps sensitive data local, improving privacy and security

The Neural Networks for IoT

Neural networks are computational models inspired by the way neurons in the human brain process information. They are a key component of AI and are used to perform complex tasks like image recognition, speech processing, and anomaly detection. These are composed by:

- **Input Layer**
  Accepts data such as sensor readings (e.g., temperature, humidity, or motion).

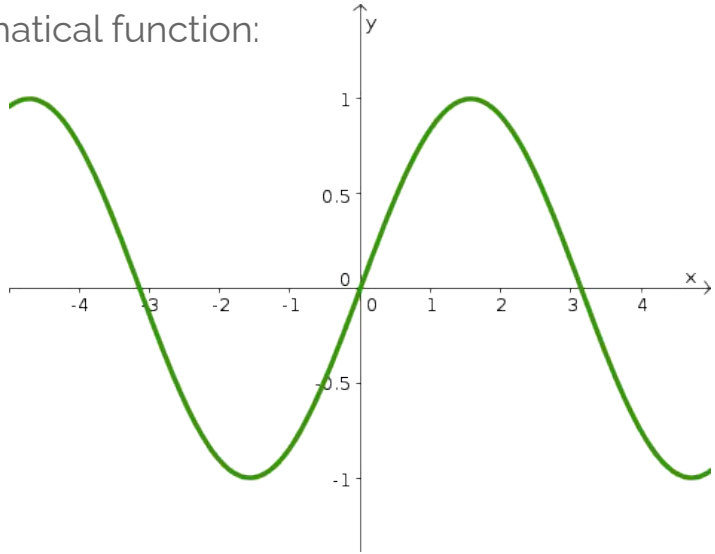
- **Hidden Layers**
  Perform computations using neurons. Each neuron combines inputs using weights and biases, processes them using an activation function, and passes the result to the next layer.

- **Output Layer**
  Produces the final prediction or classification (e.g., "normal" or "anomalous").

The Neural Networks for IoT

Neural networks are essentially a mathematical functions and are designed to resolve one specific problem in a specific environment! The basic example using these are the regressions, that are equivalent to find the right f(x) function having the y axis values!

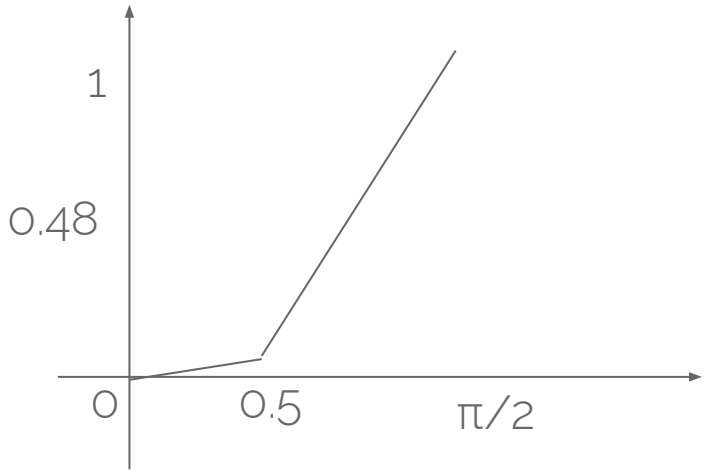
Consider the Sin(x) mathematical function:



The Neural Networks for IoT

Now image that you do not know the original function but have only a small set of dataset to redraw the function in the axis graph:

x	f(x) | y
0	0
π/2	1
0.5	0.009

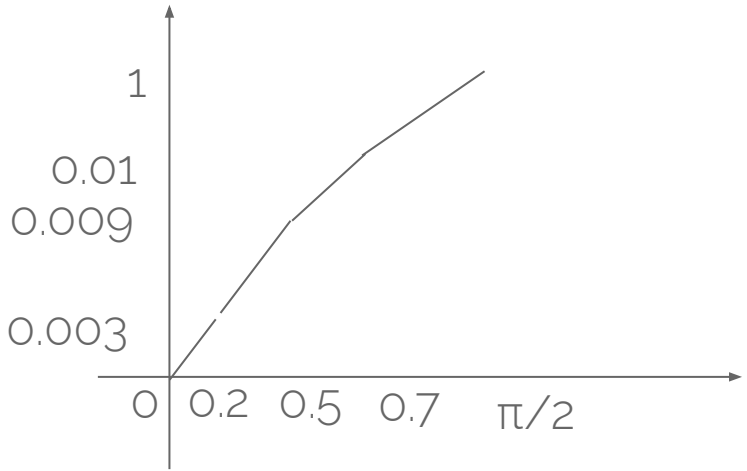


As you can see, the data is not enough to Redraw the original function, now try to add more data.

The Neural Networks for IoT

As you can see, more data we have and more similar will be the final function on the graph to the original Sin(x).

x	f(x) | y
0	0
π/2	1
0.5	0.009
0.2	0.003
0.7	0.1



The Neural Networks for IoT

Neural networks **excel at interpolation** (making predictions within the range of the training data) but struggle with **extrapolation** (predicting beyond the training range).
But there is a problem, image to explain the Sin(x) function to a child that does not know anything about that.

Let us assume that, if $x=0$ then $Sin(x) = 0$ but also if $x=\pi$ then $Sin(x) = 0$.

Considering these factors the child will immediately understand that theoretically there is an absurd because for two values of different x exist the same values.

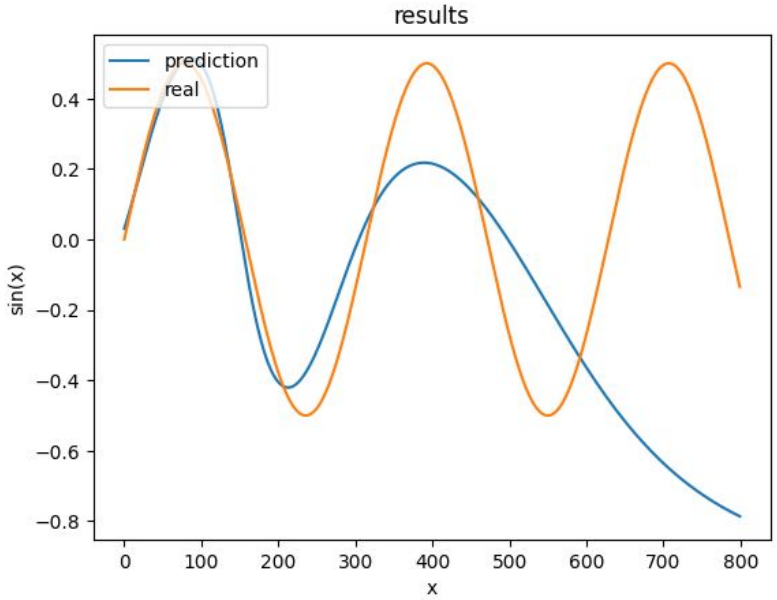
To resolve that, we will teach that also for that value the Sin(x) will be zero.

But also if $x = 2\pi$ then $Sin(x) = 0$, then we will teach the rule thar for each n in Naturals, the $Sin(n\pi) = 0$.

The Neural Networks for IoT

Mathematically, the sine function can be represented using a Taylor series or an infinite sum of terms. To approximate Sin(x) perfectly, a neural network would require infinite neurons or layers, which **is not feasible in practice**.
Even with practical networks, high accuracy would require many layers and neurons, increasing computational cost and the risk of overfitting.
Approximating Sin(x) to high precision can lead to difficulties in convergence and may require very small learning rates and careful tuning.

https://colab.research.google.com/drive/13OcxAGBTFyPCFFBGBCmDJEv3vwXaYNvY?usp=sharing



The Tools for IA for IoT

- **Google Collab**
  Colab is a hosted Jupyter Notebook service that requires no setup to use and provides free of charge access to computing resources, including GPUs and TPUs. Colab is especially well suited to machine learning, data science, and education.

- **TensorFlow**
  TensorFlow is a software library for machine learning and artificial intelligence. It can be used across a range of tasks, but is used mainly for training and inference of neural networks.

- **Keras**
  Keras is an open-source library that provides a Python interface for artificial neural networks. Keras was first independent software, then integrated into the TensorFlow library, and later supporting more.

- **TensorFlow Lite**
  TensorFlow Lite provides a set of tools that enables on-device machine learning by allowing developers to run their trained models on mobile, embedded, and IoT devices and computers. It supports platforms such as embedded Linux, Android, iOS, and MCU.

The Model on ESP32

There are a lot of libraries that can be used to load a TensorFlow Lite model and use it on microcontroller, one of these is EloquentTinyML.

https://github.com/eloquentarduino/EloquentTinyML.git#2.4.0

Color classification:
https://colab.research.google.com/drive/1DMyVwSMTvLQfUPn0nLtUTZr1nT1wQQEB?usp=sharing

The Model on ESP32

```
#include <iot_board.h>
#include "EloquentTinyML.h"
#include "model.h"

#define NUMBER_OF_INPUTS 1
#define NUMBER_OF_OUTPUTS 1
#define TENSOR_ARENA_SIZE 2*1024

Eloquent::TinyML::TfLite<NUMBER_OF_INPUTS, NUMBER_OF_OUTPUTS, TENSOR_ARENA_SIZE> ml;

void setup() {
   IoTBoard::init_serial();
   if (!ml.begin(model_data)){ Serial.println("Errore");}
}

void loop() {
   // pick up a random x and predict its sine
   float x = 3.14 * random(100) / 100;
   float y = sin(x);
   float input[1] = { x };
   float predicted = ml.predict(input);
   Serial.print("sin(");
   Serial.print(x);
   Serial.print(") = ");
   Serial.print(y);
   Serial.print("\t predicted: ");
   Serial.println(predicted);
   delay(1000);
}
```

The Next steps

- Analyse the problem
- Collect data from sensors
- Save them and label them into files using Serial or other communication systems
- Design the Neural Network
- Train the Model
- Export the model using TensorFlow Lite
- Import the model into the microcontroller
- Predict!

6

IoT Security

Oh no! My smartwatch was hacked!

Is the IoT Secured?



IoT devices were not built with security in mind, leading to potential vulnerabilities in a multiple device system.

The Mirai Malware Attack



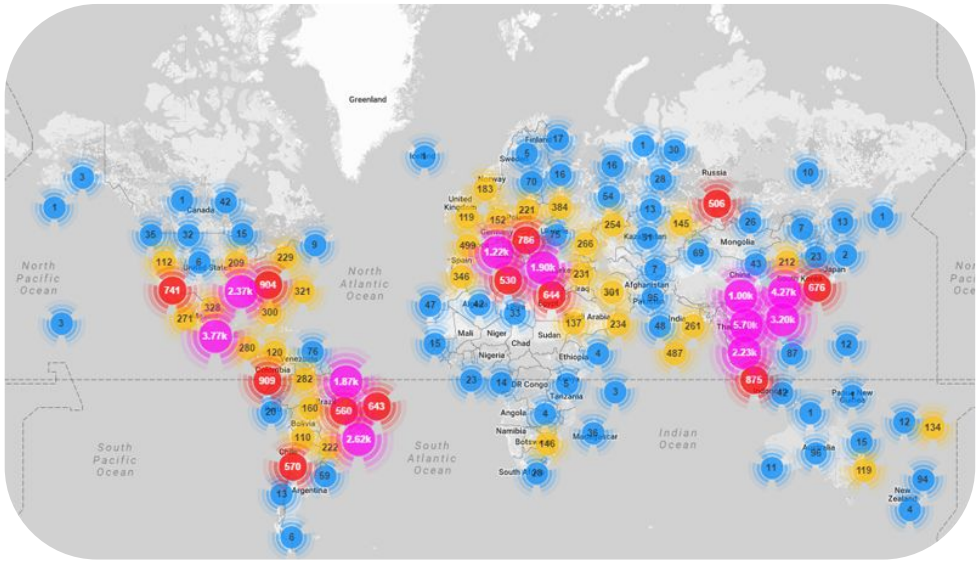
- **Mirai** is a malware that turns networked devices running Linux into remotely controlled bots.
- It primarily targets online consumer devices such as IP cameras and home routers.
- The software was initially used by the creators to DDoS Minecraft servers and companies offering DDoS protection to Minecraft servers,
- Devices infected by Mirai continuously scan the internet for the IP address of IoT devices.
- Mirai includes a table of IP address ranges that it will not infect,

How Mirai Works?

- Mirai then identifies vulnerable IoT devices using a table of more than 60 common **factory default usernames and passwords**.
- Infected devices will continue to function normally, except for occasional sluggishness and an increased use of bandwidth.
- A device remains infected until it is rebooted, which may involve simply turning the device off and after a short wait turning it back on.
- After a reboot, unless the login password is changed immediately, the device will be reinfected within minutes.

Why use a Botnet ?

- A botnet is a **cluster of machines** that are infected with malware, enabling hackers to control them and unleash a string of attacks.
- This attack is commonly used for DDos attacks.
- IoT connected devices = **Huge collective computing power**!



Mirai infected devices in 2016

The importance of Security



- Hardware, software and connectivity will all need to be secure for IoT objects to work effectively.
- Without security for IoT, **any connected object**, from refrigerators to manufacturing bots, **can be hacked**.
- As numerous devices are connected often without important security settings, the IoT has inevitably become an enticing playground for cybercriminals.
- Once hackers gain control, they can usurp the object's functionality and steal the user's digital data.
- IoT security vulnerabilities are found in everything from vehicles and smart grids to watches and smart home devices.

What is IoT Security ?



- IoT security is an umbrella term that covers the strategies, tools, processes, systems, and methods used to protect all aspects of the internet of things.
- Includes: the protection of the physical components, applications, data and network connections to ensure the availability, integrity, and confidentiality of IoT ecosystems.
- Robust IoT security includes all facets of protection, including hardening components, monitoring, keeping firmware updated, access management, threat response and remediation of vulnerabilities.
- **Security challenges abound**, because of the high volume of flaws regularly discovered in IoT systems.

IoT Security Challenges

IoT devices were not built with security in mind. This results in myriad IoT security challenges that can lead to disastrous situations. Among the many IoT security issues are the following:

- **Lack of visibility**
  Users often deploy IoT devices without the knowledge of IT departments, which makes it impossible to have an accurate inventory of what needs to be protected and monitored.
- **Limited security integration**
  Because of the variety and scale of IoT devices, integrating them into security systems ranges from challenging to impossible.

IoT Security Challenges



- **Open-source code vulnerabilities**
  Firmware developed for IoT devices often includes open-source software, which is prone to bugs and vulnerabilities.
- **Overwhelming data volume**
  The amount of data generated by IoT devices make data oversight, management, and protection difficult.
- **Poor testing**
  Firmware developed for IoT devices often includes open-source software, which is prone to bugs and vulnerabilities.

IoT Security Challenges



- **Overwhelming data volume**
  The amount of data generated by IoT devices make data oversight, management, and protection difficult.
- **Unpatched vulnerabilities**
  Many IoT devices have unpatched vulnerabilities for many reasons, including patches not being available and difficulties accessing and installing patches.
- **Vulnerable APIs**
  APIs are often used as entry points to command-and-control centers from which attacks are launched, such as SQL injection, distributed denial of service (DDoS), man-in-the-middle (MITM), and breaching networks

IoT Security Challenges



- **Weak passwords**
  IoT devices are commonly shipped with default passwords that many users fail to change, giving cyber criminals easy access. In other cases, users create weak passwords that can be guessed.

Addressing these challenges and ensuring security in IoT products and services is a fundamental priority. Also, since IoT can drastically change the ways personal data is collected, analysed, used, and protected, privacy concerns have been raised.

These need to be addressed to ensure user trust and confidence in the Internet, connected devices, and related services.

Addressing IoT Security Challenges



To addressing these challenges, the **define of a set of Baseline Security Recommendations** for IoT is needed.

To do this, is mandatory to provide insight into the security requirements of IoT, mapping critical assets and relevant threats, assessing possible attacks and identifying potential good practices and security measures to apply in order to protect IoT systems.

It is there that **ENISA** comes in.

The ENISA



- The European Union Agency for Network and Information Security (**ENISA**) is a centre of network and information security expertise for the EU, its member states, the private sector and Europe's citizens.
- ENISA works with these groups to develop advice and recommendations on good practice in information security.
- ENISA seeks to enhance existing expertise in EU member states by supporting the development of cross-border communities committed to improving network and information security throughout the EU.
- For ENISA, **IoT is an emerging concept comprising a wide ecosystem of interconnected services and devices**.

The ENISA 's Scope

- ENISA identified and analysed existing IoT security practices, security guidelines, relevant industry standards and research initiatives in the area of IoT security for Critical Information Infrastructures.
- ENISA focused, among other topics, on IoT resilience and communication, on the interoperability with proprietary systems, and on the reliability of IoT.
- In 2017, ENISA launched the IoT Security Experts Group (**IoTSEC**) that is an information exchange platform that brings together experts to ensure the security and resilience of the entire Internet of Things ecosystem.

Threats and Risks related to IoT



The following are generic issues identified by this study that hinder the consolidation of secure IoT ecosystems:

- **Very large attack surface**
  At its basic level, an attack surface is the total number of entry points for unauthorized system access. An IoT attack surface goes beyond entry points and includes all possible security vulnerabilities for IoT devices, connected software and network connections.
- **Limited device resources**
  The majority of IoT devices have limited capabilities (processing, memory and power) and therefore advanced security controls cannot be effectively applied.

Threats and Risks related to IoT



- **Complex ecosystem**
  Security concerns are exacerbated since IoT should not be seen as a collection of independent devices, but rather as a rich, diverse and wide ecosystem involving aspects such as devices, communications, interfaces, and people.
- **Security Integration**
  Different IoT devices and systems may be based on different authentication solutions, which must be integrated and made interoperable
- **Safety aspects**
  They are very relevant in the IoT context because of the presence of actuators, **which act on the physical world**.

Threats and Risks related to IoT



- **Low Cost**
  Manufacturers might be inclined to limit security features to ensure a low cost and thus product security might not be able to protect against certain types of IoT attacks.
- **Lack of Expertise**
  This is a rather novel domain and therefore there is a lack of people with the suitable skills and expertise in IoT cybersecurity.
- **Security Updates**
  Applying security updates to IoT is extremely challenging, since the particularity of the user interfaces available to users does not allow traditional update mechanisms.

Threats and Risks related to IoT



- **Insecure Programming**
  Companies developing IoT products generally place more emphasis on functionality and usability than on security
- **Unclear Liabilities**
  The lack of a clear assignment of liabilities might lead to ambiguities and conflicts in case of a security incident, especially considering the large and complex supply chain involved in IoT.

For each IoT environment it is necessary to carry out a risk assessment to go through the threats that can affect the different assets, define the plausible attack scenarios, and put them in the context of the IoT service defined, working out which hazards are critical or not and which ones can be mitigated.

Examples of Attacks



- **Attack to the network link between controller(s) and actuators**
  Man in the middle attack types allow an attacker to extract sensitive and operational information that can be used for multiple malicious activities, including later attacks against IoT systems.
  - Impact: leakage of data;
  - Threats: leakage of sensitive data;
- **Attack to the sensors**
  The attacker manipulates the configuration of the sensors, changing the threshold values established on the sensors, to allow out-of-range read values to be accepted when they should not.
  - Impact: allowing sensors to report and accept incorrect values;
  - Threats: modification of information;

Examples of Attacks



- **Attack to the actuators**
  Manipulation of the actuators' configuration making them use wrong configurations, thresholds or data, and therefore affecting their normal behaviour.
  - Impact: It can affect production processes;
  - Threats: network outage by malicious devices;
- **Attack to the administration systems of IoT**
  An attacker tries to gain full control over the administration system of an IoT system or device, potentially compromising the whole environment.
  - Impact: manipulation or interruption of certain IoT systems;
  - Threats: weak passwords, malware and DDoS;

Examples of Attacks



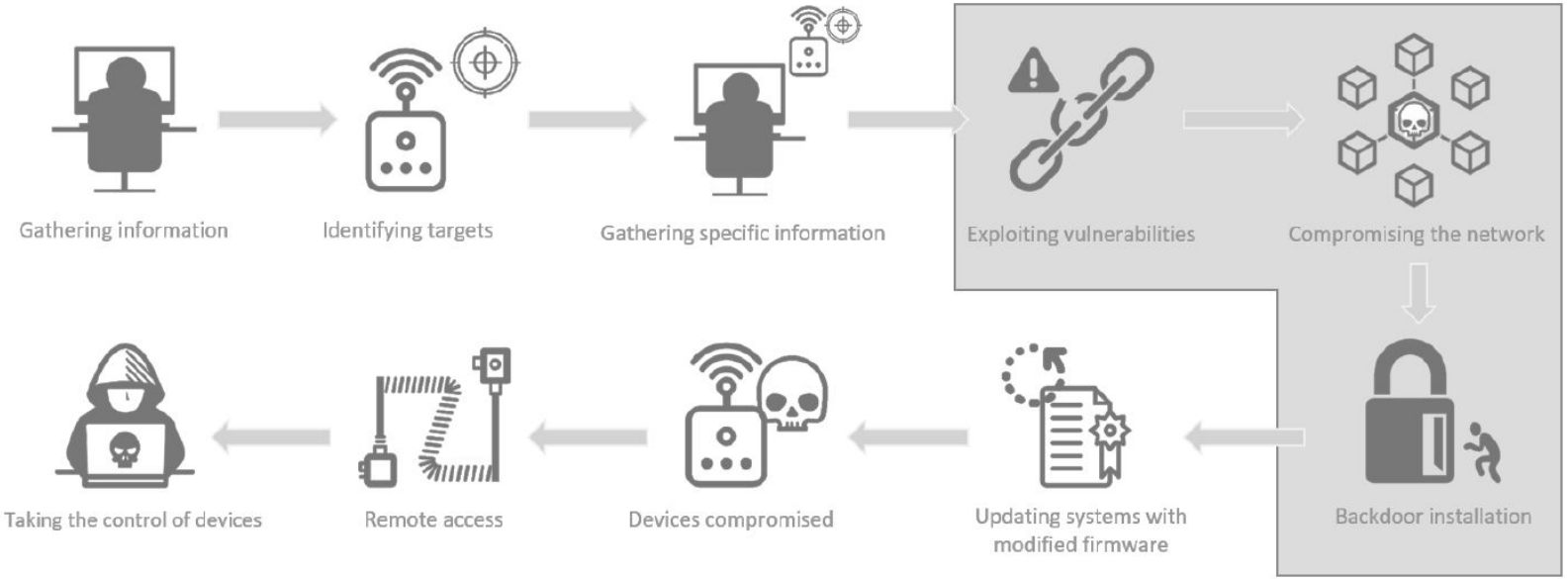
- **Attack to the protocols**
  This type of exploitation is usually the gateway to launch other types of attacks. Exploits are used to gain privileged unauthorised access to a system, which can lead to the installation of other malicious content or backdoors.
  - Impact: creates an entry point to a system;
  - Threats: : exploit kits and malwares;
- **Attack to the devices**
  This type of attack takes place when an attacker injects and executes commands with privileges in a compromised system.
  - Impact: produce a cascade effect on the system;
  - Threats: exploit kits, DDoS and network outage;

Examples of Attacks



- **Attack using IoT botnets**
  This type of attack does not target IoT devices themselves, but instead it uses them to attack other devices, not necessarily IoT ones.
  - Impact: the target device/service will be flooded with malicious traffic;
  - Threats: : exploit kits and counterfeit by malicious devices;
- **Attack to the Power source manipulation**
  These attacks focus on manipulating power sources and exploiting vulnerabilities to modify the power data read.
  - Impact: e device not being able to operate at all;
  - Threats: malware, physical attacks;

Attack Scenarios : IoT administration



Attack Scenarios : IoT administration



This attack covers an infection designed to take **control over one or multiple IoT devices** within an IoT environment, in order to manipulate or crash them and to be able to modify values, change their functioning/behaviour or deny access to them.

- Gather information in the network about the different IoT devices used in the enterprise and their vulnerabilities.
- Exploit the different vulnerabilities found in that device.
- Ensure the persistence of the access to the system by configuring a backdoor.
- Update the system with a modified firmware.
- Gain the ability to see all the data and information of the device and its network.

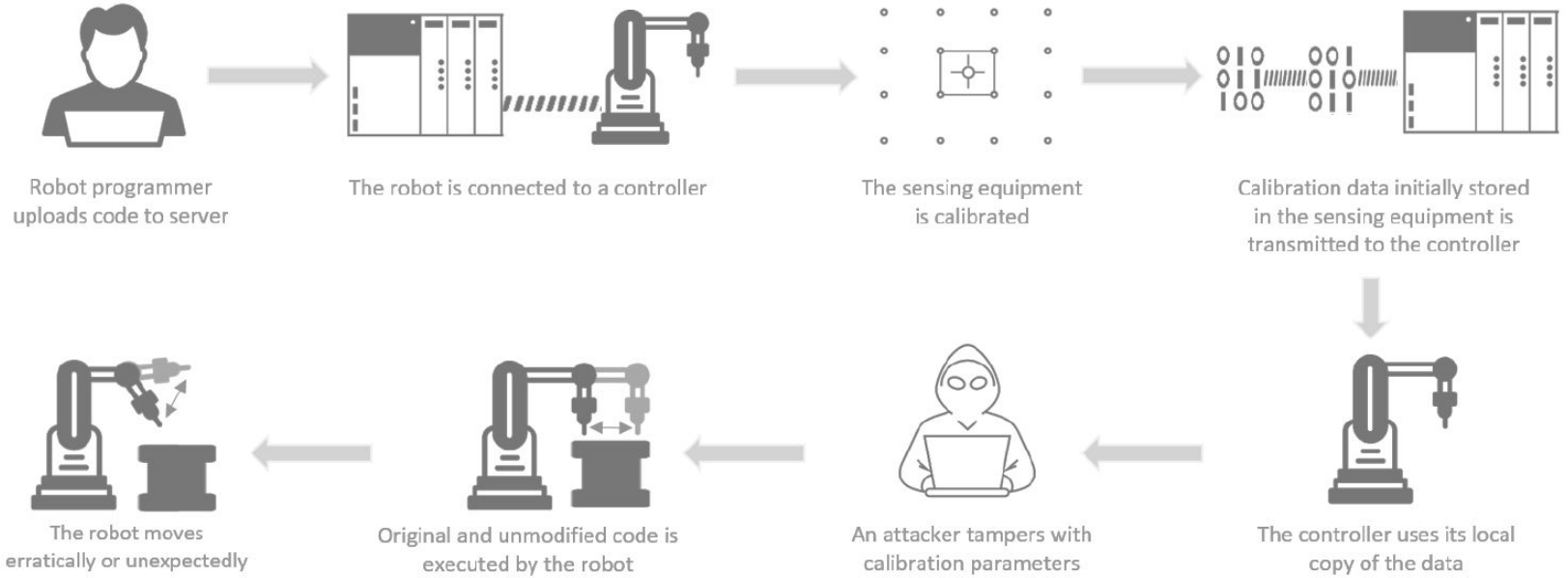
Attack Scenarios : IoT administration

- **Impact**
  The compromise of an IoT administration system grants the attacker access to all the assets (devices, networks, etc.) which fall under the control of that administration system.
- **Detection**
  The changes to an IoT administration system can be detected through a correct monitoring and a proper logging system.
- **Cascade Effect**
  The risk entailed is that, once an IoT device belonging to a specific IoT network is compromised, it becomes very easy to compromise the rest.

Attack Scenarios : IoT administration

- **Recovery Effort**
  It depends on the perimeter of the assets compromised and on the number of assets infected.
- **Challenges and Errors**
  Insecure design or development and lack of proper product lifecycle management.
- **Countermeasures**
  Control the installation of software on operational systems, enable a system to return to a state that is known to be secure, ensure default passwords and even default usernames are changed during the initial setup, limit the permissions of actions, implement a logging system and a regular monitoring system to verify the device behaviour.

Attack Scenarios : Value manipulation



Attack Scenarios : Value manipulation

The manipulation of calibration parameters established for the sensors allows undesired values to be accepted when they should not. This attack targets the sensor processing and knowledge model levels of the control system of an industrial robot in an Industry 4.0 environment.

- Calibration of a robot sensing equipment after a configuration change.
- Calibration data is transmitted to the controller during the system boot.
- An attacker remotely or locally tampers with calibration parameters.
- Original and unmodified code is executed by the robot.
- The robot moves erratically or unexpectedly because the true error is different from the error that the controller knows.

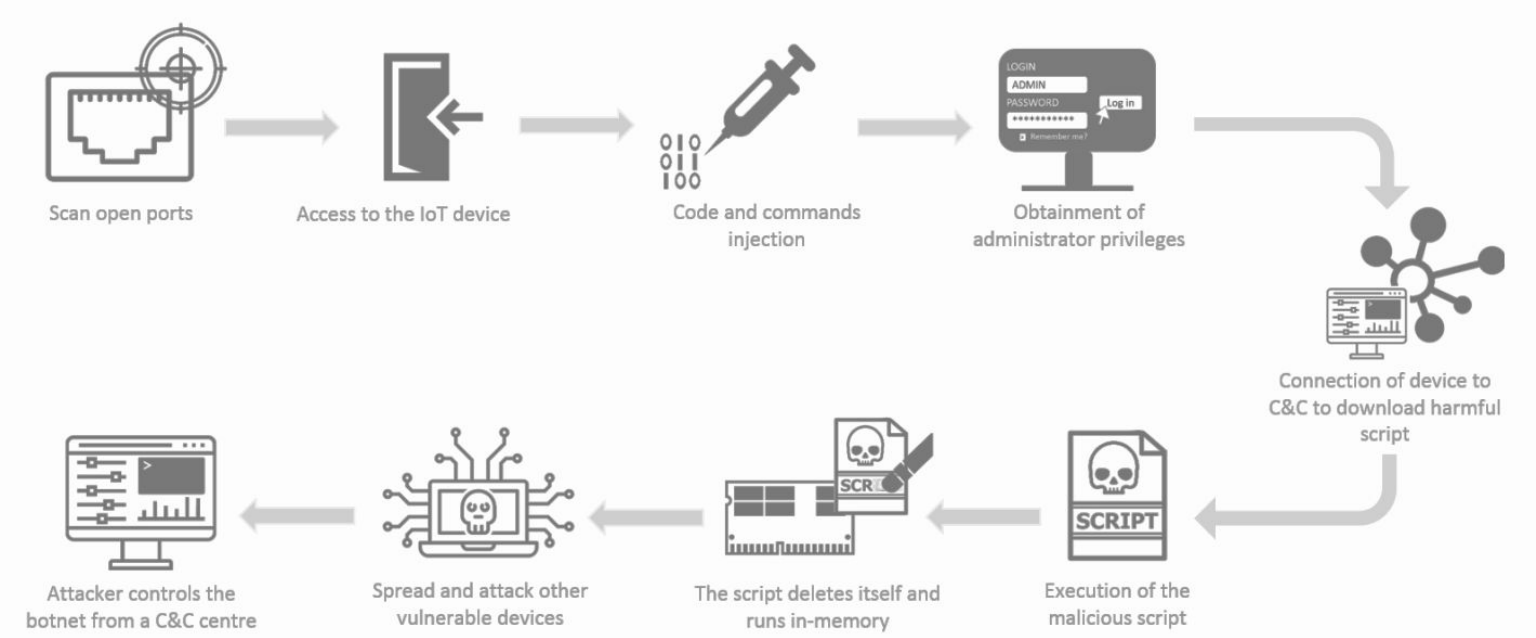
Attack Scenarios : Value manipulation

- **Impact**
  The IoT environment is put at risk. A malfunctioning industrial robot can cause severe physical damage to whatever it is working with, and in the worst case scenario, to the people working with it.
- **Detection**
  Its detection is between easy and medium since an operator can see whether the outcome and the robot's behaviour are correct or not.
- **Cascade Effect**
  It can vary depending on the number of sensors compromised in the robot, and on the number of robots involved.

Attack Scenarios : Value manipulation

- **Recovery Effort**
  Depending on the number of sensors, and the robots involved.
- **Challenges and Errors**
  Insecure design or development and lack of awareness and knowledge.
- **Countermeasures**
  Establish and maintain asset management procedures and configuration controls, design with system and operational disruption in mind, measures for tamper protection and detection, data input validation and output filtering, Implement regular monitoring to verify the device behaviour and train employees in good privacy and security practices.

Attack Scenarios : Commands injection



Attack Scenarios : Commands injection

This attack entails the exploitation of some vulnerability inside a device to inject commands and obtain administrator privileges, with the purpose of creating a botnet made up of those vulnerable IoT devices.

- Scanning open ports in IoT devices that are accessible over the Internet.
- Inject commands into the device's console in order to obtain administrator privileges.
- Make the device connect to a Command and Control under attacker control and download and execute a malicious script.
- The script will be executed, deleting itself afterward, running in-memory and begin to spread.

Attack Scenarios : Commands injection

- **Impact**
  The impact of the attacks carried out by a botnet ranges from high to critical, depending on the volume of the distributed attack.
- **Detection**
  These attacks tend to be hard to detect and identify the source, which allows them to pass undetected for long periods of time, and they are also complex to investigate and recover from.
- **Cascade Effect**
  This type of attack has a tremendous cascade effect. Once a device is infected, the goal is to identify other vulnerable devices to extend the network.

Attack Scenarios : Commands injection

- **Recovery Effort**
  The main issue is the amount of time it takes to detect that the system that has been manipulated.
- **Challenges and Errors**
  Insecure design or development and lack of proper product lifecycle management.
- **Countermeasures**
  Sign code cryptographically, implement run-time protection and secure execution monitoring, control the installation of software on operational systems, enable a system to return to a state that is known to be secure (recovery) and implement regular monitoring to verify the device behaviour.



Security Good Practices

Security measures and good practices have been defined with the aim to apply to the different IoT environments and deployments in a horizontal manner.

Security Good Practises



The different security measures and good practices identified fall into several security domains. This domain division's purpose is to cover every IoT environment horizontally.

- **Information System Security Governance & Risk Management**
  Risk analysis, policy, accreditation, indicators and audit, and human resource security.
- **Ecosystem Management**
  Ecosystem mapping and ecosystem relations.
- **IT Security Architecture**
  Systems configuration, asset management, system segregation, traffic filtering and cryptography.
- **IT Security Administration**
  Administration accounts and administration information systems.

Security Good Practises

- **Identity and access management**
  Authentication, identification and access rights.
- **IT security maintenance**
  IT security maintenance procedures and remote access.
- **Physical and environmental security**
  Installation and environment monitoring.
- **Detection**
  Logging and log correlation and analysis.
- **Computer security incident management**
  Information system security incident analysis.
- **Continuity of Operations**
  Business continuity management and disaster recovery management.
- **Crisis Management**
  Crisis management organization and process.

Security Good Practises



These security domains **have been considered when developing the different security measures and good practices for IoT**. These security domains classify the security measures based on which area of an IoT ecosystem they apply to. Apart from their area of application, each security measure can be arranged according to its nature.

They can be **policies** that must be taken into account when developing the devices, **organisational measures** focused on the business and employees that need to be adopted by the organisation itself, and finally, **technical measures** aimed at reducing the potential risks that the IoT devices and other elements of the IoT ecosystem may be subject to.

Security Policies



When it comes to IoT the cyber risk is context-dependent (i.e. based on the application scenario) and in this respect the security measures should be applied with this consideration in mind.

- **Security by design**
  Consider the security of the whole IoT system from a consistent and holistic approach during its whole lifecycle, ensure the ability to integrate different security policies and techniques and security must consider the **risk posed to human safety**. It is necessary to implement test plans to verify whether the product performs as it is expected. Also, it is important to conduct **code review during implementation** as it helps to reduce bugs in a final version of a product.

Security Policies



- **Privacy by design**
  Make privacy an integral part of the system and perform privacy impact assessments before any new applications are launched.
- **Asset Management**
  Establish and maintain asset management procedures and configuration controls for key network and information systems.
- **Risk and Threat Identification and Assessment**
  Identify significant risks using a defence-in-depth approach and the intended use and environment of a given IoT device.

Security Organisational Measures



- **End-of-life support**
  Develop an end-of-life strategy for IoT products.
- **Proven solutions**
  Use proven solutions recognized by the scientific community.
- **Management of security vulnerabilities**
  Establish procedures for analysing and handling security incidents and participate in information-sharing platforms to report vulnerabilities and receive timely and critical information about current cyber threats and vulnerabilities.
- **Human Resources Security Training**
  Ensure the personnel practices promote privacy and security.

Security Technical Measures



The security measures and good practices **need to consider and cover the technical elements**, in order to diminish the vulnerabilities of IoT. Applying these technical measures should take into account the particularities of the IoT ecosystem such as **scalability**, namely given the huge number of involved devices certain measures might need to be carried out at the level of specialised architectural components.

- **Hardware security**
  Employ a hardware-based immutable root of trust and use hardware that incorporates security features to strengthen the protection and integrity of the device.

Security Technical Measures

- **Trust and Integrity Management**
  Trust must be established in the boot environment before any trust in any other software or executable program can be claimed and sign code cryptographically to ensure it has not been tampered with after signing it as safe for the device, and implement run-time protection and secure execution monitoring to make sure malicious attacks do not overwrite code after it is loaded.
- **Strong default security and privacy**
  Any applicable security features should be enabled by default, and any unused or insecure functionalities should be disabled by default.
- **System safety and reliability**
  Design with system and operational disruption in mind, preventing the system from causing an unacceptable risk of injury or physical damage.

Security Technical Measures



- **Data protection and compliance**
  Personal data must be collected and processed fairly and lawfully, it should never be collected and processed without the data subject's consent, make sure that personal data is used for the specified purposes for which they were collected, and that any further processing of personal data is compatible and that the data subjects are well informed. Also, IoT stakeholders must be compliant with the **EU General Data Protection Regulation** (**GDPR**).
- **Secure Software / Firmware updates**
  Ensure that the device software/firmware, its configuration and its applications have the ability to update **Over-The-Air (OTA)**, that the update server is secure, that the update file is transmitted via a secure connection, that it does not contain sensitive data, that it is signed by an authorised trust entity and encrypted using accepted encryption methods.

Security Technical Measures



- **Authentication**
  Design the authentication and authorisation schemes and mechanisms must use strong passwords or personal identification numbers (PINs), and should consider using two-factor authentication (2FA) or multi-factor authentication (MFA) like Smartphones and Biometrics.
- **Authorisation**
  Limit the actions allowed for a given system by Implementing fine-grained authorisation mechanisms and using the **Principle of least privilege (POLP)**.
- **Physical and Environmental security**
  Data integrity and confidentiality must be enforced by access controls, the device cannot be easily disassembled and that the data storage medium is encrypted at rest and cannot be easily removed.

Security Technical Measures



- **Cryptography**
  Ensure a proper and effective use of cryptography to protect the confidentiality, authenticity and/or integrity of data and information in transit and in rest.
- **Secure and trusted communications**
  Guarantee the different security aspects as **privacy, integrity, availability and authenticity of the information in transit on the networks** or stored in the IoT application or in the Cloud. IoT devices should be restrictive rather than permissive in communicating.
- **Secure Interfaces and network services**
  Protocols should be designed to ensure that, if a single device is compromised, it does not affect the whole set, implement a DDoS-resistant and Load-Balancing infrastructure and ensure that web interfaces fully encrypt the user session, from the device to the backend services.

Security Technical Measures



- **Logging**
  Implement a logging system that records events relating to user authentication, management of accounts and access rights, modifications to security rules, and the functioning of the system..
- **Monitoring and Auditing**
  Implement regular monitoring to verify the device behaviour, to detect malware and to discover integrity errors. Conduct periodic audits and reviews of security controls to ensure that the controls are effective. Perform penetration tests at least biannually.

Security Recommendations



Security measures and good practices have been defined with the aim to apply to the different IoT environments and deployments in a horizontal manner.

Security Recommendations

- **Promote harmonization of IoT security initiatives and regulations**
  The current fragmentation of IoT security guidelines, initiatives, standards and other schemes **needs to be addressed**. A first and solid step in the direction is to define a list of best practices and guidelines for IoT security and privacy, which can be used as a baseline for the development and deployment of IoT systems in the market.
  The recommendation to counter this fragmentation refers to establishing a set of practices, guidelines and security requirements in IoT, which are common over Europe. Subsequently, each sector can focus on defining the specific sets of practices, guidelines, requirements for its own needs based on the particular context and risk factors inherent in each sector.
  The procurement process is another means to impose harmonization of baseline standards and requirements for IoT systems. The harmonization should consider that there are many different sectors.

Security Recommendations

- **Raise awareness for the need for IoT cybersecurity**
  Cybersecurity is a shared responsibility among all involved stakeholders. It is thus essential for these stakeholders to have a thorough understanding of related risks and threats, as well as ways to secure and protect against them. Raising awareness is therefore of paramount importance and initiatives to do so are highly recommended. As evidenced by the growing threat landscape and the numerous security incidents concerning IoT, **there is lack of knowledge present within IoT developers, industries as well as end users and consumers**.
  - Security education and training needs to be established in industries.
  - End users and consumers have to be educated to be able to make informed decisions when buying IoT devices and systems.
  - Corporate trainings focused on IoT security are also beneficial and should be pursued.

Security Recommendations

- **Define secure software/hardware development lifecycle guidelines for IoT**
  Developers, manufacturers and providers of IoT products and solutions **should integrate and adopt a secure software development lifecycle (SSDLC)** for their IoT offerings and incorporate relevant processes in their operations. Security must be implemented as a whole, at the application level, and in each of the phases of the SDLC. It is therefore important to encourage more companies to offer secure components that are at the same time usable for developers and end users/consumers.
  The notions of **security and privacy by default** and **security and privacy by design** naturally emerge as being foundation cornerstones of IoT security. It is challenging to apply these concepts in several different environments that will have particular characteristics. As far as developers are concerned, **secure by design hackathons and use of best practice cookbooks for IoT security can greatly enhance their perception of using principle of security and privacy by default and by design**.

Security Recommendations

- **Achieve consensus for interoperability across the IoT ecosystem**
  The issue of interoperability is very pertinent to the IoT ecosystem due to the very **large scale and penetration of the IoT ecosystem**, the long and complex supply chains and the numerous involved stakeholders. Ensuring and fostering interoperability of IoT devices, platforms and frameworks, as well as security practices is therefore an essential element of IoT security and should thus be encouraged.

  - **Encourage the use of open interoperability frameworks that incorporate security**
  - **Provide transparency on the security of interoperability frameworks**
  - **Promote open and accessible interoperability laboratories and testbeds for security**

Security Recommendations

- **Foster economic and administrative incentives for IoT security**
  It is clear that **lack of security impacts business continuity** and this is indeed the case also for IoT **that is driven by R&D activities** and a rush to push products and services in the market, that demands on cybersecurity are somewhat low because of the lack of consumer perception in the added value of cybersecurity. Consumer involvement is quite important and it should be supported more. Communication campaigns should be implemented by the government in order to increase and sustain said perception and thus inherently necessitate the adoption of further mechanisms to promote IoT cybersecurity.

  In the case of IoT, competitive advantage is currently placed and focussed on the time to market rather than secure to market. This balance should be shifted so that a specific level of security and privacy before market deployment is encouraged. Defining security frameworks supported by baseline security measures can be a way forward in this direction. Use of other schemes such as certification and labelling can also encourage better understanding and transparency in terms of IoT security.

Security Recommendations

- **Establishment of secure IoT product/service lifecycle management**
  Security plays an important role within all the phases of an IoT product's/service's lifecycle. These phases include design, development, testing, production, deployment, maintenance, end-of-support, and end-of life. It is recommended that specific, focussed and targeted security processes be defined for all these phases. Furthermore, security processes have to be properly implemented. In order to satisfy this need, fundamental security requirements and building blocks have to be specified to be available within each phase.

  A noteworthy aspect involves security updates that constitute a significant issue in the context of IoT. After deployment, **security updates need to be provided where practically possible without special knowledge requirements or financial obligations on the end user/consumer within a defined term and conditions until "end-of-support"**. The latter must be clearly defined by the manufacturer/provider of the IoT product and must be clearly communicated to the end user/consumer.

Security Recommendations

- **Establishment of secure IoT product/service lifecycle management**
  Security plays an important role within all the phases of an IoT product's/service's lifecycle. These phases include design, development, testing, production, deployment, maintenance, end-of-support, and end-of life. It is recommended that specific, focussed and targeted security processes be defined for all these phases. Furthermore, security processes have to be properly implemented. In order to satisfy this need, fundamental security requirements and building blocks have to be specified to be available within each phase.

  A noteworthy aspect involves security updates that constitute a significant issue in the context of IoT. After deployment, **security updates need to be provided where practically possible without special knowledge requirements or financial obligations on the end user/consumer within a defined term and conditions until "end-of-support"**. The latter must be clearly defined by the manufacturer/provider of the IoT product and must be clearly communicated to the end user/consumer.

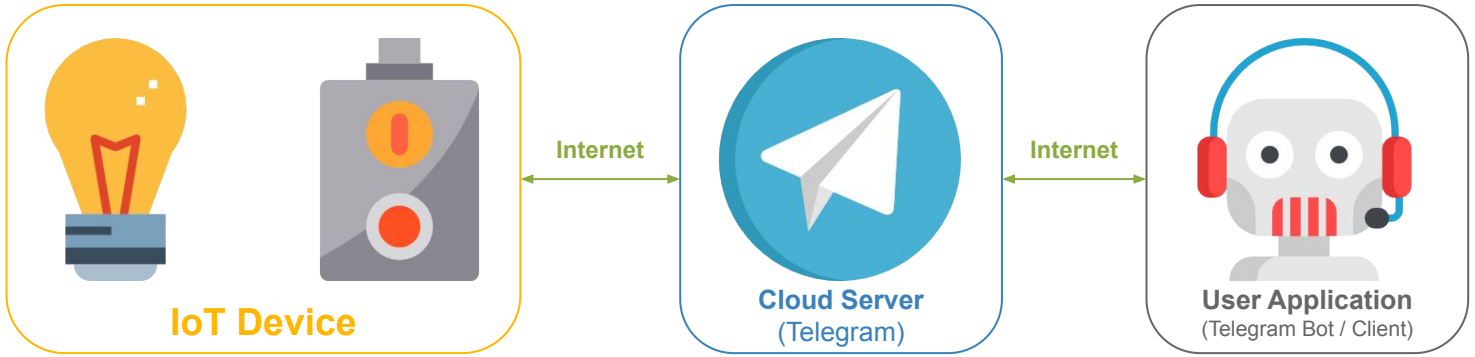
5

Let’s make an IoT device!

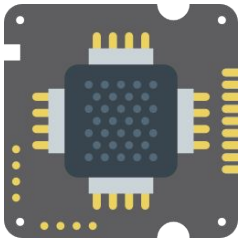
Yes we are going to do something!

The Project of our IoT device

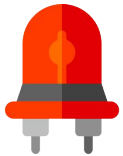
Our project will be similar to a Smartlight. The device must be able to connect to the Internet independently and register itself with the Telegram service and to be able to communicate through a Bot, created specifically, to manage the status of the light bulb of our device.



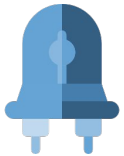
Our IoT device in detail



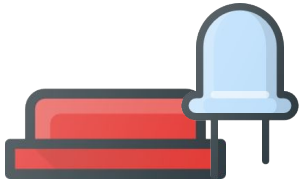
ESP32



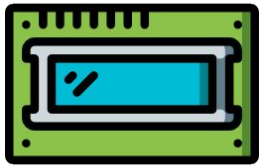
Red Led



Blue Led



Button with Led



OLED I²C

Our device, in addition to the ESP8266, will be equipped with two LEDs, one red (which we will use for feedbacks) and one Blue (that is our light bulb).

We will use a button to control locally the Blue Led (and then send notifications) equipped with a Led that will turn on or not depending on the status of the Blue Led.

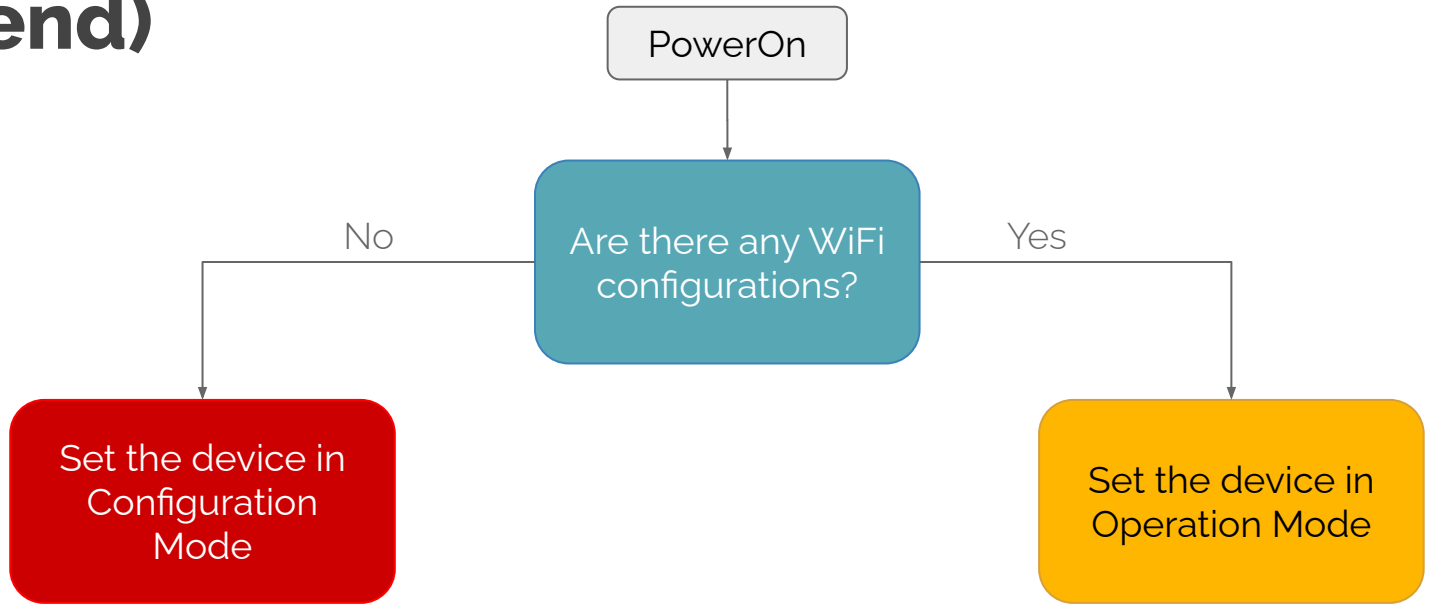
Finally, (optionally) we will use an OLED display to show some feedbacks.

Some libraries that will be used

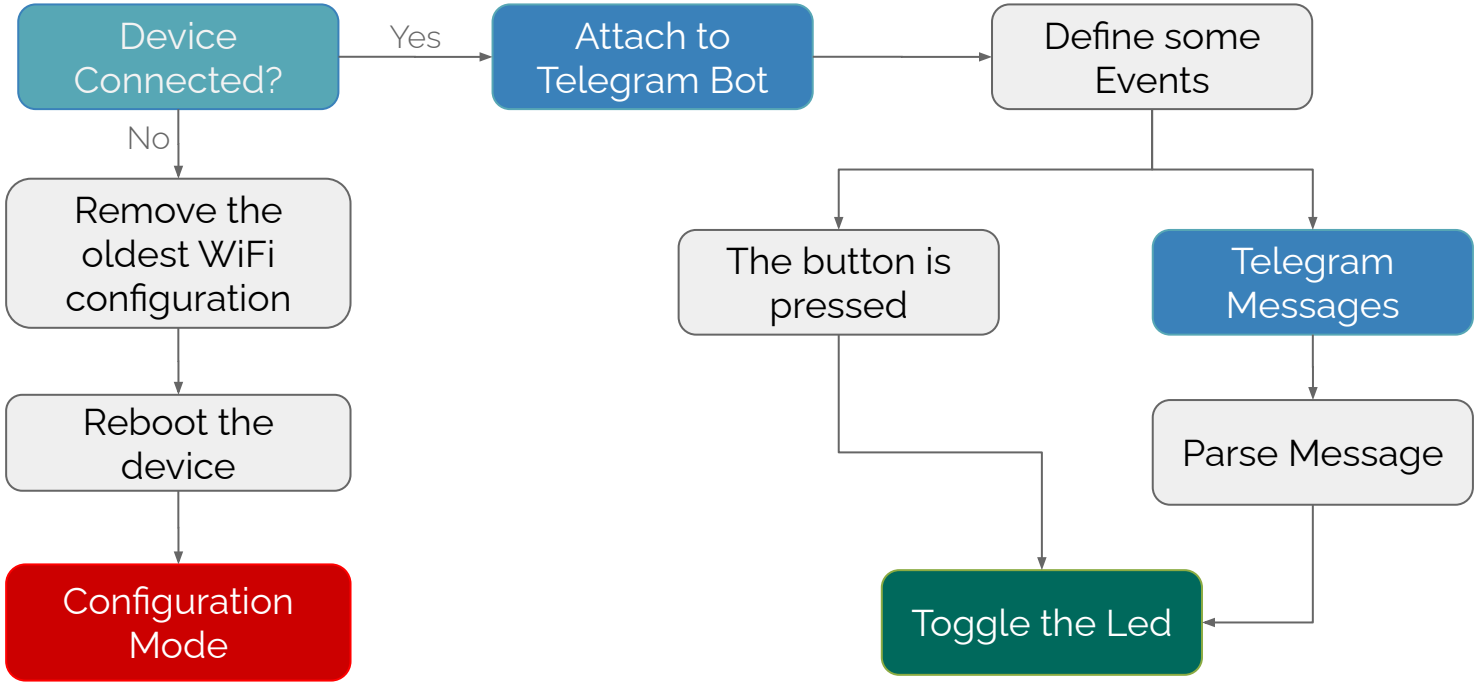


- **Arduino** - Our SDK
- **ESP8266WiFi** - To manage the ESP8266's Wi-Fi
- **ESP8266HTTPClient** - An HTTP Client to make requests
- **Adafruit_SSD1306** - To manage the OLED Display
- **WiFiClientSecure** - To improve the HTTP client with SSL (HTTPS)
- **ArduinoJson** - To manage the JSON structure
- **TelegramBot** - To make the required HTTPs request to Telegram
- **ESP8266WebServer** - To serve an Web server to configure our device
- **WiFiManager** - To manage the configuration of our connection

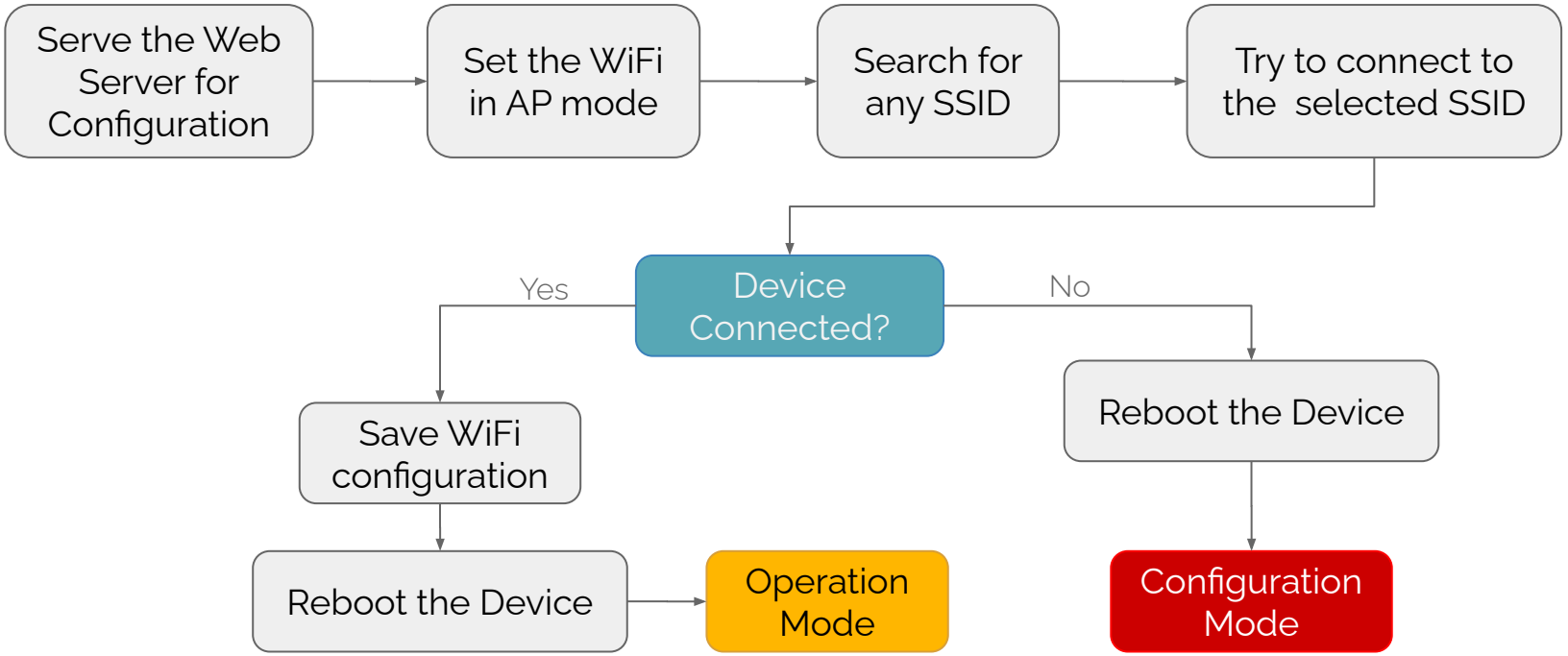
The logic of our device (at the end)



The Operation Mode



The Configuration Mode



The code structure (Arduino)



```
#include <Arduino.h>
/* LIBRARIES */

/* VARS DEFINITIONS */

/* FUNCTIONS DEFINITIONS */

void setup()

void loop()

/* FUNCTIONS */
```

The code of our device is base on Arduino code.

There are two main functions:

- **Setup**
  This one is called once at the beginning of our device on device boot. Typically, in this function the various pins and variables will be define.
- **Loop**
  The main loop of our code.

Let's blink leds !



```
#define LED D0
#define BUTTON D6
#define LED_RED D4
#define LED_BLUE D5

int interruptButtonFlag = 0;

void lightOn();
void lightOff();
void lightToggle();
void callbackButton();
void callbackButtonRelease();
```

Let's define some basic functions and the digital pins

- **lightOn**
  Set the digital pin LED_BLUE to 1
  Set the digital pin LED to 0
- **lightOff**
  Set the digital pin LED_BLUE to 0
  Set the digital pin LED to 1
- **lightToggle**
  Invert the value of digital pins LED and LED_BLUE
- **callbackButton** and **callbackButtonRelease**
  These will be called when the button is pressed and released

Let's blink leds !



```
void setup()
{
   Serial.begin(115200);
   pinMode(LED, OUTPUT);          // LED pin as output
   pinMode(LED_RED, OUTPUT);      // LED pin as output
   pinMode(LED_BLUE, OUTPUT);     // LED pin as output
   pinMode(BUTTON, INPUT_PULLUP); // Button pin as input

   digitalWrite(LED_RED, 1);  // turn on the red led on powered on
   digitalWrite(LED_BLUE, 1); // turn on the blue led on powered on

   attachInterrupt(BUTTON, callbackButton, RISING); // attach the interrupt event on the Button pin
      . . . . . .
```

Let's blink leds !



```
//setup - optional if you have the OLED screen
if (WITH_OLED_DISPLAY)
   {
       display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
       // Clear the buffer.
       display.clearDisplay();
       display.display();
       display.setTextSize(1);
       display.setTextColor(INVERSE);
       display.setCursor(0, 0);
       display.println("Booting...");
       display.display();
   }
```

Let's blink leds !



```
    //setup - then turn off all leds
    delay(100);

    // turn off all LEDs
    digitalWrite(LED_RED, 0);
    digitalWrite(LED_BLUE, 0);
    digitalWrite(LED, 0);
}
```

Let's blink leds !



```
    //setup - then turn off all leds
    delay(100);

    // turn off all LEDs
    digitalWrite(LED_RED, 0);
    digitalWrite(LED_BLUE, 0);
    digitalWrite(LED, 0);
}

void loop()
{
    lightToggle();
    delay(1000);

}
```

Manage the button !



```
void callbackButton()
{
   // we need to detach the press interrupt event
   detachInterrupt(BUTTON);
   // then attach the release interrupt event
   attachInterrupt(BUTTON, callbackButtonRelease, FALLING);
}

void callbackButtonRelease()
{
   detachInterrupt(BUTTON);
   // so we set the interrupt button flag to 1
   interruptButtonFlag = 1;
}
```

Manage the button !



```
void loop()
{
   if(interruptButtonFlag)
   {
       // the interrupt flag must be resetted
       interruptButtonFlag = 0;
       lightToggle();
       // then re-attach the interrupt pressed event
       attachInterrupt(BUTTON, callbackButton, RISING);
   }

}
```

Let's manage the Wi-Fi !



```
// VARS
// WiFi Events
WiFiEventHandler gotIpEventHandler, disconnectedEventHandler;

// WiFi vars
WiFiManager wifiManager;      // the wifi manager
WiFiClientSecure net_ssl;     // the ssl plugin to made HTTPS requests
DynamicJsonBuffer jsonBuffer; // to read and write some JSON format strings

/* FUNCTIONS DEFINITIONS */
void wifiConnect();  // try to connect and then set the device mode related to the connection status
void wifiConnect(const char *ssid, const char *password); // try to connect with saved configuration
void configModeCallback(WiFiManager *myWiFiManager);    // if the device goes into the Conf. mode
void saveConfigCallback();      // this function is used to save the wifi configuration
```

Let's manage the Wi-Fi !



```
// setup....
wifiManager.setAPCallback(configModeCallback);  // It is called when the device is in Conf. Mode
wifiManager.setSaveConfigCallback(saveConfigCallback); // It is called when the device saves the WiFi configurations
// the callback that is called when the device gets an IP address, so in Operation Mode
gotIpEventHandler = WiFi.onStationModeGotIP([](const WiFiEventStationModeGotIP &event) {
       Serial.println("Connected");
       Serial.println("Device connected:");
       Serial.println(WiFi.localIP());
       digitalWrite(LED_BLUE, 1);
});

wifiConnect();

// setup....
```

setAPCallback is called when the device enters in Access Point mode, so in Configuration Mode.

When the WiFiManager library confirms that settings are ok, then it calls the **saveConfigCallback** function.

Finally, when the device gets an IP which means that the device is connected to the Gateway, so it will enter in **Operation Mode**.

Let's manage the Wi-Fi !



```
void configModeCallback(WiFiManager *myWiFiManager)
{
   if (WITH_OLED_DISPLAY)
   {
       emptyDisplay();
       display.println("= Configuration Mode =");
       display.println(WiFi.softAPIP());
       display.println(myWiFiManager->getConfigPortalSSID());
       display.display();
   }
   digitalWrite(LED_RED, 1);
   status = CONFIGURATION;
}
```

Let's manage the Wi-Fi !



```
void wifiConnect()
{
   // check if the filesystem is loaded
   if (SPIFFS.begin())
   {
       // check if configuration file exits
       if (SPIFFS.exists("/config.json"))
       {
           // try to connect.....
       }
       else
       {
           if (WITH_OLED_DISPLAY)
           {
               display.println("Cannot connect");
               display.display();
           }
           // enter in configuration mode
           wifiManager.resetSettings();
           delay(1000);
           wifiManager.autoConnect("IoT-Device");
       }
   }
   else
   {
       Serial.println("failed to mount FS");
       digitalWrite(LED_RED, 1);
       return;
   }
}
```

The main logic of the **wifiConnect** function is that if some configuration file (config.json) exists, then it tries to connect with that configuration.

If there is not a configuration file, then the device is set in Configuration mode.

When the device is in Configuration Mode, it will set the Wi-Fi in Access Mode and serve a "Captive Portal" to configure the device.

All these operations are managed by the **WiFiManager** library:

resetSettings will delete all configurations saved in memory and **autoConnect** will launch the access point mode.

Let's manage the Wi-Fi !



```
//FILE EXISTS
File configFile = SPIFFS.open("/config.json", "r");
if (configFile)
{
    if (WITH_OLED_DISPLAY)
    {
        display.println("Conf. loaded");
        display.display();
    }
    size_t size = configFile.size();
    // Allocate a buffer to store contents of the file.
    std::unique_ptr<char[]> buf(new char[size]);

    configFile.readBytes(buf.get(), size);
    DynamicJsonBuffer jsonBuffer;
    JsonObject &json = jsonBuffer.parseObject(buf.get());
    // json.printTo(Serial);
    const char *ssidp = json["ssid"];
    const char *passwordp = json["password"];
    if (json.success())
    {

        if (WITH_OLED_DISPLAY)
        {
            display.println("Connecting to:");
            display.println(ssidp);
            display.display();
        }

       // ----->
        wifiConnect(ssidp, passwordp);
    }
    else
    {
        Serial.println("failed to load json config");
        digitalWrite(LED_RED, 1);
        return;
    }
}
```

Let's manage the Wi-Fi !



```
void wifiConnect(const char *ssid, const char *password)
{
   if (!WITH_OLED_DISPLAY)
   {
       display.println("Connecting to:\n");
       display.printf("%s\n", ssid);
       display.println();
       display.display();
   }
   delay(100);
   WiFi.begin(ssid, password);
   int perc = 0;
   while (WiFi.status() != WL_CONNECTED)
   {
       led_blink ^= led_blink;
       delay(500);
       digitalWrite(LED, !digitalRead(LED));
       perc += 10;
       if (perc > 100)
           perc = 0;
       drawProgressBar(display.getCursorX(), display.getCursorY(), 126, 10,
perc);
   }
   emptyDisplay();
   display.println("= Operation Mode =");
   display.println("Device connected:");
   display.println(WiFi.localIP());
   display.display();
   status = OPERATION;
}
```

Finally, our device will try to connect with the Wi-Fi settings saved before.

If connection is successful, then the device enters in **Operation Mode**.

Let's manage the Wi-Fi !



```
void saveConfigCallback()
{
   if (WITH_OLED_DISPLAY)
   {
       display.println("saving config");
       display.display();
   }
   DynamicJsonBuffer jsonBuffer;
   JsonObject &json = jsonBuffer.createObject();
   json["ssid"] = wifiManager.getSSID();
   json["password"] = wifiManager.getPassword();

   File configFile = SPIFFS.open("/config.json", "w");
   if (!configFile)
   {
       Serial.println("failed to open config file for writing");
       digitalWrite(LED_RED, 1);
       return;
   }
   json.printTo(configFile);
   configFile.close();
   if (WITH_OLED_DISPLAY)
   {
       display.println("Saved! Rebooting...");
       display.display();
   }
   delay(2000);
   ESP.reset();
}
```

When the device is connected for the first time, it will save the information in its filesystem, such as JSON file.

Then, after saving, the device will be rebooted.

If you want to try to connect and then remove the file, you could use the SPIFFS.format() function in setup.

7

Let’s set up the Telegram bot!

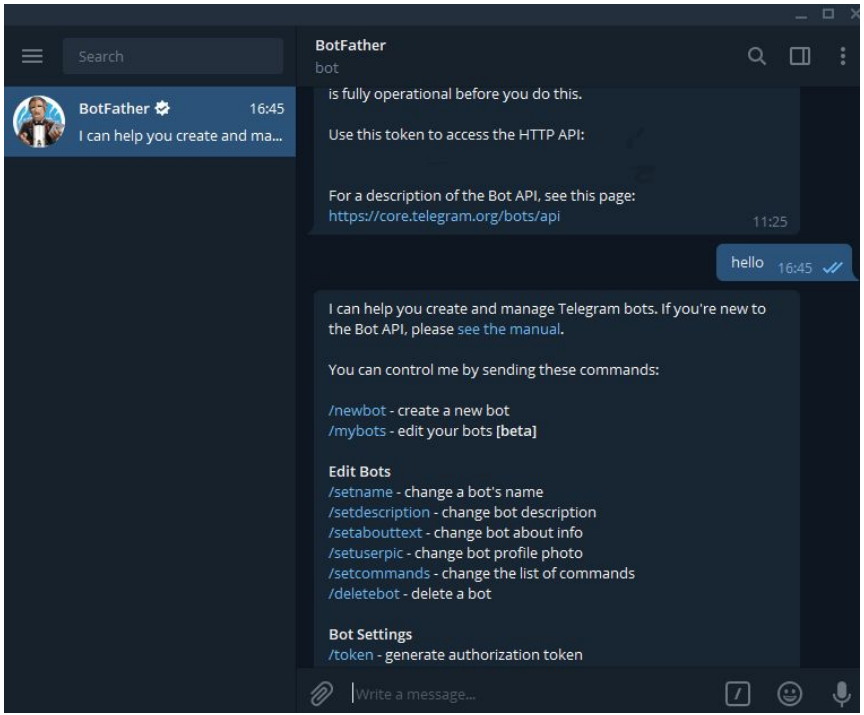
“Hello sir!”, BotFather says

Telegram bots are third-party application that run inside Telegram

You can develop your own Telegram Bot to do everything, for instance: notify when there are some updates or to send some commands to your services.



How to create a Telegram Bot ?



You need to search **BotFather** contact and then send any message to this Bot.

It will response with a Help message.

To create a new bot you must send the **/newbot** command and response to the questions.
Finally, you get the contact of your bot and the secret to use the HTTP API with your Bot.

Let's manage the Telegram Bot !



```
// VARS
// Telegram Vars
const char BotToken[] = ""; // token of the bot
int Bot_mtbs = 200;       // mean time between scan messages
long Bot_lasttime;         // last time messages' scan done
int bulk_messages_mtbs = 1500;        // mean time between send messages, 1.5 seconds
int messages_limit_per_second = 25;    // telegram API has limit for bulk messages ~30 messages per second
UniversalTelegramBot bot(BotToken, net_ssl);    // the telegram bot


/* FUNCTIONS DEFINITIONS */
void handleNewMessages(telegramMessage m);  // check if bot has any new messages
```

Let's manage the Telegram Bot !



```
void handleNewMessages(telegramMessage m)
{
   String chat_id = m.chat_id; // the id of sender
   String text = m.text; // the received message
   String from_name = m.from_name;       //nickname of sender
   if (from_name == "")
       from_name = "Guest";

   if (WITH_OLED_DISPLAY)
   {
       display.display();
       display.setCursor(0, 0);
       display.clearDisplay();
       display.printf("%s:\n", from_name.c_str());
       display.printf("%s\n", text.c_str());
       display.display();
   }

   // MANAGE COMMANDS

}
```

This is our **parser** for the incoming messages from the Telegram Bot.

From here we can recognize who the sender is, what he sent and we can send a response to the sender.

We will manage the message from the user after

Let's manage the Telegram Bot !



```
//LOOP

else if (!interruptButtonFlag && millis() > Bot_lasttime + Bot_mtbs)
{
   int numNewMessages = bot.getUpdates(bot.last_message_received + 1);

     while (numNewMessages)
     {
       for (int i = 0; i < numNewMessages; i++)
       {
          handleNewMessages(bot.messages[i]);
       }
       numNewMessages = bot.getUpdates(bot.last_message_received + 1);
      }
     Bot_lasttime = millis();
}
```

In the main loop we need to query Telegram service if our bot has any new messages.

If there are some new messages, we use the **handleNewMessages** to parse them.

Let's try if our bot can receive the messages!

Let's manage the Telegram Bot !



```
if (text == "/on")
   {
       lightOn();
       bot.sendMessage(chat_id, "Led On");

   } else if(text == "/off")
   {
       lightOff();
       bot.sendMessage(chat_id, "Led Off");
   }
   else if(text == "/status")
   {
       String statusMessage;
       if(digitalRead(LED_BLUE)) statusMessage = "The led in On";
       else statusMessage = "The led in Off";
       bot.sendMessage(chat_id, statusMessage);
   }
   else if (text == "/options")
   {
     String keyboardJson = "[[\"/on\", \"/off\"],[\"/status\"]]";
     bot.sendMessageWithReplyKeyboard(chat_id, "Choose from one of the following options", "",
keyboardJson, true);
   }
   // ---->
```

```
else
{
    String welcome = "Welcome to IoT Device, " + from_name + ".\n";
    welcome += "/ledon : to switch the Led ON\n";
    welcome += "/ledoff : to switch the Led OFF\n";
    welcome += "/status : Returns current status of LED\n";
    welcome += "/options : returns the reply keyboard\n";
    bot.sendMessage(chat_id, welcome, "Markdown");
}
```

In our case we want to manage general messages with a response that is a list of commands of our bot and manage these commands in our device.